

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
дисциплины
«Основы нейронных сетей»
Вариант № 14

Выполнила:
Текеева Мадина Азрет-Алиевна
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии Воронкин Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Изучение архитектуры и принципов работы рекуррентных и одномерных сверточных нейронных сетей для задач обработки естественного языка.

Цель работы: освоение теоретических основ и приобретение практических навыков построения нейронных сетей для классификации текстов на основе моделей Bag of Words, Embedding, рекуррентных и одномерных свёрточных архитектур.

Репозиторий: https://github.com/bickrosss/NN_lab4

Порядок выполнения работы:

Задание 1. Создать нейронную сеть, распознающую авторство текстов шести русских писателей (Булгаков, Рэй Брэдберри, Стругацкие, Макс Фрай, Клиффорд Саймак, О. Генри). Используя подготовленный корпус из обучающей и тестовой выборки, выполнить токенизацию, формирование сбалансированных выборок методом скользящего окна и обучить серию нейронных сетей с различными архитектурами: модели на основе Bag of Words с полносвязными слоями, рекуррентные модели на базе LSTM и одномерные свёрточные модели Conv1D. Функция ошибки – категориальная кроссэнтропия, оптимизатор – Adam, размер батча – 128, количество эпох – 20.

Вывести графики обучения (доли верных ответов и функции ошибки на обучающей и проверочной выборках), матрицы ошибок и итоговые значения средней точности распознавания авторов для каждой модели.

1.1 Реализация средствами библиотеки Keras.

1.1.1 Подключение необходимых для работы библиотек.

Для решения задачи распознавания авторства текстов используется набор инструментов библиотек tensorflow.keras и scikit-learn. Подключаются модули для работы с массивами (numpy), категориальными данными (utils), построения последовательной модели (Sequential) и базовых слоёв нейронных сетей (Dense, Dropout, SpatialDropout1D, BatchNormalization, Embedding,

Flatten, Activation). Для преобразования текстов в последовательности используется Tokenizer, для оценки качества классификации – confusion_matrix и ConfusionMatrixDisplay. Дополнительно подключаются библиотеки для загрузки датасетов из облачного хранилища (gdown), работы с операционной системой (os), временем (time), регулярными выражениями (re) и отрисовки графиков (matplotlib.pyplot).

Импорт библиотек

```
[ ] # Работа с массивами данных
import numpy as np

# Функции-утилиты для работы с категориальными данными
from tensorflow.keras import utils

# Класс для конструирования последовательной модели нейронной сети
from tensorflow.keras.models import Sequential

# Основные слои
from tensorflow.keras.layers import Dense, Dropout, SpatialDropout1D, BatchNormalization, Embedding, Flatten, Activation

# Токенизатор для преобразования текстов в последовательности
from tensorflow.keras.preprocessing.text import Tokenizer

# Матрица ошибок классификатора
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Загрузка датасетов из облака google
import gdown

# Функции операционной системы
import os

# Работа со временем
import time

# Регулярные выражения
import re

# Запись в файлы и чтение из файлов структур данных Python
import pickle

# Отрисовка графиков
import matplotlib.pyplot as plt

%matplotlib inline
```

Рисунок 1.1.1 – Подключение библиотек

1.1.2 Загрузка датасета и просмотр состава корпуса.

Корпус текстов 20writers (адаптированный для шести классов) загружается с облачного хранилища УИИ при помощи gdown. После скачивания архив writers.zip распаковывается в директорию writers/, и выводится список содержащихся в ней файлов. В корпусе представлено по две группы файлов на каждого автора: с признаком «Обучающая» в имени и с признаком «Тестовая». Это даёт возможность готового разделения текстов на обучающую и тестовую выборки без дополнительной обработки.

✓ Загрузка датасета и подготовка данных

```
[ ] # Загрузка датасета из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/17/writers.zip', None, quiet=True)

✓ 'writers.zip'

[ ] # Распаковка архива в папку writers
!unzip -qo writers.zip -d writers/

# Просмотр содержимого папки
!ls writers

✓ '(Булгаков) Обучающая_5 вместе.txt'
'(Булгаков) Тестовая_2 вместе.txt'
'(Клиффорд_Саймак) Обучающая_5 вместе.txt'
'(Клиффорд_Саймак) Тестовая_2 вместе.txt'
'(Макс Фрай) Обучающая_5 вместе.txt'
'(Макс Фрай) Тестовая_2 вместе.txt'
'(О. Генри) Обучающая_50 вместе.txt'
'(О. Генри) Тестовая_20 вместе.txt'
'(Рэй Брэдберри) Обучающая_22 вместе.txt'
'(Рэй Брэдберри) Тестовая_8 вместе.txt'
'(Стругацкие) Обучающая_5 вместе.txt'
'(Стругацкие) Тестовая_2 вместе.txt'

[ ] # Настройка констант для загрузки данных
FILE_DIR = 'writers'
SIG_TRAIN = 'обучающая'
SIG_TEST = 'тестовая'
```

Рисунок 1.1.2 – Загрузка и распаковка датасета writers.zip

1.1.3 Настройка констант и подготовка структур для загрузки

текстов.

Перед загрузкой текстов задаются константы: `FILE_DIR` – имя директории с файлами корпуса, `SIG_TRAIN` и `SIG_TEST` – строки-маркеры обучающей и тестовой выборки, по которым в имени файла определяется его принадлежность.

Листинг 1.1.1 – Настройка констант для загрузки данных

```
FILE_DIR = 'writers'
SIG_TRAIN = 'обучающая'
SIG_TEST = 'тестовая'
```

1.1.4 Загрузка текстов и формирование классов.

Загрузка текстов реализована единым циклом по всем файлам директории. Для каждого файла регулярным выражением извлекаются имя класса (имя автора в круглых скобках) и тип выборки (Обучающая / Тестовая). Если автор встречается впервые, он добавляется в общий список `CLASS_LIST`, и для него создаются пустые строки в `text_train` и `text_test`. Затем содержимое

файла читается и дописывается в соответствующую выборку нужного класса с заменой переводов строк на пробелы. Такой подход устойчив к произвольному порядку файлов в директории и к наличию нескольких файлов на один класс. По окончании цикла подсчитывается итоговое количество классов CLASS_COUNT, равное 6, что соответствует условию задачи.

```
[ ] # Загрузка датасета
CLASS_LIST = []
text_train = []
text_test = []

for file_name in os.listdir(FILE_DIR):
    m = re.match(r'\\((.+?)\\) (\\S+)_', file_name)
    if m:
        class_name = m[1]
        subset_name = m[2].lower()
        is_train = SIG_TRAIN in subset_name
        is_test = SIG_TEST in subset_name

        if is_train or is_test:
            if class_name not in CLASS_LIST:
                print(f'Добавление класса "{class_name}"')
                CLASS_LIST.append(class_name)
                text_train.append('')
                text_test.append('')

            cls = CLASS_LIST.index(class_name)
            print(f'Добавление файла "{file_name}" в класс "{CLASS_LIST[cls]}"', {subset_name} выборка.')
            with open(f'{FILE_DIR}/{file_name}', 'r') as f:
                text = f.read()
                subset = text_train if is_train else text_test
                subset[cls] += ' ' + text.replace('\n', ' ')

CLASS_COUNT = len(CLASS_LIST)
print(f'\nКлассов: {CLASS_COUNT}')
```

... Добавление класса "Макс Фрай"
Добавление файла "(Макс Фрай) Тестовая_2 вместе.txt" в класс "Макс Фрай", тестовая выборка.
Добавление класса "Рэй Брэдберри"
Добавление файла "(Рэй Брэдберри) Обучающая_22 вместе.txt" в класс "Рэй Брэдберри", обучающая выборка.
Добавление класса "Клиффорд Саймак"
Добавление файла "(Клиффорд Саймак) Обучающая_5 вместе.txt" в класс "Клиффорд Саймак", обучающая выборка.
Добавление класса "О. Генри"
Добавление файла "(О. Генри) Обучающая_50 вместе.txt" в класс "О. Генри", обучающая выборка.
Добавление класса "Булгаков"
Добавление файла "(Булгаков) Обучающая_5 вместе.txt" в класс "Булгаков", обучающая выборка.
Добавление файла "(О. Генри) Тестовая_20 вместе.txt" в класс "О. Генри", тестовая выборка.
Добавление файла "(Булгаков) Тестовая_2 вместе.txt" в класс "Булгаков", тестовая выборка.
Добавление файла "(Рэй Брэдберри) Тестовая_8 вместе.txt" в класс "Рэй Брэдберри", тестовая выборка.
Добавление файла "(Макс Фрай) Обучающая_5 вместе.txt" в класс "Макс Фрай", обучающая выборка.
Добавление класса "Стругацкие"
Добавление файла "(Стругацкие) Тестовая_2 вместе.txt" в класс "Стругацкие", тестовая выборка.
Добавление файла "(Стругацкие) Обучающая_5 вместе.txt" в класс "Стругацкие", обучающая выборка.
Добавление файла "(Клиффорд Саймак) Тестовая_2 вместе.txt" в класс "Клиффорд Саймак", тестовая выборка.

Классов: 6

Рисунок 1.1.3 – Результат загрузки текстов и формирования списка классов

1.1.5 Параметры преобразования.

Для подготовки текстов к подаче в нейросеть задаются три ключевых параметра: VOCAB_SIZE = 15000 – размер словаря, WIN_SIZE = 1000 – длина отрезка текста в словах, WIN_HOP = 100 – шаг скользящего окна. Эти значения подобраны как компромисс между объёмом обучающей выборки (чем меньше шаг и окно, тем больше примеров получится после нарезки),

полнотой словаря (15000 наиболее частотных слов покрывают основную авторскую лексику) и сложностью модели (чем больше окно, тем длиннее входной вектор и больше параметров).

Листинг 1.1.2 – Параметры преобразования текстов

```
VOCAB_SIZE = 15000  
WIN_SIZE   = 1000  
WIN_HOP    = 100
```

1.1.6 Токенизация и построение частотного словаря.

Для преобразования текстов в последовательности целых чисел применяется класс `Tokenizer` из `tensorflow.keras.preprocessing.text`. В параметре `filters` задаётся набор символов для удаления (знаки препинания, переводы строк, неразрывные пробелы и спецсимволы), параметр `lower=True` обеспечивает приведение к нижнему регистру, `oov_token='неизвестное_слово'` резервирует индекс для слов, не вошедших в топ-`VOCAB_SIZE` наиболее частотных. Метод `fit_on_texts` строит словарь по обучающей выборке. Полный размер построенного словаря составил 133 070 уникальных слов, из которых для дальнейшей работы будут использоваться только 15 000 наиболее частотных. Время токенизации – около 5,1 секунды.

Токенизация и построение частотного словаря

<pre>class timex: def __enter__(self): self.t = time.time() return self def __exit__(self, type, value, traceback): print('Время обработки: {:.2f} с'.format(time.time() - self.t))</pre>	
<pre>with timex(): tokenizer = Tokenizer(num_words=VOCAB_SIZE, filters='!"#\$%&()*+,-./:;<=>?@[\]^_`{ }~\n\t\r\n\x00\xff', lower=True, split=' ', oov_token='неизвестное_слово', char_level=False) tokenizer.fit_on_texts(text_train) items = list(tokenizer.word_index.items()) print('Размер словаря:', len(items))</pre>	
Время обработки: 5.10 с Размер словаря: 133070	
<pre>with timex(): seq_train = tokenizer.texts_to_sequences(text_train) seq_test = tokenizer.texts_to_sequences(text_test)</pre>	
Время обработки: 4.15 с	

Рисунок 1.1.4 – Размер построенного частотного словаря и время обработки

После построения словаря методы `texts_to_sequences` применяются к обучающей и тестовой выборкам, преобразуя каждый текст автора в одну длинную последовательность индексов. Время обработки – около 4,2 секунды.

1.1.7 Формирование обучающей и тестовой выборок методом скользящего окна.

Поскольку каждый автор представлен одной длинной последовательностью индексов, для получения обучающих примеров применяется метод скользящего окна: длинная последовательность нарезается на отрезки фиксированной длины `WIN_SIZE` с шагом `WIN_HOP`. Функция `split_sequence` возвращает список таких отрезков, а функция `vectorize_sequence` обходит все классы и собирает тензоры `x` (отрезки) и `y` (метки в формате One-Hot Encoding). В результате обучающая выборка содержит 17 640 отрезков длиной 1000 слов, тестовая – 6 686 отрезков. Время формирования – около 2,8 секунды.

Листинг 1.1.3 – Функции формирования выборок методом скользящего окна

```
def split_sequence(sequence, win_size, hop):
```

```

    return [sequence[i:i + win_size] for i in range(0, len(sequence) -
win_size + 1, hop)]

```

```

def vectorize_sequence(seq_list, win_size, hop):
    class_count = len(seq_list)
    x, y = [], []
    for cls in range(class_count):
        vectors = split_sequence(seq_list[cls], win_size, hop)
        x += vectors
        y += [utils.to_categorical(cls, class_count)] * len(vectors)
    return np.array(x), np.array(y)

```

1.1.8 Преобразование последовательностей в формат Bag of Words.

Для моделей класса Bag of Words (BoW), в которых порядок слов не учитывается, последовательности отрезков преобразуются в матрицы счётчиков частот при помощи метода `tokenizer.sequences_to_matrix`. Полученные матрицы `x_train_01` и `x_test_01` имеют форму (число_примеров, VOCAB_SIZE) и содержат количества вхождений каждого слова словаря в каждом отрезке текста. Размерность входного вектора одной модели – 15 000. Время преобразования – около 9 секунд.

```

with timex():
    x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
    x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)
    print(x_train.shape, y_train.shape)
    print(x_test.shape, y_test.shape)

```

```

(17640, 1000) (17640, 6)
(6686, 1000) (6686, 6)
Время обработки: 2.84 с

```

```

with timex():
    x_train_01 = tokenizer.sequences_to_matrix(x_train.tolist())
    x_test_01 = tokenizer.sequences_to_matrix(x_test.tolist())
    print(x_train_01.shape)

```

```

(17640, 15000)
Время обработки: 9.00 с

```

Рисунок 1.1.6 – Преобразование выборок в формат Bag of Words

1.1.9 Сервисные функции для обучения и оценки моделей.

Чтобы избежать дублирования кода при сравнении нескольких моделей, обучение и оценка реализованы в виде сервисных функций. Функция `compile_train_model` выполняет компиляцию модели (функция ошибки `categorical_crossentropy`, оптимизатор `adam`, метрика `accuracy`), вывод архитектуры через `model.summary()`, запуск обучения методом `fit` (`epochs=20`, `batch_size=128`) и построение двух графиков по результатам обучения: динамика `accuracy` на обучающей и проверочной выборках и динамика `loss` на тех же выборках.

Функция `eval_model` получает предсказания модели на тестовой выборке, строит нормализованную матрицу ошибок (нормировка `normalize='true'` делит значения на сумму элементов соответствующей строки, что позволяет интерпретировать диагональ как долю верно классифицированных примеров каждого класса), отображает её через `ConfusionMatrixDisplay` и выводит подробный текстовый отчёт по классам с указанием, какой класс был предсказан и верно ли это. Возвращаемое значение – средняя точность распознавания, рассчитанная как среднее по диагонали матрицы.

Функция `compile_train_eval_model` объединяет вызовы двух предыдущих функций и сохраняет результаты в общем словаре `results`, что упрощает последующее сведение метрик нескольких моделей в единую таблицу.

✓ Сервисные функции

```
[15]
✓ 0 сек.
def compile_train_model(model, x_train, y_train, x_val, y_val,
                        optimizer='adam', epochs=20, batch_size=128, figsize=(20, 5)):
    model.compile(optimizer=optimizer,
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    model.summary()

    history = model.fit(x_train, y_train,
                       epochs=epochs,
                       batch_size=batch_size,
                       validation_data=(x_val, y_val))

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=figsize)
    fig.suptitle('График процесса обучения модели')
    ax1.plot(history.history['accuracy'], label='Обучающая выборка')
    ax1.plot(history.history['val_accuracy'], label='Проверочная выборка')
    ax1.set_xlabel('Эпоха обучения')
    ax1.set_ylabel('Доля верных ответов')
    ax1.legend()

    ax2.plot(history.history['loss'], label='Обучающая выборка')
    ax2.plot(history.history['val_loss'], label='Проверочная выборка')
    ax2.set_xlabel('Эпоха обучения')
    ax2.set_ylabel('Ошибка')
    ax2.legend()
    plt.show()

def eval_model(model, x, y_true, class_labels=[], cm_round=3, title='', figsize=(10, 10)):
    y_pred = model.predict(x)
    cm = confusion_matrix(np.argmax(y_true, axis=1), np.argmax(y_pred, axis=1), normalize='true')
    cm = np.around(cm, cm_round)

    fig, ax = plt.subplots(figsize=figsize)
    ax.set_title(f'Нейросеть {title}: матрица ошибок', fontsize=18)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_labels)
    disp.plot(ax=ax)
    plt.gca().images[-1].colorbar.remove()
    plt.xlabel('Предсказанные классы', fontsize=16)
    plt.ylabel('Верные классы', fontsize=16)
    fig.autofmt_xdate(rotation=45)
    plt.show()

    print('-'*100)
    print(f'Нейросеть: {title}')
    for cls in range(len(class_labels)):
```

Рисунок 1.1.7 – Сервисные функции compile_train_model, eval_model и compile_train_eval_model (часть 1)

```

print('-'*100)
print(f'Нейросеть: {title}')
for cls in range(len(class_labels)):
    cls_pred = np.argmax(cm[cls])
    msg = 'ВЕРНО :-)' if cls_pred == cls else 'НЕВЕРНО :-('
    print('Класс: {:<20} {:3.0f}% сеть отнесла к классу {:<20} - {}'.format(
        class_labels[cls], 100. * cm[cls, cls_pred], class_labels[cls_pred], msg))
accuracy = 100. * cm.diagonal().mean()
print('\nСредняя точность распознавания: {:3.0f}%'.format(accuracy))
return accuracy

def compile_train_eval_model(model, x_train, y_train, x_test, y_test,
                             class_labels=CLASS_LIST, title='', optimizer='adam',
                             epochs=20, batch_size=128, graph_size=(20, 5), cm_size=(10, 10)):
    compile_train_model(model, x_train, y_train, x_test, y_test,
                        optimizer=optimizer, epochs=epochs, batch_size=batch_size, figsize=graph_size)
    accuracy = eval_model(model, x_test, y_test, class_labels=class_labels, title=title, figsize=cm_size)
    return accuracy

results = {}

```

Рисунок 1.1.7 – Сервисные функции (часть 2)

1.1.10 Архитектура и обучение Модели 1: BoW + Dense(200) + Dropout(0.25).

Первая модель построена по простейшей архитектуре Bag of Words и состоит из четырёх слоёв: входного полносвязного слоя Dense на 200 нейронов с активацией ReLU и входной размерностью VOCAB_SIZE=15000, регуляризационного слоя Dropout с долей отключаемых нейронов 0.25, слоя пакетной нормализации BatchNormalization и выходного полносвязного слоя Dense на 6 нейронов (по числу классов) с активацией softmax.

Листинг 1.1.4 – Архитектура Модели 1

```

model_1 = Sequential()
model_1.add(Dense(200, input_dim=VOCAB_SIZE, activation='relu'))
model_1.add(Dropout(0.25))
model_1.add(BatchNormalization())
model_1.add(Dense(CLASS_COUNT, activation='softmax'))

acc_1 = compile_train_eval_model(model_1, x_train_01, y_train, x_test_01,
y_test,

                                title='Модель 1: BoW Dense(200)')
results['Модель 1: BoW Dense(200)'] = acc_1

```

Общее количество параметров модели – 3 002 206, из которых обучаемых 3 001 806 и необучаемых 400 (статистика BatchNormalization).

Подавляющее большинство параметров (3 000 200 – около 99,9%) сосредоточено во входном слое Dense, что обусловлено большой входной размерностью 15 000.

9 моделей с различными гиперпараметрами

Модель 1: BoW + Dense(200) + Dropout(0.25)

```
model_1 = Sequential()
model_1.add(Dense(200, input_dim=VOCAB_SIZE, activation='relu'))
model_1.add(Dropout(0.25))
model_1.add(BatchNormalization())
model_1.add(Dense(CLASS_COUNT, activation='softmax'))

acc_1 = compile_train_eval_model(model_1, x_train_01, y_train, x_test_01, y_test,
                                title='Модель 1: BoW Dense(200)')
results['Модель 1: BoW Dense(200)'] = acc_1
```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape`/`input\_dim`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 200)	3,000,200
dropout (Dropout)	(None, 200)	0
batch_normalization (BatchNormalization)	(None, 200)	800
dense_1 (Dense)	(None, 6)	1,206

Total params: 3,002,206 (11.45 MB)
Trainable params: 3,001,806 (11.45 MB)
Non-trainable params: 400 (1.56 KB)

Epoch 1/20
138/138 — 8s 35ms/step - accuracy: 0.9838 - loss: 0.0494 - val_accuracy: 0.8901 - val_loss: 0.3936
Epoch 2/20
138/138 — 1s 8ms/step - accuracy: 1.0000 - loss: 4.9279e-04 - val_accuracy: 0.8955 - val_loss: 0.3374
Epoch 3/20
138/138 — 1s 8ms/step - accuracy: 1.0000 - loss: 2.6719e-04 - val_accuracy: 0.8950 - val_loss: 0.3284
Epoch 4/20
138/138 — 1s 9ms/step - accuracy: 1.0000 - loss: 1.5808e-04 - val_accuracy: 0.8965 - val_loss: 0.3231
Epoch 5/20
138/138 — 1s 9ms/step - accuracy: 1.0000 - loss: 1.0791e-04 - val_accuracy: 0.8955 - val_loss: 0.3233
Epoch 6/20
138/138 — 1s 8ms/step - accuracy: 1.0000 - loss: 8.4864e-05 - val_accuracy: 0.8971 - val_loss: 0.3208
Epoch 7/20
138/138 — 1s 9ms/step - accuracy: 1.0000 - loss: 6.4268e-05 - val_accuracy: 0.8987 - val_loss: 0.3143
Epoch 8/20
138/138 — 1s 9ms/step - accuracy: 1.0000 - loss: 5.0193e-05 - val_accuracy: 0.8998 - val_loss: 0.3104
Epoch 9/20

Рисунок 1.1.8 – Структура Модели 1 и начало процесса обучения

1.1.11 Анализ процесса обучения Модели 1.

Из выводимых в процессе обучения метрик видно характерное для BoW-моделей поведение: точность на обучающей выборке практически мгновенно достигает 100% (уже к 2-й эпохе loss падает с 0,049 до $4,9 \cdot 10^{-4}$), что указывает на полное «запоминание» обучающей выборки. Точность на проверочной выборке стабилизируется в диапазоне 89–90%, при этом val_loss плавно снижается с 0,394 (1-я эпоха) до 0,303 (20-я эпоха) – значит, несмотря на 100% точность на train, модель не теряет способности обобщать.

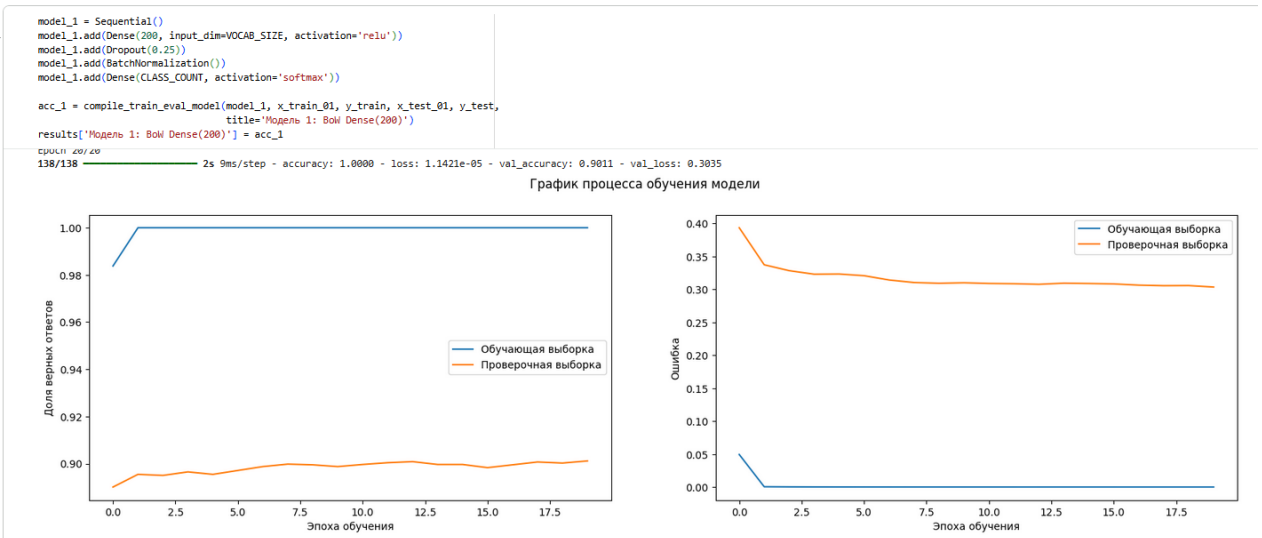


Рисунок 1.1.9 – График процесса обучения Модели 1

Графики подтверждают наблюдения по логам: кривая ассигуры на обучающей выборке мгновенно поднимается до 1,0 и остаётся плоской, тогда как на проверочной выборке плавно растёт с 0,89 до 0,90. Кривая loss на обучении почти сразу прижимается к нулю, а на проверке устойчиво снижается без признаков расходимости. Такая картина характерна для классификаторов на основе BoW: высокоразмерное пространство счётчиков слов хорошо разделяет тексты разных авторов, а большое количество параметров позволяет легко запомнить обучающие примеры. Дальнейший рост val_accuracy упирается в принципиальное ограничение модели – игнорирование порядка слов.

1.1.12 Матрица ошибок Модели 1.

Для подробного анализа качества классификации строится нормализованная матрица ошибок. Значение на пересечении строки «верный класс» и столбца «предсказанный класс» показывает, какая доля примеров данного автора была отнесена сетью к соответствующему классу. По диагонали матрицы расположены доли верно распознанных примеров каждого автора.

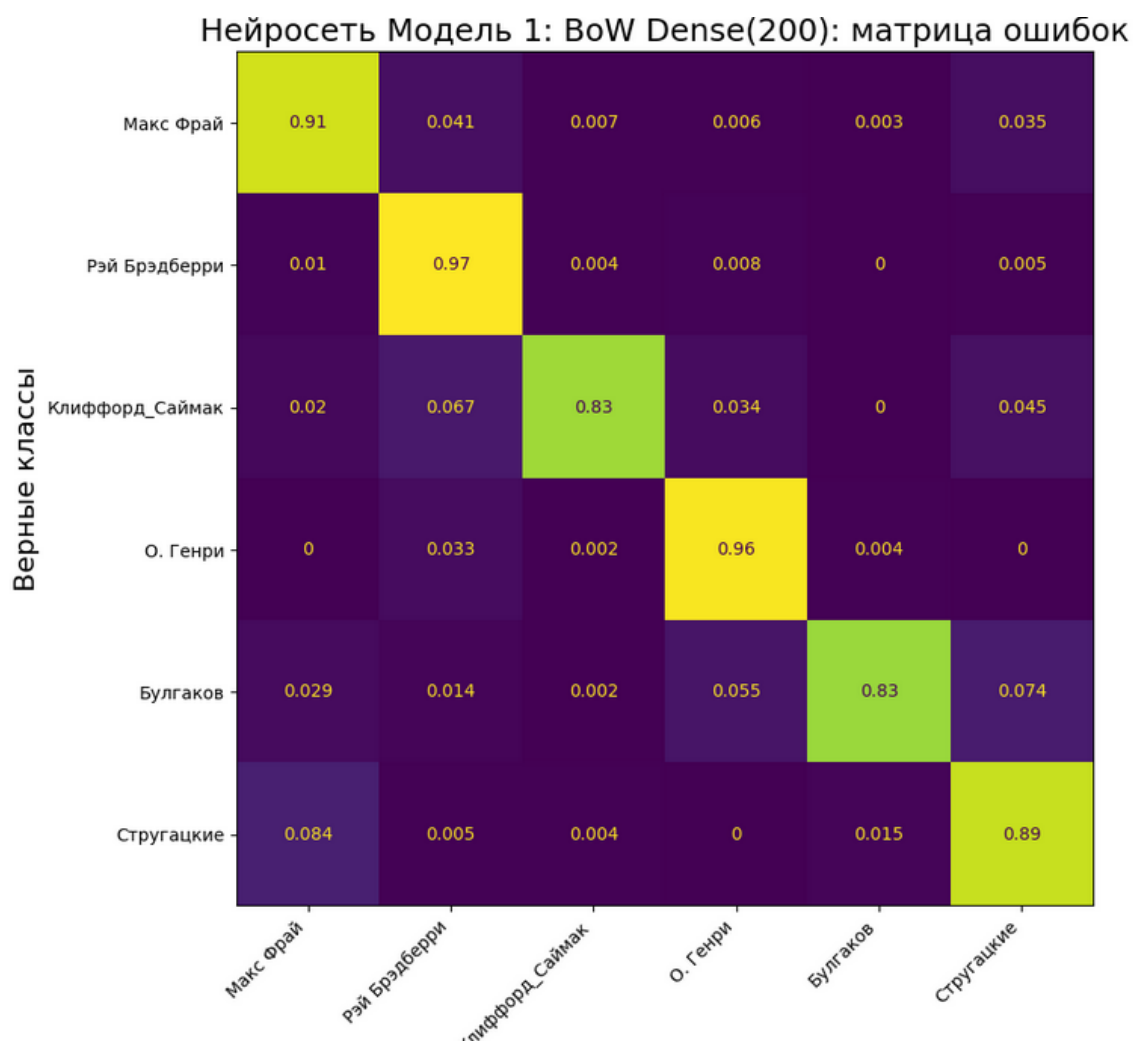


Рисунок 1.1.10 – Матрица ошибок Модели 1

Анализ матрицы ошибок показывает, что Модель 1 уверенно различает шесть авторов с точностью на диагонали в диапазоне 0,83–0,97. Наиболее уверенно распознаются Рэй Брэдберри (97%) и О. Генри (96%) – их стилистика наиболее характерна и не пересекается с другими авторами в наблюдаемом наборе. Сложнее всего модели даются Клиффорд Саймак (83%) и Булгаков (83%): фрагменты Клиффорда Саймака в 6,7% случаев отнесены к Рэю Брэдберри (что объяснимо общей принадлежностью обоих авторов к научной фантастике XX века), фрагменты Булгакова в 5,5% случаев – к О. Генри. Симметричных «массовых» путаниц нет: ни один из посторонних классов не превышает порога в 8% ошибок при предсказании какого-либо автора, что говорит о хорошей разделимости стилистических признаков на уровне частот слов.

Предсказанные классы

Нейросеть: Модель 1: BoW Dense(200)

Класс: Макс Фрай	91% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	97% сеть отнесла к классу Рэй Брэдберри	- ВЕРНО :-)
Класс: Клиффорд_Саймак	83% сеть отнесла к классу Клиффорд_Саймак	- ВЕРНО :-)
Класс: О. Генри	96% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	83% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	89% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 90%

Рисунок 1.1.11 – Текстовый отчёт о точности распознавания Модели 1

Итоговая средняя точность распознавания Модели 1 составила 90%, что является высоким результатом для модели Bag of Words без учёта порядка слов и подтверждает применимость подхода для задачи определения авторства.

1.1.13 Архитектура и обучение Модели 2: BoW + Dense(400) + Dropout(0.4).

Вторая модель имеет такую же общую структуру, как и Модель 1, но с увеличенным до 400 нейронов скрытым слоем и более агрессивной регуляризацией Dropout(0.4) для борьбы с возможным переобучением. Архитектура: входной Dense(400) с активацией ReLU и входной размерностью VOCAB_SIZE=15000, Dropout(0.4), BatchNormalization, выходной Dense(6) с активацией softmax.

Листинг 1.1.5 – Архитектура Модели 2

```
model_2 = Sequential()  
model_2.add(Dense(400, input_dim=VOCAB_SIZE, activation='relu'))  
model_2.add(Dropout(0.4))  
model_2.add(BatchNormalization())  
model_2.add(Dense(CLASS_COUNT, activation='softmax'))
```

Общее количество параметров модели – 6 004 406 (около 22,9 МБ), из которых обучаемых 6 003 606 и необучаемых 800. Удвоение размера скрытого слоя по сравнению с Моделью 1 привело к двукратному росту количества параметров, основная масса которых сосредоточена во входном слое (6 000 400 параметров).

Модель 2: BoW + Dense(400) + Dropout(0.4)

```
model_2 = Sequential()
model_2.add(Dense(400, input_dim=VOCAB_SIZE, activation='relu'))
model_2.add(Dropout(0.4))
model_2.add(BatchNormalization())
model_2.add(Dense(CLASS_COUNT, activation='softmax'))

acc_2 = compile_train_eval_model(model_2, x_train_01, y_train, x_test_01, y_test,
                                title='Модель 2: BoW Dense(400)')
results['Модель 2: BoW Dense(400)'] = acc_2
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_d
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 400)	6,000,400
dropout_1 (Dropout)	(None, 400)	0
batch_normalization_1 (BatchNormalization)	(None, 400)	1,600
dense_3 (Dense)	(None, 6)	2,406

```
Total params: 6,004,406 (22.90 MB)
Trainable params: 6,003,606 (22.90 MB)
Non-trainable params: 800 (3.12 KB)
Epoch 1/20
138/138 ————— 8s 37ms/step - accuracy: 0.9866 - loss: 0.0466 - val_accuracy: 0.8771 - val_loss: 0.3826
Epoch 2/20
138/138 ————— 1s 10ms/step - accuracy: 1.0000 - loss: 2.6940e-04 - val_accuracy: 0.8857 - val_loss: 0.3350
Epoch 3/20
138/138 ————— 2s 9ms/step - accuracy: 1.0000 - loss: 1.2698e-04 - val_accuracy: 0.8851 - val_loss: 0.3291
Epoch 4/20
138/138 ————— 1s 9ms/step - accuracy: 1.0000 - loss: 8.9449e-05 - val_accuracy: 0.8845 - val_loss: 0.3280
Epoch 5/20
138/138 ————— 1s 9ms/step - accuracy: 1.0000 - loss: 5.7935e-05 - val_accuracy: 0.8827 - val_loss: 0.3298
Epoch 6/20
138/138 ————— 2s 11ms/step - accuracy: 1.0000 - loss: 4.6280e-05 - val_accuracy: 0.8830 - val_loss: 0.3283
Epoch 7/20
138/138 ————— 2s 12ms/step - accuracy: 1.0000 - loss: 3.2800e-05 - val_accuracy: 0.8835 - val_loss: 0.3254
Epoch 8/20
138/138 ————— 1s 10ms/step - accuracy: 1.0000 - loss: 2.7806e-05 - val_accuracy: 0.8836 - val_loss: 0.3249
Epoch 9/20
138/138 ————— 1s 9ms/step - accuracy: 1.0000 - loss: 2.3714e-05 - val_accuracy: 0.8839 - val_loss: 0.3242
Epoch 10/20
138/138 ————— 1s 9ms/step - accuracy: 1.0000 - loss: 1.9018e-05 - val_accuracy: 0.8841 - val_loss: 0.3238
Epoch 11/20
```

Рисунок 1.1.12 – Структура и процесс обучения Модели 2

В процессе обучения наблюдается такая же картина быстрого «запоминания» обучающей выборки, как у Модели 1: уже к 2-й эпохе `train_ассигу` выходит на 100% и далее не меняется. Точность на проверочной выборке колеблется в диапазоне 87,7–88,9% с плавным улучшением: `val_loss` снижается с 0,383 (1-я эпоха) до 0,321 (20-я эпоха), `val_ассигу` улучшается с 0,8771 до 0,8883. По сравнению с Моделью 1 итоговый `val_ассигу` несколько ниже, что свидетельствует о слабом эффекте увеличения скрытого слоя для данной задачи.

Модель 2: BoW + Dense(400) + Dropout(0.4)

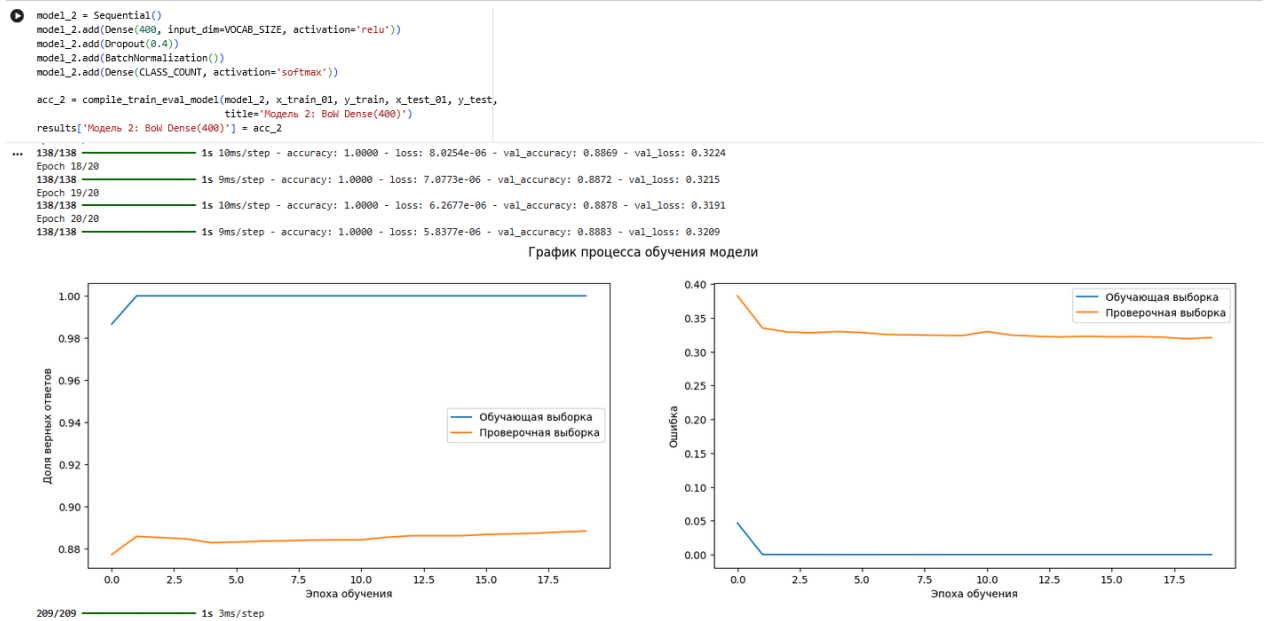


Рисунок 1.1.13 – График процесса обучения Модели 2

Графики подтверждают логи: кривая `train_ассигасу` мгновенно поднимается к единице, `val_ассигасу` плавно растёт от 0,88 до 0,89; `train_loss` близок к нулю, `val_loss` медленно снижается, без признаков расхождения. Эти признаки указывают на стабильное обучение без переобучения, несмотря на удвоенное число параметров.

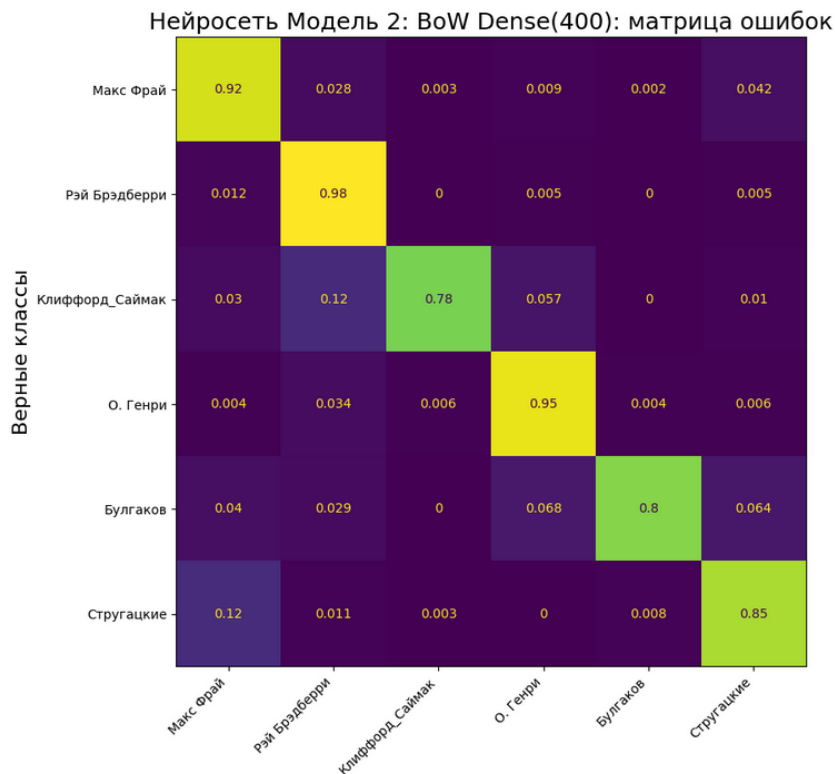


Рисунок 1.1.14 – Матрица ошибок Модели 2

Анализ матрицы ошибок показывает, что Модель 2 распознаёт классы с долями на диагонали 0,80–0,98. Лучше всего распознаются Рэй Брэдберри (98%) и О. Генри (95%), хуже всего – Клиффорд Саймак (78%) и Булгаков (80%). По сравнению с Моделью 1 наблюдается специфическое перераспределение ошибок: фрагменты Клиффорда Саймака теперь в 12% случаев попадают к Рэю Брэдберри (против 6,7% у Модели 1), а Стругацкие в 12% случаев ошибочно отнесены к Макс Фраю. Это говорит о том, что увеличенный скрытый слой даёт чуть менее устойчивое разделение пограничных стилистических классов.

Предсказанные классы

```
-----
Нейросеть: Модель 2: BoW Dense(400)
Класс: Макс Фрай          92% сеть отнесла к классу Макс Фрай          - ВЕРНО :-)
Класс: Рэй Брэдберри      98% сеть отнесла к классу Рэй Брэдберри      - ВЕРНО :-)
Класс: Клиффорд_Саймак    78% сеть отнесла к классу Клиффорд_Саймак    - ВЕРНО :-)
Класс: О. Генри           95% сеть отнесла к классу О. Генри           - ВЕРНО :-)
Класс: Булгаков           80% сеть отнесла к классу Булгаков           - ВЕРНО :-)
Класс: Стругацкие        86% сеть отнесла к классу Стругацкие        - ВЕРНО :-)

Средняя точность распознавания: 88%
```

Рисунок 1.1.15 – Текстовый отчёт о точности распознавания Модели 2

Итоговая средняя точность распознавания Модели 2 составила 88%, что на 2 процентных пункта ниже, чем у Модели 1. Этот результат показывает, что в задаче BoW-классификации простое увеличение количества параметров без изменения архитектуры не приводит к улучшению качества – модель сразу же запоминает обучающие данные и упирается в принципиальный потолок BoW-подхода.

1.1.14 Архитектура и обучение Модели 3: BoW + Dense(300→100).

Третья модель углубляет архитектуру за счёт добавления второго скрытого слоя: Dense(300) → Dropout(0.3) → BatchNormalization → Dense(100) → Dropout(0.3) → BatchNormalization → выходной Dense(6) с softmax. Дополнительный слой даёт модели возможность построения более сложных нелинейных комбинаций признаков bag-of-words.

Листинг 1.1.6 – Архитектура Модели 3

```
model_3 = Sequential()
model_3.add(Dense(300, input_dim=VOCAB_SIZE, activation='relu'))
```

```
model_3.add(Dropout(0.3))
model_3.add(BatchNormalization())
model_3.add(Dense(100, activation='relu'))
model_3.add(Dropout(0.3))
model_3.add(BatchNormalization())
model_3.add(Dense(CLASS_COUNT, activation='softmax'))
```

Общее количество параметров модели – 4 532 606 (17,29 МБ), из которых обучаемых 4 531 806 и необучаемых 800. Несмотря на наличие двух скрытых слоёв, общее число параметров получилось меньше, чем у Модели 2, так как уменьшен размер первого Dense (300 против 400).

Модель 3: BoW + Dense(300→100)

```
model_3 = Sequential()
model_3.add(Dense(300, input_dim=VOCAB_SIZE, activation='relu'))
model_3.add(Dropout(0.3))
model_3.add(BatchNormalization())
model_3.add(Dense(100, activation='relu'))
model_3.add(Dropout(0.3))
model_3.add(BatchNormalization())
model_3.add(Dense(CLASS_COUNT, activation='softmax'))

acc_3 = compile_train_eval_model(model_3, x_train_01, y_train, x_test_01, y_test,
                                title='Модель 3: BoW Dense(300→100)')
results['Модель 3: BoW Dense(300→100)'] = acc_3
```

```
... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` arg
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 300)	4,500,300
dropout_2 (Dropout)	(None, 300)	0
batch_normalization_2 (BatchNormalization)	(None, 300)	1,200
dense_5 (Dense)	(None, 100)	30,100
dropout_3 (Dropout)	(None, 100)	0
batch_normalization_3 (BatchNormalization)	(None, 100)	400
dense_6 (Dense)	(None, 6)	606

Total params: 4,532,606 (17.29 MB)
Trainable params: 4,531,806 (17.29 MB)
Non-trainable params: 800 (3.12 KB)

Epoch 1/20
138/138 ————— 10s 41ms/step - accuracy: 0.9692 - loss: 0.1118 - val_accuracy: 0.9022 - val_loss: 0.4033

Epoch 2/20
138/138 ————— 1s 9ms/step - accuracy: 0.9999 - loss: 0.0042 - val_accuracy: 0.9112 - val_loss: 0.3138

Epoch 3/20
138/138 ————— 2s 11ms/step - accuracy: 1.0000 - loss: 0.0020 - val_accuracy: 0.9091 - val_loss: 0.2901

Epoch 4/20
138/138 ————— 2s 11ms/step - accuracy: 1.0000 - loss: 0.0012 - val_accuracy: 0.9067 - val_loss: 0.2873

Epoch 5/20
138/138 ————— 1s 9ms/step - accuracy: 1.0000 - loss: 8.1435e-04 - val_accuracy: 0.9089 - val_loss: 0.2776

Epoch 6/20
138/138 ————— 1s 9ms/step - accuracy: 1.0000 - loss: 6.4404e-04 - val_accuracy: 0.9098 - val_loss: 0.2692

Epoch 7/20

Рисунок 1.1.16 – Структура и процесс обучения Модели 3

Процесс обучения демонстрирует характерную для VoW-моделей быструю сходимость на обучающей выборке (train_accuracy достигает 100% уже к 2-й эпохе) и стабильно высокий результат на проверочной: val_accuracy растёт с 0,9022 (1-я эпоха) до 0,9131 (20-я эпоха) с пиковыми значениями 0,9150 на 19-й эпохе. Кривая val_loss монотонно снижается с 0,403 до 0,265, что говорит о хорошем качестве обобщения. Это лучший результат среди всех VoW-моделей.

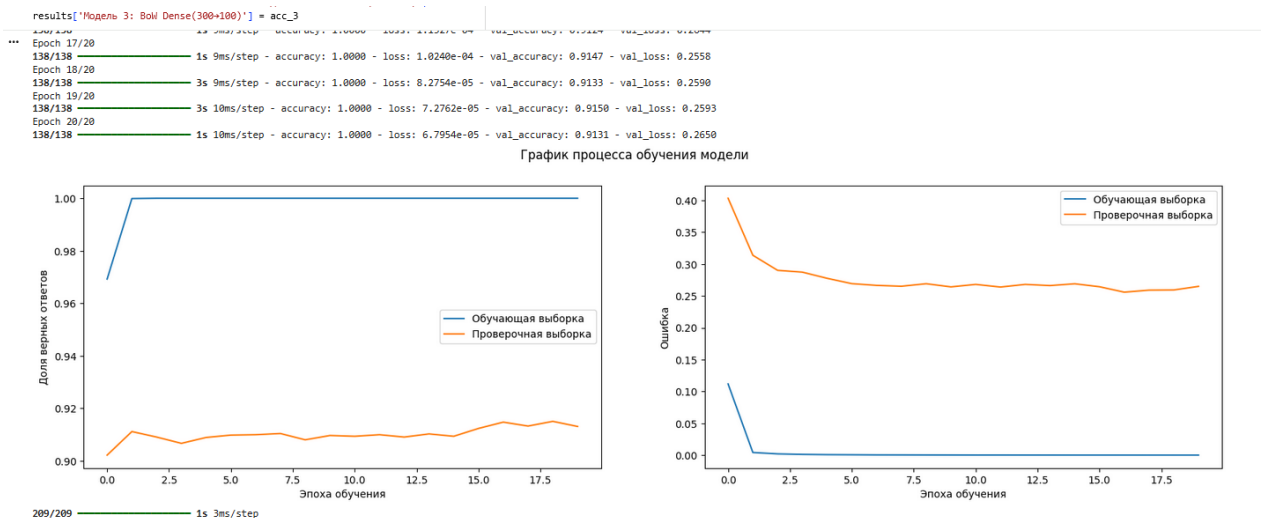


Рисунок 1.1.17 – График процесса обучения Модели 3

Из графиков видно, что углубление архитектуры за счёт второго скрытого слоя даёт небольшой, но устойчивый прирост val_accuracy: кривая аккуратно стабилизируется в диапазоне 0,90–0,92, кривая val_loss остаётся ниже, чем у Моделей 1 и 2. Расхождения между обучающей и проверочной кривыми не нарастают, что подтверждает отсутствие переобучения.

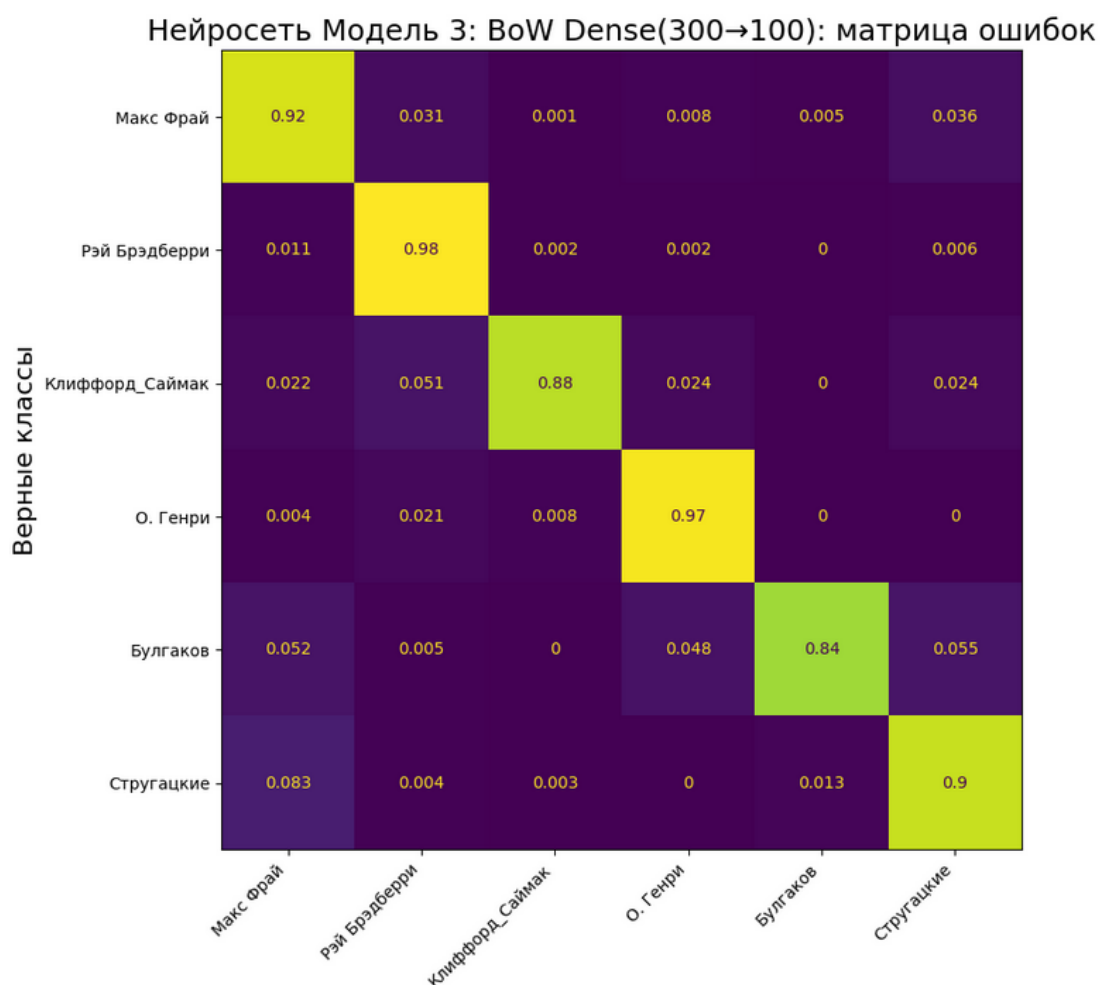


Рисунок 1.1.18 – Матрица ошибок Модели 3

Анализ матрицы ошибок показывает диагональ 0,84–0,98. Наиболее уверенно распознаются Рэй Брэдберри (98%) и О. Генри (97%); хуже всего распознаётся Булгаков (84%), фрагменты которого в 5,5% случаев отнесены к Стругацким, в 5,2% – к Макс Фраю и в 4,8% – к О. Генри. Клиффорд Саймак распознан с точностью 88% (5,1% ошибочно отнесены к Рэю Брэдберри). По сравнению с Моделями 1 и 2 расхождения по «трудным» классам сократились, что и обеспечило прирост средней точности.

Предсказанные классы

Нейросеть: Модель 3: BoW Dense(300→100)

Класс: Макс Фрай	92% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	98% сеть отнесла к классу Рэй Брэдберри	- ВЕРНО :-)
Класс: Клиффорд Саймак	88% сеть отнесла к классу Клиффорд Саймак	- ВЕРНО :-)
Класс: О. Генри	97% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	84% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	90% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 91%

Рисунок 1.1.19 – Текстовый отчёт о точности распознавания Модели 3

Итоговая средняя точность распознавания Модели 3 составила 91%, что на 1 процентный пункт лучше Модели 1 и на 3 процентных пункта лучше Модели 2. Это подтверждает гипотезу о пользе углубления архитектуры BoW-классификатора за счёт добавления второго скрытого слоя.

1.1.15 Архитектура и обучение Модели 4: BoW + Dense(500→200→50).

Четвёртая модель развивает идею углубления архитектуры до трёх скрытых слоёв с убывающим количеством нейронов: Dense(500) → Dropout(0.3) → BatchNormalization → Dense(200) → Dropout(0.3) → BatchNormalization → Dense(50) → Dropout(0.2) → BatchNormalization → Dense(6) с softmax. Постепенное сужение размерности (500→200→50→6) направлено на иерархическое выделение признаков – от широких комбинаций слов к более компактным стилистическим маркерам.

Листинг 1.1.7 – Архитектура Модели 4

```
model_4 = Sequential()
model_4.add(Dense(500, input_dim=VOCAB_SIZE, activation='relu'))
model_4.add(Dropout(0.3))
model_4.add(BatchNormalization())
model_4.add(Dense(200, activation='relu'))
model_4.add(Dropout(0.3))
model_4.add(BatchNormalization())
model_4.add(Dense(50, activation='relu'))
model_4.add(Dropout(0.2))
model_4.add(BatchNormalization())
model_4.add(Dense(CLASS_COUNT, activation='softmax'))
```

Общее количество параметров модели – 7 614 056 (29,05 МБ), из которых обучаемых 7 612 556 и необучаемых 1 500. Это самая массивная из BoW-моделей; основная часть параметров (7 500 500) сосредоточена в первом Dense(500), а каждое последующее сужение добавляет существенно меньше параметров благодаря уменьшению размерностей.

Модель 4: BoW + Dense(500→200→50)

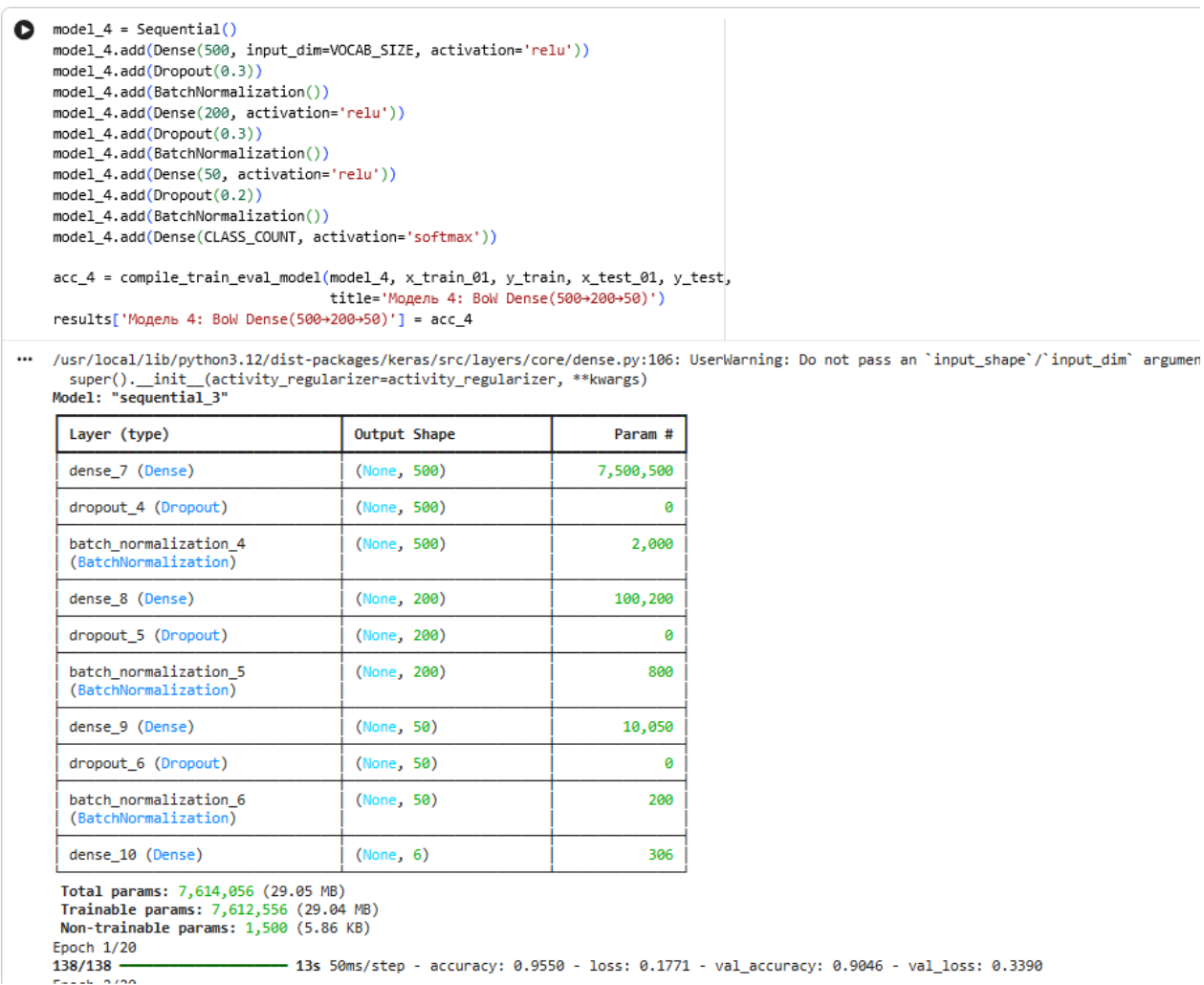


Рисунок 1.1.20 – Структура и процесс обучения Модели 4

В отличие от Моделей 1–3, обучение Модели 4 демонстрирует признаки переобучения. Train_ассигуру достигает 100% к 2-й эпохе, как у предыдущих BoW-моделей, однако val_ассигуру ведёт себя нестабильно: после первого пика 0,917 на 2-й эпохе она постепенно снижается до 0,886 на 20-й эпохе. Кривая val_loss проходит локальный минимум 0,266 на 6-й эпохе и далее растёт до 0,355, что является явным сигналом переобучения. Глубокая полносвязная сеть с большим количеством параметров «застряла» на запоминании обучающей выборки и потеряла способность улучшать обобщение.

```
results['Modelo 4: BoW Dense(500+200+50)'] = acc_4
```

```
138/138 — 2s 10ms/step - accuracy: 1.0000 - loss: 2.9399e-04 - val_accuracy: 0.8851 - val_loss: 0.3346
Epoch 16/20
138/138 — 2s 12ms/step - accuracy: 1.0000 - loss: 2.4531e-04 - val_accuracy: 0.8854 - val_loss: 0.3396
Epoch 17/20
138/138 — 2s 11ms/step - accuracy: 1.0000 - loss: 2.3174e-04 - val_accuracy: 0.8887 - val_loss: 0.3302
Epoch 18/20
138/138 — 1s 10ms/step - accuracy: 1.0000 - loss: 2.1998e-04 - val_accuracy: 0.8871 - val_loss: 0.3421
Epoch 19/20
138/138 — 3s 13ms/step - accuracy: 1.0000 - loss: 1.9071e-04 - val_accuracy: 0.8878 - val_loss: 0.3426
Epoch 20/20
138/138 — 1s 10ms/step - accuracy: 1.0000 - loss: 1.6264e-04 - val_accuracy: 0.8857 - val_loss: 0.3553
```

График процесса обучения модели

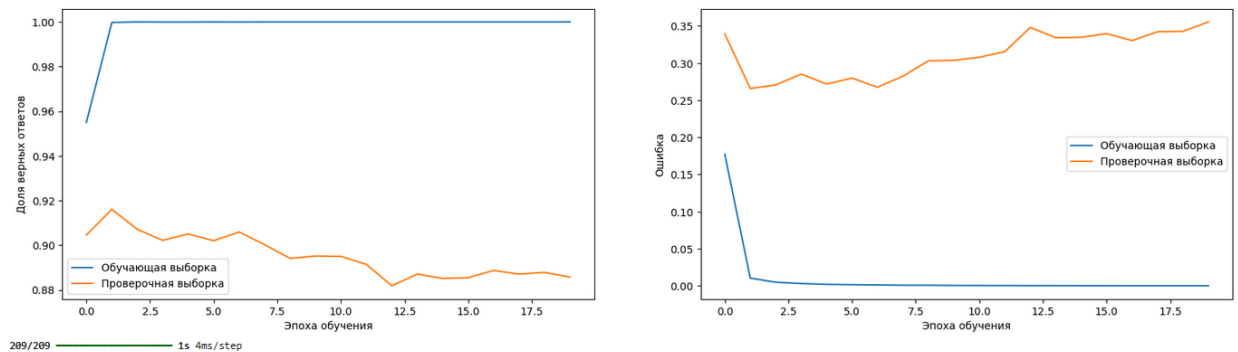


Рисунок 1.1.21 – График процесса обучения Модели 4

Графики наглядно подтверждают переобучение: кривая `val_loss` после 6-й эпохи устойчиво растёт, тогда как `train_loss` остаётся около нуля – расхождение нарастает. Кривая `val_аccuracy` осциллирует и в среднем снижается. Этот результат показывает, что простое наращивание глубины BoW-сети без введения принципиально новых механизмов (учёт порядка слов, эмбединги) не приносит пользы, а при чрезмерной глубине ухудшает качество.

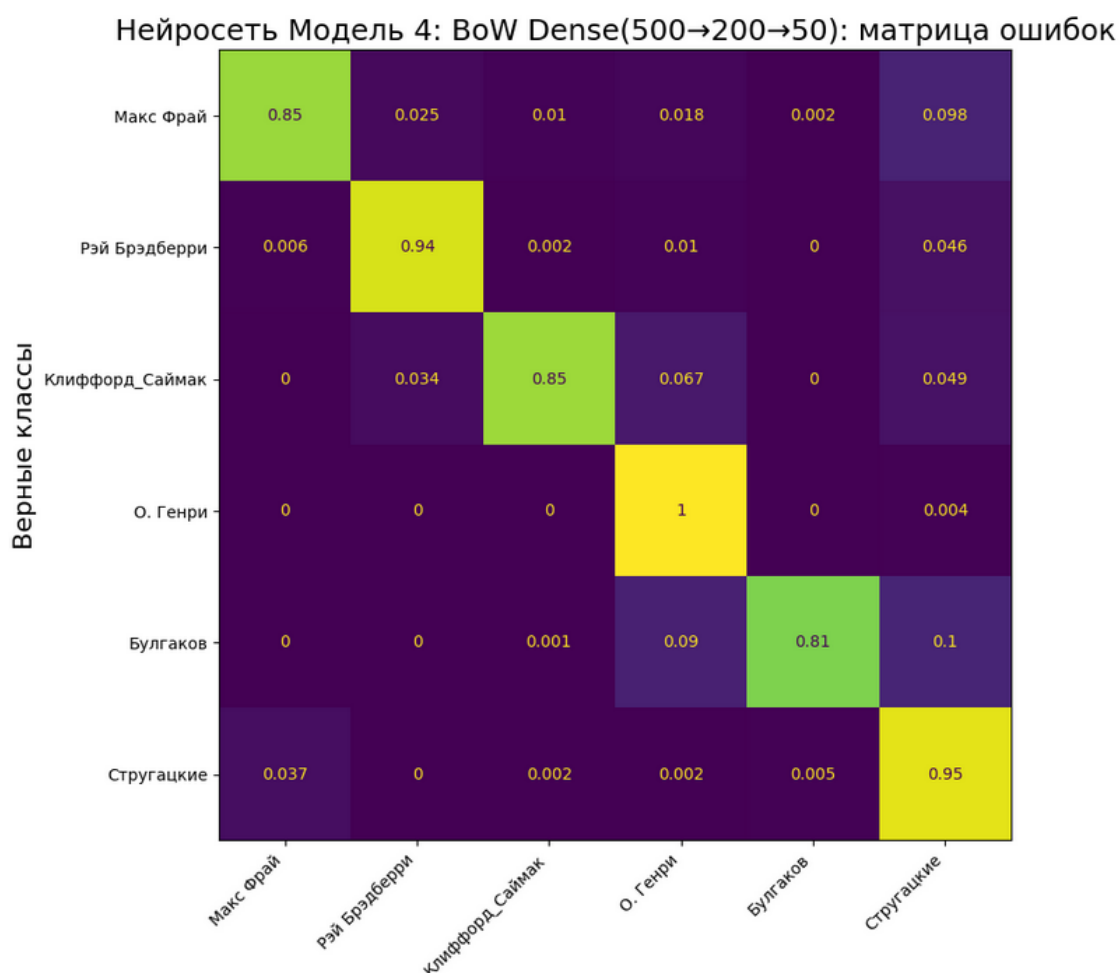


Рисунок 1.1.22 – Матрица ошибок Модели 4

Анализ матрицы ошибок выявляет интересную картину. Для одних классов модель показывает выдающиеся результаты: О. Генри распознаётся со 100% точностью, Стругацкие – 95%. Для других результаты значительно хуже: Булгаков – 81% (10% его фрагментов попали к Стругацким и 9% – к О. Генри), Макс Фрай – 85% (9,8% его фрагментов попали к Стругацким). То есть неравномерное переобучение «разрывает» матрицу: модель уверенно идентифицирует часть классов и при этом теряет различимость других.

Предсказанные классы

Нейросеть: Модель 4: BoW Dense(500→200→50)

Класс: Макс Фрай	85% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	94% сеть отнесла к классу Рэй Брэдберри	- ВЕРНО :-)
Класс: Клиффорд_Саймак	85% сеть отнесла к классу Клиффорд_Саймак	- ВЕРНО :-)
Класс: О. Генри	100% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	80% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	96% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 90%

Рисунок 1.1.23 – Текстовый отчёт о точности распознавания Модели 4

Итоговая средняя точность распознавания Модели 4 составила 90%, что на 1 процентный пункт ниже Модели 3 и совпадает с результатом Модели 1. Глубокая трёхслойная архитектура не оправдала себя на задаче BoW-классификации, и при дальнейшем сравнении с другими типами моделей становится очевидно, что предел возможностей подхода Bag of Words на данной задаче лежит в районе 90–91%.

1.1.16 Архитектура и обучение Модели 5: Embedding(20) + Flatten + Dense(200).

Начиная с Модели 5, происходит переход от BoW-представления к моделям с обучаемыми эмбедингами. Принципиальное отличие: входом теперь служит не разреженный вектор счётчиков длины VOCAB_SIZE, а последовательность индексов слов длины WIN_SIZE=1000, которая преобразуется слоем Embedding в плотные векторы. Это позволяет модели учитывать порядок слов в тексте и выделять стилистические сочетания.

Архитектура Модели 5: Embedding(input_dim=15000, output_dim=20, input_length=1000) → SpatialDropout1D(0.2) → Flatten → BatchNormalization → Dense(200, ReLU) → Dropout(0.2) → BatchNormalization → Dense(6, softmax). Размерность эмбединга сознательно выбрана малой (20) – это первый эксперимент с переходом к плотным представлениям, и в нём проверяется, насколько даже компактные эмбединги способны заменить высокоразмерное BoW-представление.

Листинг 1.1.8 – Архитектура Модели 5

```
model_5 = Sequential()
model_5.add(Embedding(input_dim=VOCAB_SIZE, output_dim=20,
input_length=WIN_SIZE))
model_5.add(SpatialDropout1D(0.2))
model_5.add(Flatten())
model_5.add(BatchNormalization())
model_5.add(Dense(200, activation='relu'))
model_5.add(Dropout(0.2))
model_5.add(BatchNormalization())
model_5.add(Dense(CLASS_COUNT, activation='softmax'))
```

Модель 5: Embedding(20) + Flatten + Dense(200)

```

model_5 = Sequential()
model_5.add(Embedding(input_dim=VOCAB_SIZE, output_dim=20, input_length=WIN_SIZE))
model_5.add(SpatialDropout1D(0.2))
model_5.add(Flatten())
model_5.add(BatchNormalization())
model_5.add(Dense(200, activation='relu'))
model_5.add(Dropout(0.2))
model_5.add(BatchNormalization())
model_5.add(Dense(CLASS_COUNT, activation='softmax'))

acc_5 = compile_train_eval_model(model_5, x_train, y_train, x_test, y_test,
                                title='Модель 5: Embedding(20)')
results['Модель 5: Embedding(20) + Flatten'] = acc_5

```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input\_length` is deprecated. Just remove it.
warnings.warn(
Model: "sequential_4"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
spatial_dropout1d (SpatialDropout1D)	?	0
flatten (Flatten)	?	0 (unbuilt)
batch_normalization_7 (BatchNormalization)	?	0 (unbuilt)
dense_11 (Dense)	?	0 (unbuilt)
dropout_7 (Dropout)	?	0
batch_normalization_8 (BatchNormalization)	?	0 (unbuilt)
dense_12 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Epoch 1/20
138/138 — 9s 30ms/step - accuracy: 0.7616 - loss: 0.7495 - val_accuracy: 0.3836 - val_loss: 1.5942
Epoch 2/20
138/138 — 1s 7ms/step - accuracy: 0.9989 - loss: 0.0148 - val_accuracy: 0.4049 - val_loss: 1.3240
Epoch 3/20
138/138 — 1s 6ms/step - accuracy: 0.9994 - loss: 0.0071 - val_accuracy: 0.6319 - val_loss: 1.0289
Epoch 4/20
138/138 — 1s 8ms/step - accuracy: 0.9996 - loss: 0.0048 - val_accuracy: 0.6965 - val_loss: 0.8375
Epoch 5/20

Рисунок 1.1.24 – Структура Модели 5 и начало обучения

Поведение модели в процессе обучения резко контрастирует с BoW-моделями. На обучающей выборке точность стартует с 0,7616 на 1-й эпохе (тогда как BoW-модели сразу выходили на 0,95+) и достигает 100% только к 4-й эпохе. На проверочной выборке точность ведёт себя нестабильно: 0,3836 на 1-й эпохе → 0,4049 на 2-й → 0,6319 на 3-й → 0,6965 на 4-й, после чего колеблется между 0,65 и 0,75 без устойчивой тенденции.

```

results['Модель 5: Embedding(20) + Flatten'] = acc_5
138/138 — 1s 8ms/step - accuracy: 0.9995 - loss: 0.0011 - val_accuracy: 0.7134 - val_loss: 0.9701
Epoch 20/20
138/138 — 1s 6ms/step - accuracy: 0.9998 - loss: 0.0011 - val_accuracy: 0.6585 - val_loss: 1.3090

```

График процесса обучения модели

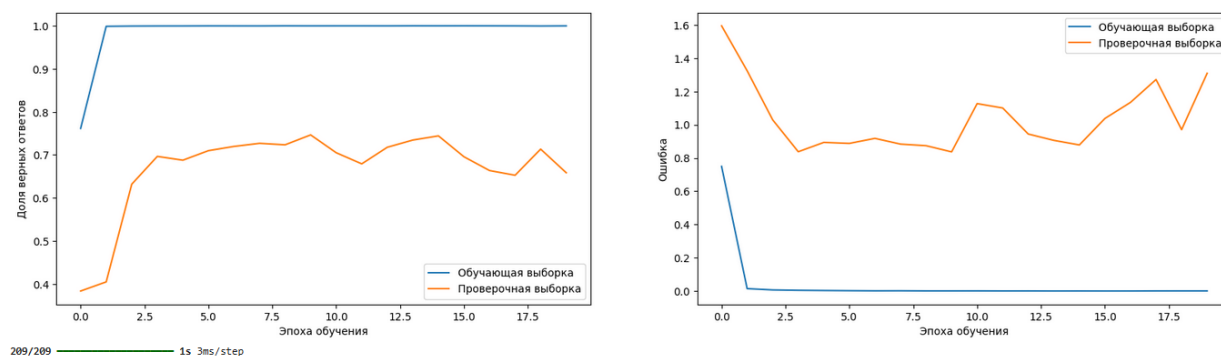


Рисунок 1.1.25 – График процесса обучения Модели 5

Графики хорошо иллюстрируют патологический характер обучения: кривая `val_accuracy` сильно колеблется и в финале составляет лишь 0,6585 (на последней эпохе), кривая `val_loss` после первоначального снижения вновь растёт от 0,84 (4-я эпоха) до 1,31 (20-я эпоха). Это явное переобучение, усугублённое связкой `Embedding+Flatten`: после `Flatten` вектор имеет размерность $1000 \times 20 = 20\,000$, и `Dense(200)` обучается находить очень специфичные позиционные паттерны в обучающей выборке, которые не обобщаются.

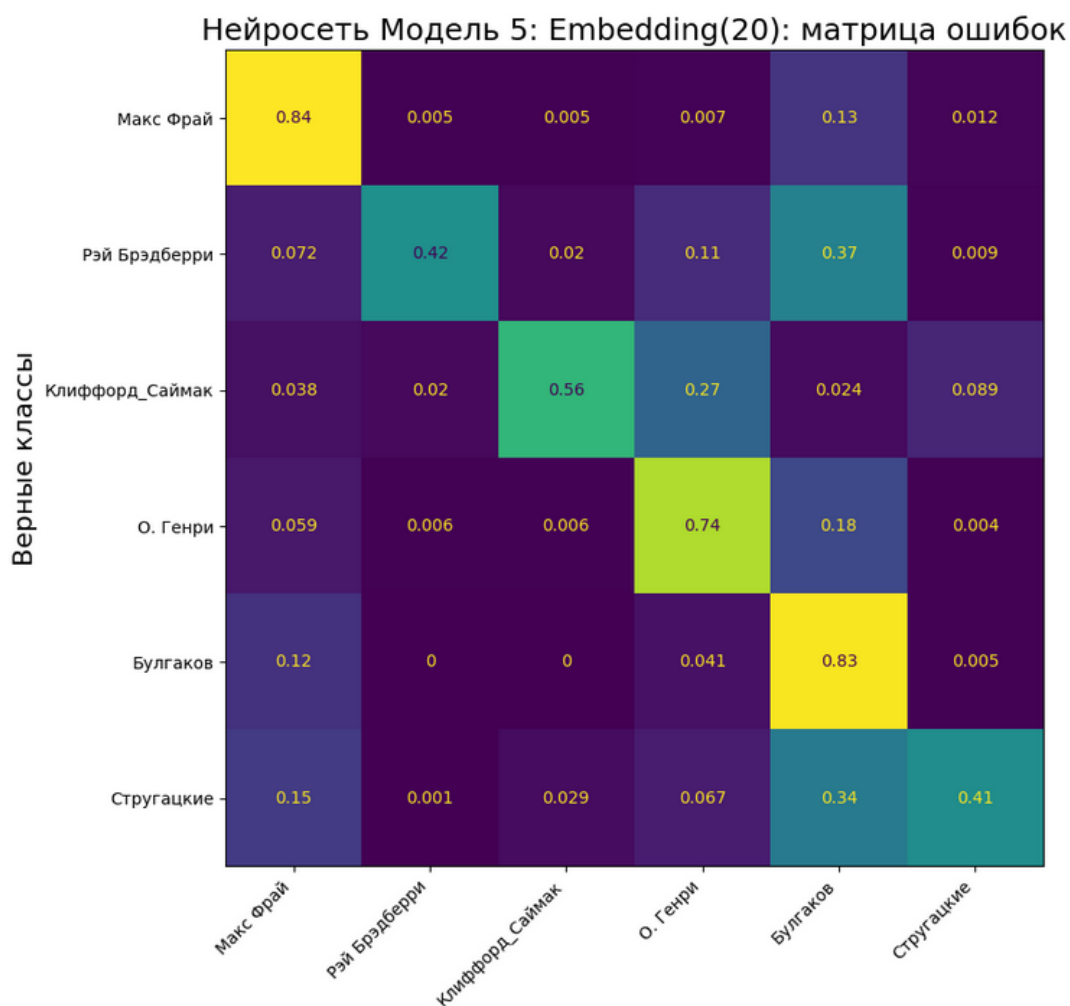


Рисунок 1.1.26 – Матрица ошибок Модели 5

Матрица ошибок наглядно показывает провал модели на нескольких классах. Только Макс Фрай (84%), О. Генри (74%) и Булгаков (83%) распознаются приемлемо. Рэй Брэдберри получает лишь 42% правильных предсказаний, причём 37% его фрагментов ошибочно отнесены к Булгакову. Стругацкие распознаются на 41%, 34% их фрагментов попадают к Булгакову. Клиффорд Саймак – 56%, 27% его фрагментов отнесены к О. Генри. Получается асимметричная картина: класс Булгакова «перетягивает» на себя примеры других авторов, что характерно для модели, не освоившей задачу полностью.

Предсказанные классы

```
-----
Нейросеть: Модель 5: Embedding(20)
Класс: Макс Фрай          84% сеть отнесла к классу Макс Фрай      - ВЕРНО :-)
Класс: Рэй Брэдберри     42% сеть отнесла к классу Рэй Брэдберри      - ВЕРНО :-)
Класс: Клиффорд Саймак   56% сеть отнесла к классу Клиффорд Саймак    - ВЕРНО :-)
Класс: О. Генри           74% сеть отнесла к классу О. Генри            - ВЕРНО :-)
Класс: Булгаков           83% сеть отнесла к классу Булгаков            - ВЕРНО :-)
Класс: Стругацкие        41% сеть отнесла к классу Стругацкие         - ВЕРНО :-)

Средняя точность распознавания: 63%
```

Рисунок 1.1.27 – Текстовый отчёт о точности распознавания Модели 5

Итоговая средняя точность распознавания Модели 5 составила 63%, что на 27–28 процентных пунктов ниже лучших BoW-моделей. Этот результат показывает, что прямолинейный переход от BoW к Embedding+Flatten без учёта особенностей обучения и без специализированных слоёв обработки последовательностей не даёт автоматического улучшения – наоборот, может существенно ухудшить качество.

1.1.17 Архитектура и обучение Модели 6: Embedding(50) + Flatten + Dense(300).

Шестая модель пытается улучшить результат Модели 5 за счёт увеличения размерности эмбединга до 50, размера скрытого слоя до 300 и более агрессивного SpatialDropout1D(0.3). Идея состоит в том, чтобы дать эмбедингам больше «выразительности» и сильнее регуляризовать модель.

Листинг 1.1.9 – Архитектура Модели 6

```
model_6 = Sequential()
model_6.add(Embedding(input_dim=VOCAB_SIZE, output_dim=50,
input_length=WIN_SIZE))
model_6.add(SpatialDropout1D(0.3))
model_6.add(Flatten())
model_6.add(BatchNormalization())
model_6.add(Dense(300, activation='relu'))
model_6.add(Dropout(0.3))
model_6.add(BatchNormalization())
model_6.add(Dense(CLASS_COUNT, activation='softmax'))
```

Модель 6: Embedding(50) + Flatten + Dense(300)

```
model_6 = Sequential()
model_6.add(Embedding(input_dim=VOCAB_SIZE, output_dim=50, input_length=WIN_SIZE))
model_6.add(SpatialDropout1D(0.3))
model_6.add(Flatten())
model_6.add(BatchNormalization())
model_6.add(Dense(300, activation='relu'))
model_6.add(Dropout(0.3))
model_6.add(BatchNormalization())
model_6.add(Dense(CLASS_COUNT, activation='softmax'))

acc_6 = compile_train_eval_model(model_6, x_train, y_train, x_test, y_test,
                                title='Модель 6: Embedding(50)')
results['Модель 6: Embedding(50) + Flatten'] = acc_6
```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input\_length` is deprecated. Just remove it.
warnings.warn(
Model: "sequential_5"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
spatial_dropout1d_1 (SpatialDropout1D)	?	0
flatten_1 (Flatten)	?	0 (unbuilt)
batch_normalization_9 (BatchNormalization)	?	0 (unbuilt)
dense_13 (Dense)	?	0 (unbuilt)
dropout_8 (Dropout)	?	0
batch_normalization_10 (BatchNormalization)	?	0 (unbuilt)
dense_14 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Epoch 1/20
138/138 — 12s 46ms/step - accuracy: 0.7771 - loss: 0.7613 - val_accuracy: 0.3832 - val_loss: 1.6423
Epoch 2/20
138/138 — 2s 11ms/step - accuracy: 0.9989 - loss: 0.0133 - val_accuracy: 0.4442 - val_loss: 1.3663
Epoch 3/20
138/138 — 2s 11ms/step - accuracy: 0.9998 - loss: 0.0060 - val_accuracy: 0.5649 - val_loss: 1.1212
Epoch 4/20
138/138 — 2s 11ms/step - accuracy: 0.9998 - loss: 0.0034 - val_accuracy: 0.6560 - val_loss: 0.9244
Epoch 5/20

Рисунок 1.1.28 – Структура Модели 6 и начало обучения

Динамика обучения схожа с Моделью 5, но с менее выраженным переобучением. Train_ассураcy быстро выходит на 100%, val_ассураcy постепенно растёт от 0,3832 (1-я эпоха) до плато в районе 0,72–0,75, на 20-й эпохе фиксируется 0,7252. Val_loss стабилизируется около 0,87–0,90, без резкого роста, что говорит о работе SpatialDropout(0.3) как ограничителя переобучения.

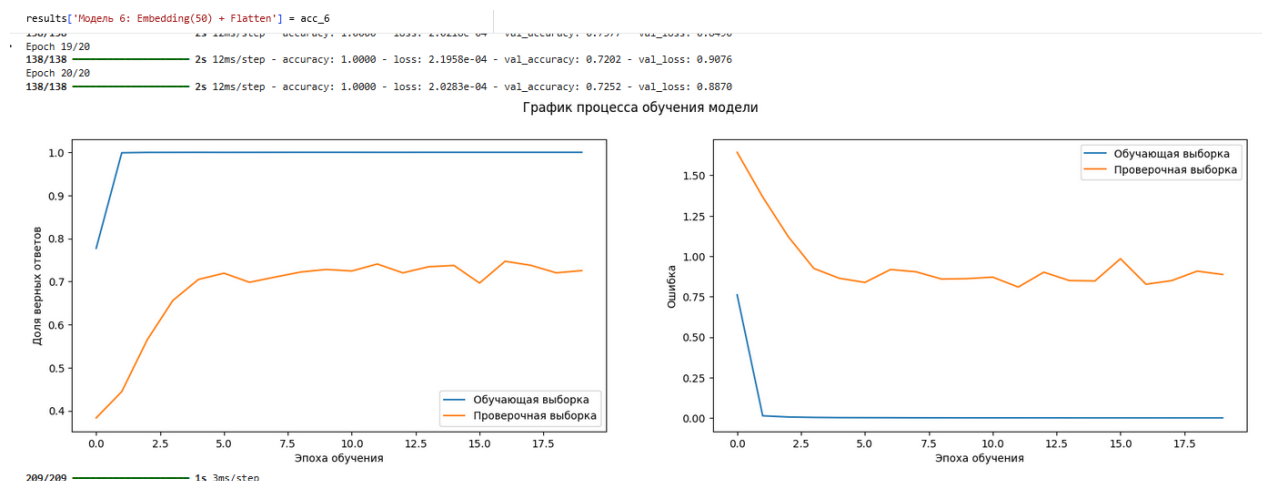


Рисунок 1.1.29 – График процесса обучения Модели 6

Графики показывают типичную для Embedding+Flatten моделей картину: train_ассурасу мгновенно достигает 1,0, val_ассурасу выходит на устойчивое плато около 0,72. Расхождение между кривыми велико, но стабильно – регуляризация удерживает модель от дальнейшей деградации, однако не способна радикально улучшить обобщение.

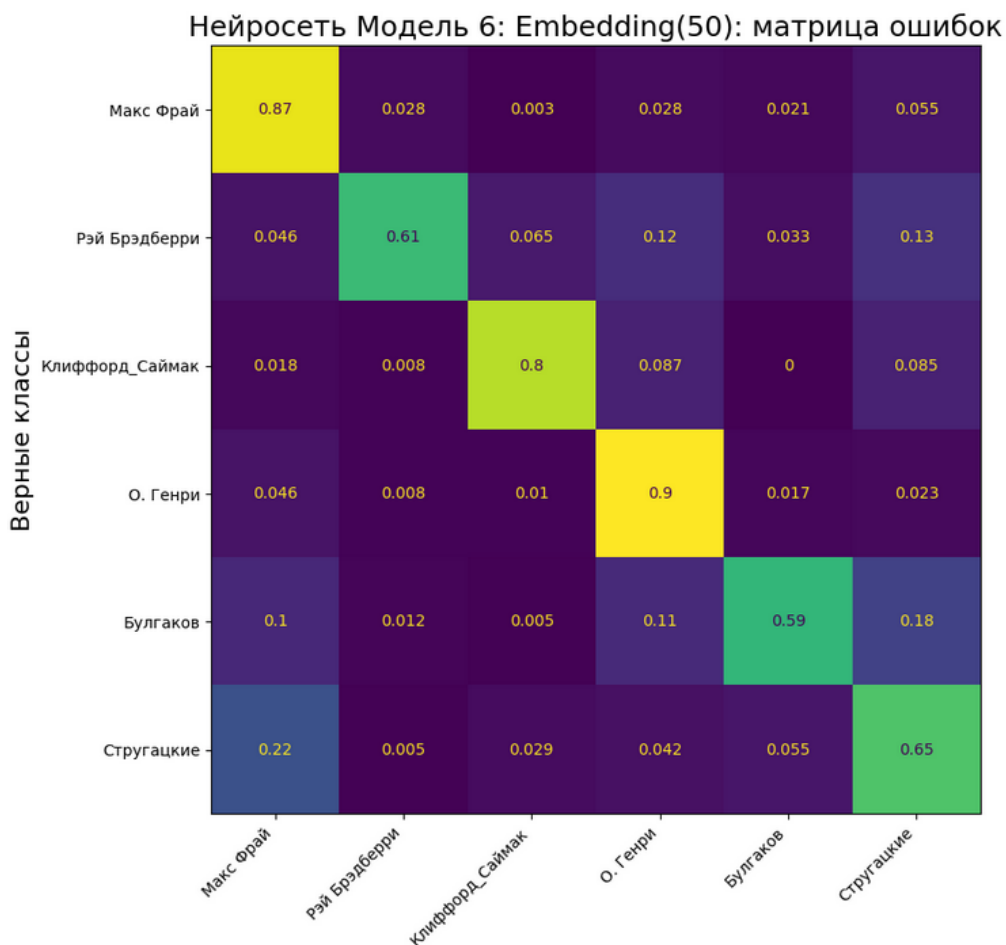


Рисунок 1.1.30 – Матрица ошибок Модели 6

Матрица ошибок Модели 6 значительно лучше, чем у Модели 5: диагональ 0,59–0,90. Лучше всего распознаются О. Генри (90%), Макс Фрай (87%) и Клиффорд Саймак (80%). Хуже всего – Булгаков (59%, 11% его фрагментов отнесены к О. Генри и 18% – к Стругацким) и Стругацкие (65%, 22% их фрагментов отнесены к Макс Фраю). Картина «перетягивания» класса, наблюдавшаяся у Модели 5, исчезла – путаницы стали более распределёнными и менее систематическими.

Предсказанные классы

Нейросеть: Модель 6: Embedding(50)

Класс: Макс Фрай	87% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	61% сеть отнесла к классу Рэй Брэдберри	- ВЕРНО :-)
Класс: Клиффорд Саймак	80% сеть отнесла к классу Клиффорд Саймак	- ВЕРНО :-)
Класс: О. Генри	90% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	59% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	65% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 74%

Рисунок 1.1.31 – Текстовый отчёт о точности распознавания Модели 6

Итоговая средняя точность распознавания Модели 6 составила 74%, что на 11 процентных пунктов лучше Модели 5. Это подтверждает, что увеличение размерности эмбединга и более агрессивная регуляризация улучшают качество в архитектурах Embedding+Flatten, однако даже Модель 6 заметно отстаёт от лучших BoW-моделей.

1.1.18 Архитектура и обучение Модели 7: Embedding(100) + Flatten + Dense(200→100).

Седьмая модель завершает серию экспериментов с Embedding+Flatten, увеличивая размерность эмбединга до 100 и добавляя второй скрытый слой Dense(100). Архитектура: Embedding(15000, 100, 1000) → SpatialDropout1D(0.2) → Flatten → BatchNormalization → Dense(200, ReLU) → Dropout(0.3) → BatchNormalization → Dense(100, ReLU) → Dropout(0.2) → BatchNormalization → Dense(6, softmax).

Листинг 1.1.10 – Архитектура Модели 7

```
model_7 = Sequential()
model_7.add(Embedding(input_dim=VOCAB_SIZE, output_dim=100,
input_length=WIN_SIZE))
```

```

model_7.add(SpatialDropout1D(0.2))
model_7.add(Flatten())
model_7.add(BatchNormalization())
model_7.add(Dense(200, activation='relu'))
model_7.add(Dropout(0.3))
model_7.add(BatchNormalization())
model_7.add(Dense(100, activation='relu'))
model_7.add(Dropout(0.2))
model_7.add(BatchNormalization())
model_7.add(Dense(CLASS_COUNT, activation='softmax'))

```

Модель 7: Embedding(100) + Flatten + Dense(200→100)

```

model_7 = Sequential()
model_7.add(Embedding(input_dim=VOCAB_SIZE, output_dim=100, input_length=WIN_SIZE))
model_7.add(SpatialDropout1D(0.2))
model_7.add(Flatten())
model_7.add(BatchNormalization())
model_7.add(Dense(200, activation='relu'))
model_7.add(Dropout(0.3))
model_7.add(BatchNormalization())
model_7.add(Dense(100, activation='relu'))
model_7.add(Dropout(0.2))
model_7.add(BatchNormalization())
model_7.add(Dense(CLASS_COUNT, activation='softmax'))

acc_7 = compile_train_eval_model(model_7, x_train, y_train, x_test, y_test,
                                title='Модель 7: Embedding(100)')
results['Модель 7: Embedding(100) + Flatten'] = acc_7

```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input\_length` is deprecated. Just remove it.
warnings.warn(
Model: "sequential_6"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
spatial_dropout1d_2 (SpatialDropout1D)	?	0
flatten_2 (Flatten)	?	0 (unbuilt)
batch_normalization_11 (BatchNormalization)	?	0 (unbuilt)
dense_15 (Dense)	?	0 (unbuilt)
dropout_9 (Dropout)	?	0
batch_normalization_12 (BatchNormalization)	?	0 (unbuilt)
dense_16 (Dense)	?	0 (unbuilt)
dropout_10 (Dropout)	?	0
batch_normalization_13 (BatchNormalization)	?	0 (unbuilt)
dense_17 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)

Рисунок 1.1.32 – Структура Модели 7

Динамика обучения улучшается по сравнению с Моделями 5 и 6: train_accuracy быстро выходит на 100%, val_accuracy плавно растёт от 0,30 (1-я эпоха) до плато около 0,77–0,78. На 20-й эпохе зафиксированы val_accuracy

= 0,7828, val_loss = 0,7601. Это лучший результат среди трёх моделей с архитектурой Embedding+Flatten.

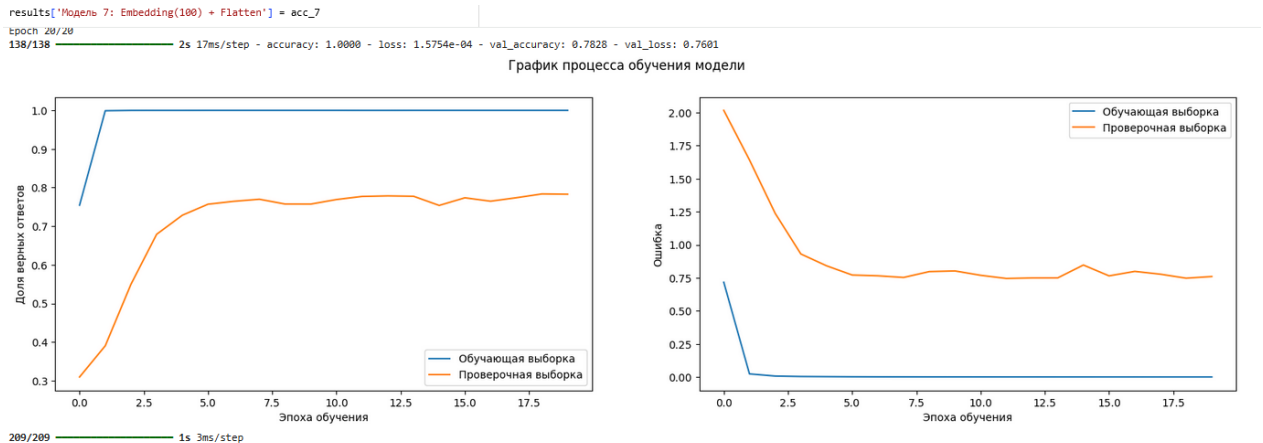


Рисунок 1.1.33 – График процесса обучения Модели 7

Графики демонстрируют более стабильное обучение, чем у Моделей 5 и 6: после быстрого выхода train_ассигасу на 1,0 кривая val_ассигасу уверенно растёт первые 5 эпох до 0,75 и далее стабилизируется на этом уровне с лёгкими колебаниями. Кривая val_loss монотонно снижается с 2,0 до 0,76, без признаков обратного роста.

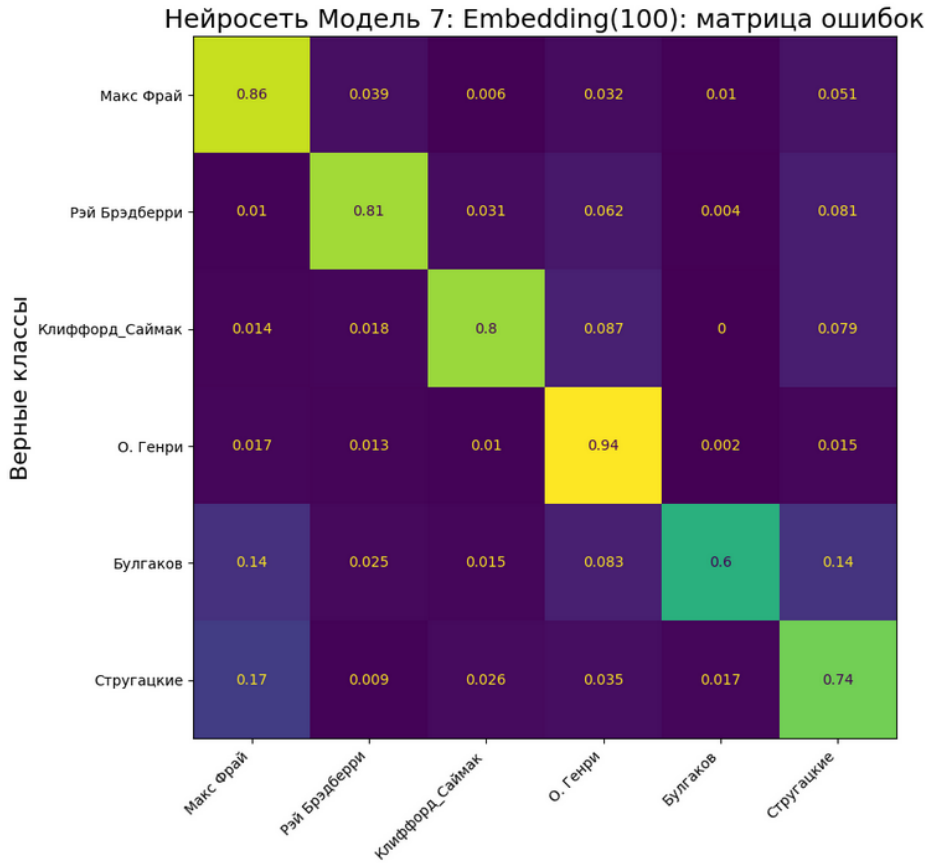


Рисунок 1.1.34 – Матрица ошибок Модели 7

Матрица ошибок Модели 7 показывает диагональ 0,60–0,94. Уверенно распознаются О. Генри (94%), Макс Фрай (86%), Рэй Брэдберри (81%) и Клиффорд Саймак (80%). Хуже распознаются Стругацкие (74%) и особенно Булгаков (60%, 14% его фрагментов попадают к Макс Фраю, 14% – к Стругацким и 8,3% – к О. Генри). По сравнению с Моделями 5 и 6 общая картина становится более сбалансированной: исчезли явные «провалы» по отдельным классам.

Итоговая средняя точность распознавания Модели 7 составила 79,3%, что является вторым результатом среди всех Embedding+Flatten моделей. Однако даже этот результат отстаёт от лучшей BoW-модели (Модель 3: 91,4%) на 12 процентных пунктов, что подтверждает: для задачи определения авторства простое сочетание эмбедингов с Flatten не способно превзойти Bag of Words. Для эффективного использования эмбедингов необходимы специализированные слои обработки последовательностей – LSTM, GRU или Conv1D.

1.1.19 Архитектура и обучение Модели 8: Embedding(200) + Flatten + Dense(200) + Dropout(0.5).

Восьмая модель является развитием Модели 7 с максимальной размерностью эмбединга среди серии Embedding+Flatten экспериментов: output_dim=200. Вводятся также самый агрессивный SpatialDropout1D(0.4) и Dropout(0.5) – для попытки ограничить переобучение на высокоразмерных эмбедингах. Архитектура: Embedding(15000, 200, 1000) → SpatialDropout1D(0.4) → Flatten → BatchNormalization → Dense(200, ReLU) → Dropout(0.5) → BatchNormalization → Dense(6, softmax).

Листинг 1.1.11 – Архитектура Модели 8

```
model_8 = Sequential()
model_8.add(Embedding(input_dim=VOCAB_SIZE, output_dim=200,
input_length=WIN_SIZE))
model_8.add(SpatialDropout1D(0.4))
model_8.add(Flatten())
model_8.add(BatchNormalization())
```

```

model_8.add(Dense(200, activation='relu'))
model_8.add(Dropout(0.5))
model_8.add(BatchNormalization())
model_8.add(Dense(CLASS_COUNT, activation='softmax'))

```

Модель 8: Embedding(200) + Flatten + Dense(200) + Dropout(0.5)

```

model_8 = Sequential()
model_8.add(Embedding(input_dim=VOCAB_SIZE, output_dim=200, input_length=WIN_SIZE))
model_8.add(SpatialDropout1D(0.4))
model_8.add(Flatten())
model_8.add(BatchNormalization())
model_8.add(Dense(200, activation='relu'))
model_8.add(Dropout(0.5))
model_8.add(BatchNormalization())
model_8.add(Dense(CLASS_COUNT, activation='softmax'))

acc_8 = compile_train_eval_model(model_8, x_train, y_train, x_test, y_test,
                                title='Модель 8: Embedding(200)')
results['Модель 8: Embedding(200) + Dropout(0.5)'] = acc_8

```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input\_length` is deprecated. Just remove it.
warnings.warn(
Model: "sequential_7"

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	?	0 (unbuilt)
spatial_dropout1d_3 (SpatialDropout1D)	?	0
flatten_3 (Flatten)	?	0 (unbuilt)
batch_normalization_14 (BatchNormalization)	?	0 (unbuilt)
dense_18 (Dense)	?	0 (unbuilt)
dropout_11 (Dropout)	?	0
batch_normalization_15 (BatchNormalization)	?	0 (unbuilt)
dense_19 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Epoch 1/20
138/138 — 16s 69ms/step - accuracy: 0.8013 - loss: 0.6471 - val_accuracy: 0.2929 - val_loss: 2.0623
Epoch 2/20
138/138 — 12s 31ms/step - accuracy: 0.9994 - loss: 0.0191 - val_accuracy: 0.3824 - val_loss: 1.3644
Epoch 3/20
138/138 — 4s 30ms/step - accuracy: 1.0000 - loss: 0.0069 - val_accuracy: 0.6418 - val_loss: 1.0133
Epoch 4/20
138/138 — 4s 31ms/step - accuracy: 0.9999 - loss: 0.0041 - val_accuracy: 0.7212 - val_loss: 0.8208

Рисунок 1.1.35 – Структура и начало обучения Модели 8

Процесс обучения отличается от предыдущих Embedding+Flatten моделей более медленным стартом: на 1-й эпохе train_accracy = 0,8013 (у Моделей 5–7 было около 0,76–0,80), что объясняется сильным SpatialDropout1D(0.4). Val_accracy растёт монотонно: 0,2929 (1-я эпоха) → 0,6418 (3-я) → 0,7212 (4-я) → плато около 0,78–0,80. На 20-й эпохе достигнуты val_accracy = 0,7810, val_loss = 0,6820 – это лучший результат на проверочной выборке среди всех Embedding+Flatten моделей.

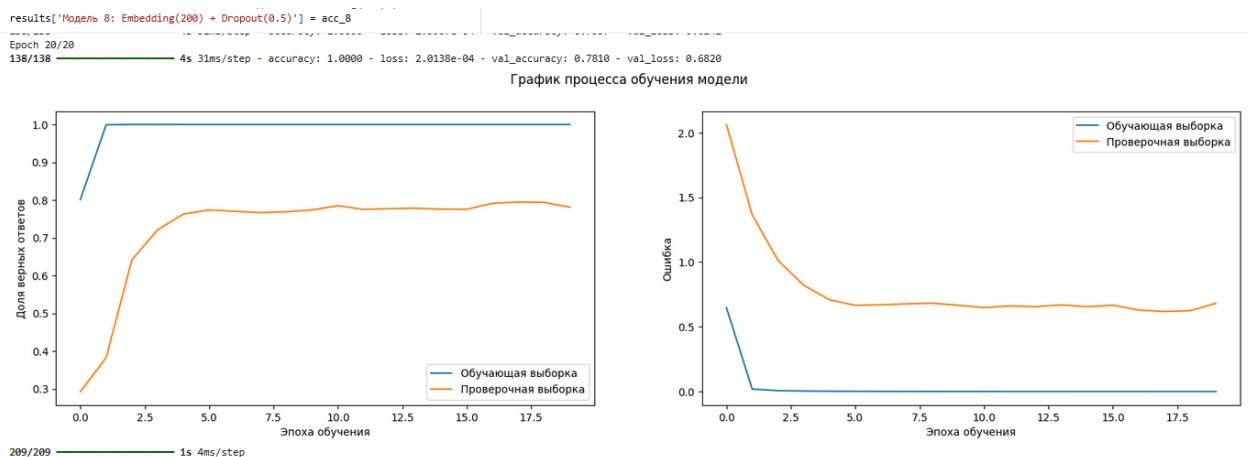


Рисунок 1.1.36 – График процесса обучения Модели 8

Графики демонстрируют наиболее стабильное среди Embedding-моделей поведение: после быстрого насыщения train_accracy кривая val_accracy плавно растёт и выходит на плато около 0,79 к 7-й эпохе и далее остаётся стабильной. Val_loss снижается с 2,06 до 0,68 без обратного роста. Отличие от Модели 7 заключается в более высокой финальной точности и большей стабильности val_loss, что свидетельствует о лучшем действии регуляризации.

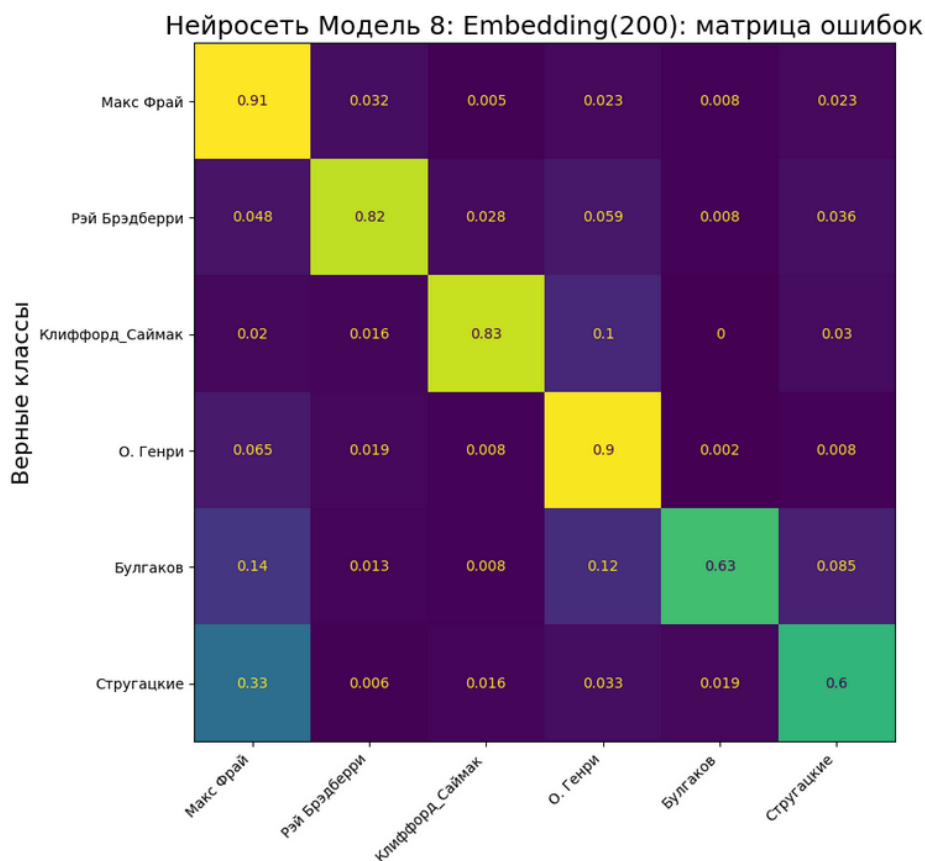


Рисунок 1.1.37 – Матрица ошибок Модели 8

Матрица ошибок показывает диагональ 0,60–0,91. Лучше всего распознаются Макс Фрай (91%) и О. Генри (90%). Хуже всего – Булгаков (63%, 14% его фрагментов отнесены к Макс Фраю и 12% – к О. Генри) и Стругацкие (60%, 33% их фрагментов ошибочно классифицированы как Макс Фрай). Путаница Стругацкие→Макс Фрай является самой значительной систематической ошибкой модели и может объясняться особенностями лексики этих авторов, частично пересекающейся в жанровом плане (отечественная фантастика и фэнтези).

Предсказанные классы

```
-----
Нейросеть: Модель 8: Embedding(200)
Класс: Макс Фрай          91% сеть отнесла к классу Макс Фрай          - ВЕРНО :-)
Класс: Рэй Брэдберри     82% сеть отнесла к классу Рэй Брэдберри       - ВЕРНО :-)
Класс: Клиффорд Саймак   83% сеть отнесла к классу Клиффорд Саймак     - ВЕРНО :-)
Класс: О. Генри          90% сеть отнесла к классу О. Генри              - ВЕРНО :-)
Класс: Булгаков          63% сеть отнесла к классу Булгаков              - ВЕРНО :-)
Класс: Стругацкие        60% сеть отнесла к классу Стругацкие          - ВЕРНО :-)

Средняя точность распознавания: 78%
```

Рисунок 1.1.38 – Текстовый отчёт о точности распознавания Модели 8

Итоговая средняя точность распознавания Модели 8 составила 78,2%, что является лучшим результатом среди всех Embedding+Flatten моделей. Ключевой вклад в улучшение по сравнению с Моделями 5–7 внесло сочетание высокоразмерных эмбедингов (200) с агрессивной двухуровневой регуляризацией: SpatialDropout1D(0.4) – на уровне эмбедингов и Dropout(0.5) – на уровне Dense-слоя.

1.1.20 Архитектура и обучение Модели 9: Embedding(200) + Flatten без регуляризации Dropout.

Девятая модель является контрольным экспериментом: из архитектуры Модели 8 полностью убраны все слои Dropout (как SpatialDropout1D, так и Dropout после Dense), при том что сам слой Embedding(200) сохраняется. Цель – экспериментально оценить вклад регуляризации Dropout в итоговое качество Embedding+Flatten моделей.

Листинг 1.1.12 – Архитектура Модели 9

```
model_9 = Sequential()
```

```

model_9.add(Embedding(input_dim=VOCAB_SIZE, output_dim=200,
input_length=WIN_SIZE))
model_9.add(Flatten())
model_9.add(BatchNormalization())
model_9.add(Dense(200, activation='relu'))
model_9.add(BatchNormalization())
model_9.add(Dense(CLASS_COUNT, activation='softmax'))

```

Модель 9: Embedding(200) + Flatten без регуляризации Dropout

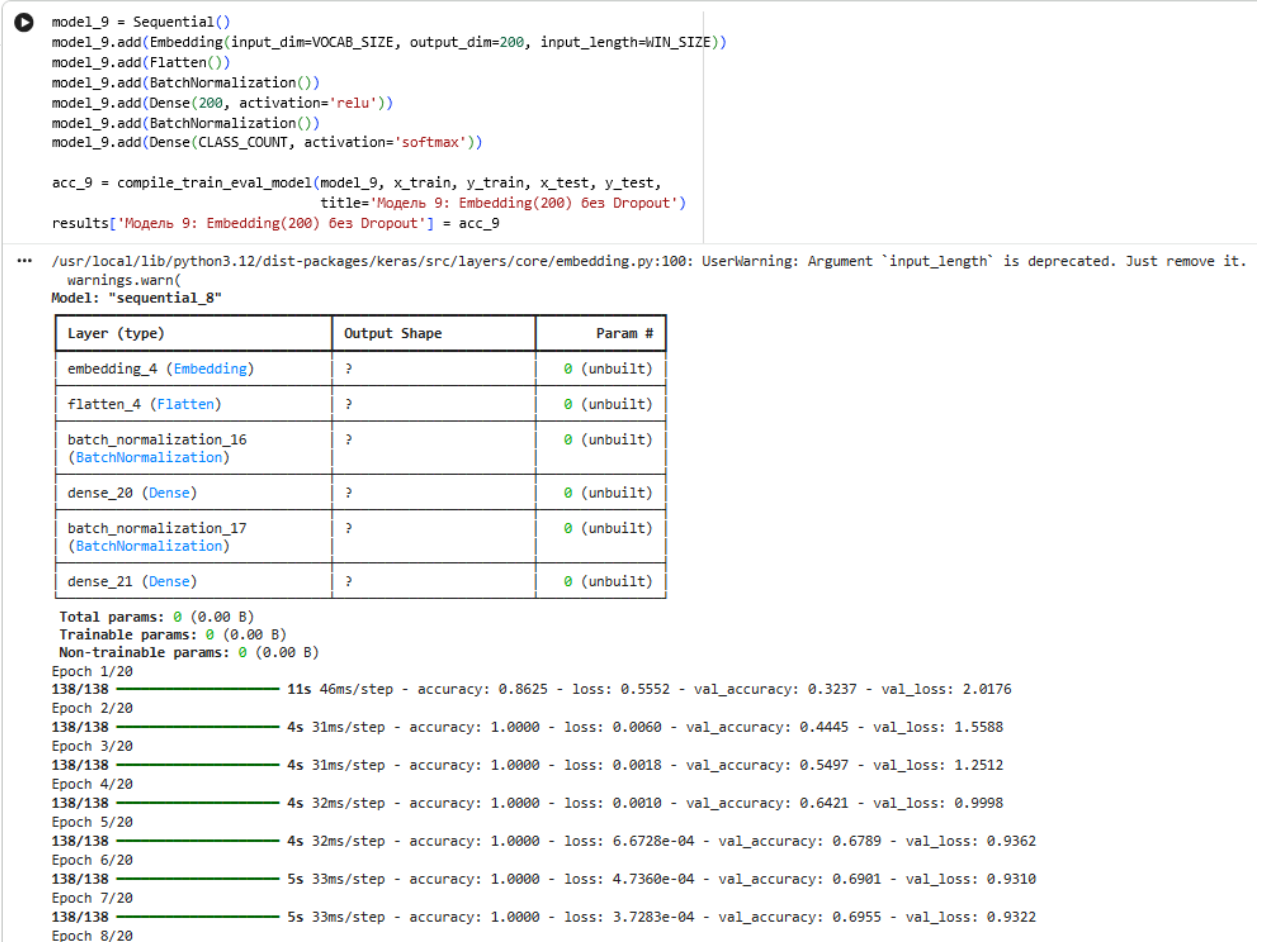


Рисунок 1.1.39 – Структура и начало обучения Модели 9

Поведение Модели 9 подтверждает значимость Dropout: без регуляризации train_ассурасу достигает 100% уже ко 2-й эпохе, а val_ассурасу выходит на значительно более низкое плато, чем у Модели 8. Финальные значения: val_ассурасу = 0,7125, val_loss = 0,9431. Val_loss после снижения с 2,02 до 0,93 стабилизируется на этом уровне к 5-й эпохе и далее практически

не меняется, что является признаком «насыщения» обобщающей способности модели без регуляризации.

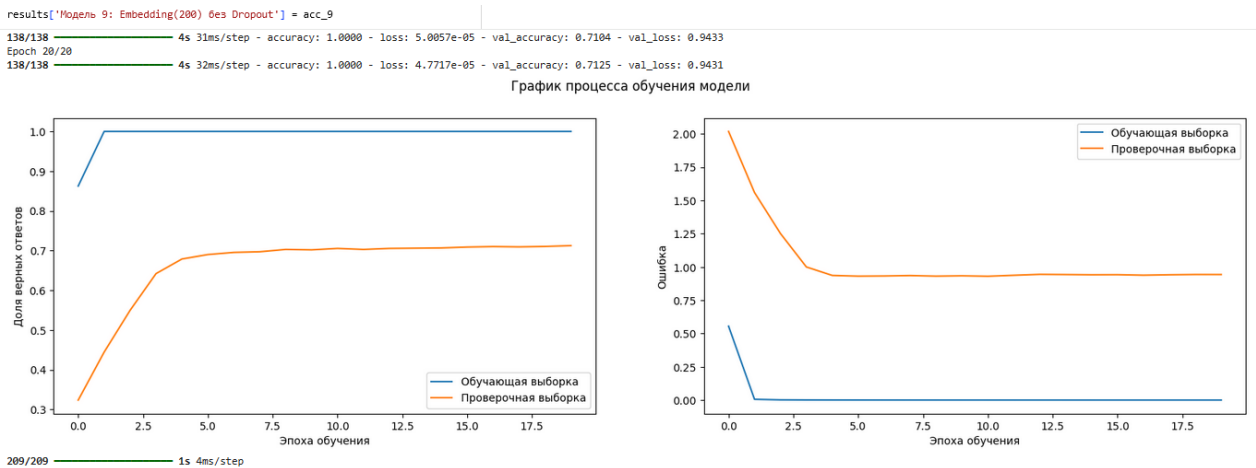


Рисунок 1.1.40 – График процесса обучения Модели 9

Графики демонстрируют классическую картину отсутствия регуляризации: train_ассигасу мгновенно уходит к 1,0, val_ассигасу выходит на плато около 0,70 уже к 5-й эпохе и далее не улучшается (в отличие от Модели 8, где плавный рост val_ассигасу продолжался до 7–10 эпохи). Это подтверждает, что SpatialDropout1D и Dropout не просто замедляют переобучение, но и позволяют модели найти более обобщающее решение за счёт имплицитной ансамблизации.

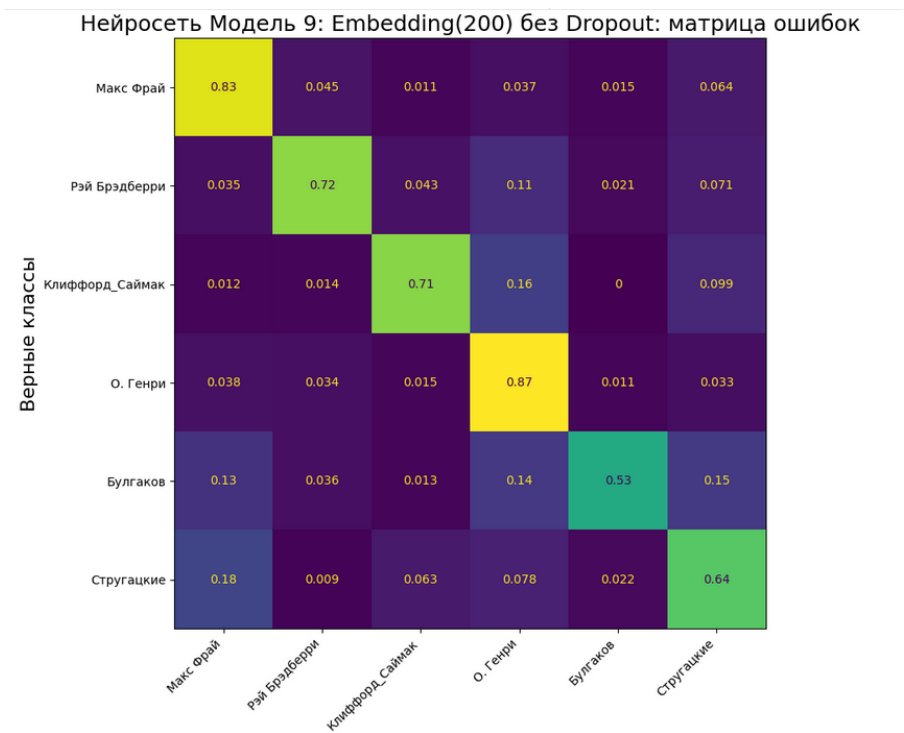


Рисунок 1.1.41 – Матрица ошибок Модели 9

Матрица ошибок Модели 9 показывает диагональ 0,53–0,87, что ниже, чем у Модели 8 (0,60–0,91). Наиболее уверенно распознаётся О. Генри (87%), хуже всего – Булгаков (53%, 14% отнесены к О. Генри, 15% – к Стругацким). Распределение ошибок в целом менее систематическое, чем у Модели 8, и это соответствует хаотичности переобучения – без регуляризации сеть запоминает случайные особенности обучающей выборки, которые плохо коррелируют с верными атрибуциями на тестовой.

Предсказанные классы

```
-----
Нейросеть: Модель 9: Embedding(200) без Dropout
Класс: Макс Фрай          83% сеть отнесла к классу Макс Фрай          - ВЕРНО :-)
Класс: Рэй Брэдберри     72% сеть отнесла к классу Рэй Брэдберри          - ВЕРНО :-)
Класс: Клиффорд_Саймак   71% сеть отнесла к классу Клиффорд_Саймак        - ВЕРНО :-)
Класс: О. Генри          87% сеть отнесла к классу О. Генри                  - ВЕРНО :-)
Класс: Булгаков          53% сеть отнесла к классу Булгаков                  - ВЕРНО :-)
Класс: Стругацкие        64% сеть отнесла к классу Стругацкие                - ВЕРНО :-)

Средняя точность распознавания: 72%
```

Рисунок 1.1.42 – Текстовый отчёт о точности распознавания Модели 9

Итоговая средняя точность распознавания Модели 9 составила 71,7%, что на 6,5 процентного пункта ниже Модели 8 с регуляризацией. Это наглядно количественно подтверждает: SpatialDropout1D и Dropout вносят существенный вклад в обобщающую способность моделей Embedding+Flatten.

1.1.21 Итоги сравнения девяти моделей.

После обучения всех девяти моделей формируется сводная таблица результатов и строится столбчатый сравнительный график. Модели 1–4 (BoW) выделены синим цветом, Модели 5–9 (Embedding+Flatten) – зелёным.

Листинг 1.1.13 – Формирование сводной таблицы результатов

```
print('СВОДНАЯ ТАБЛИЦА ТОЧНОСТИ')
for name, acc in results.items():
    print(f'{name:<50} → {acc:5.1f}%')
best_name = max(results, key=results.get)
print(f'\nЛучшая модель: {best_name}')
print(f' Точность: {results[best_name]:.1f}%')
```

Таблица 1.1.1 – Сводные результаты точности распознавания девяти моделей
(синим выделены BoW-модели, зелёным – Embedding+Flatten)

Модель 1: BoW Dense(200)	90,0%
Модель 2: BoW Dense(400)	88,0%
Модель 3: BoW Dense(300→100)	91,4% ← лучшая
Модель 4: BoW Dense(500→200→50)	89,8%
Модель 5: Embedding(20) + Flatten	63,4%
Модель 6: Embedding(50) + Flatten	73,7%
Модель 7: Embedding(100) + Flatten	79,3%
Модель 8: Embedding(200) + Dropout(0.5)	78,2%
Модель 9: Embedding(200) без Dropout	71,7%

```
fig, ax = plt.subplots(figsize=(14, 6))
short_names = [f'Модель {i+1}' for i in range(len(results))]
values = list(results.values())
colors = ['steelblue']*4 + ['green']*5

bars = ax.bar(short_names, values, color=colors)
ax.set_ylabel('Точность, %', fontsize=12)
ax.set_title('Сравнение 9 моделей', fontsize=14)
ax.set_ylim(0, 100)
ax.grid(axis='y', linestyle='--', alpha=0.5)

for bar, val in zip(bars, values):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
            f'{val:.1f}%', ha='center', fontsize=10)

plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

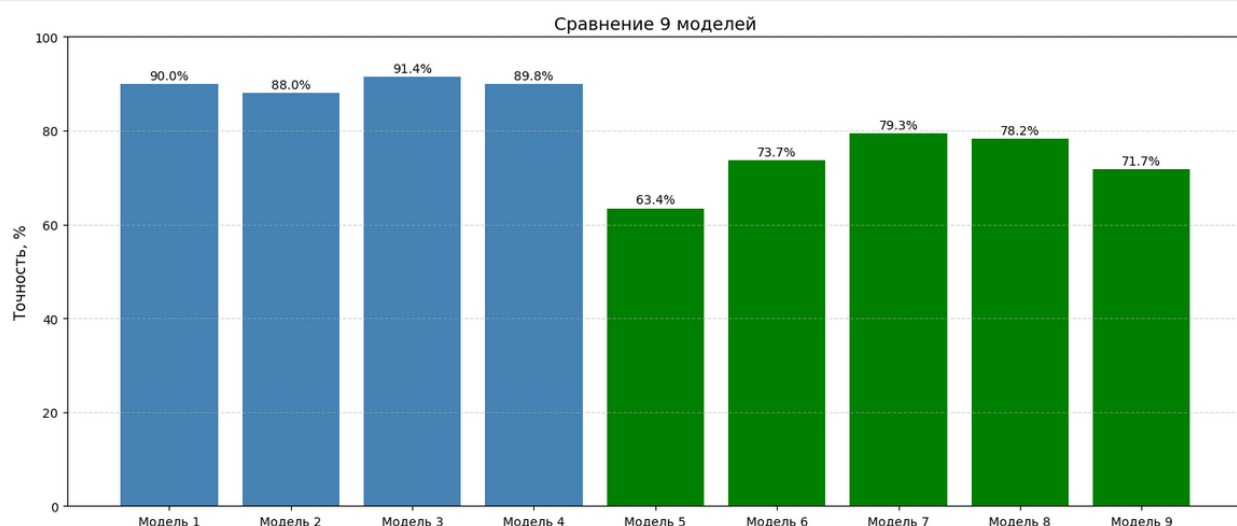


Рисунок 1.1.43 – Столбчатый сравнительный график точности девяти моделей

Сводная таблица подтверждает следующие закономерности. Лучшую точность показала Модель 3: BoW Dense(300→100) с результатом 91,4%. Среди BoW-моделей (1–4) наблюдается монотонная зависимость: небольшое углубление архитектуры до двух скрытых слоёв (Модель 3) улучшило результат, тогда как дальнейшее наращивание параметров (Модель 4, три слоя, 7,6 млн параметров) привело к переобучению и снижению точности до 89,8%. Все четыре BoW-модели превзошли все пять Embedding+Flatten моделей, диапазон точности которых составил 63,4–79,3%. В группе Embedding+Flatten прослеживается зависимость: увеличение размерности эмбединга улучшает качество (63,4% при $\text{dim}=20$ против 79,3% при $\text{dim}=100$), но эффект насыщается при $\text{dim}=200$ без Dropout (71,7%) и лишь агрессивная двухуровневая регуляризация позволяет реализовать потенциал высокоразмерных эмбедингов (78,2% у Модели 8).

Выводы: в ходе выполнения задания были построены и сравнены девять нейронных сетей для классификации авторства текстов шести русских писателей. Выявлено два принципиальных типа поведения: модели на основе Bag of Words (Модели 1–4) показали стабильно высокую точность 88–91,4% благодаря простому, но эффективному представлению текста через частоты слов; модели Embedding+Flatten (Модели 5–9) дали результаты ниже (63–79%), поскольку без специализированных слоёв обработки последовательностей LSTM, GRU или Conv1D архитектура Embedding+Flatten не способна эффективно учесть порядок слов и переобучается за счёт большой размерности вектора после Flatten ($1000 \times \text{dim}$). Лучшая модель эксперимента – Модель 3 (BoW, Dense(300→100), 91,4%), которая будет использована как ориентир при сравнении с архитектурами на основе рекуррентных и свёрточных слоёв.

1.2 Реализация средствами библиотеки PyTorch.

1.2.1 Подключение необходимых для работы библиотек.

Для реализации нейронных сетей на PyTorch подключается набор необходимых библиотек. Из стандартных инструментов подключаются:

numpy (работа с массивами), os, time, re, pickle (функции операционной системы и сериализации). Из фреймворка PyTorch используются: torch, torch.nn (построение слоёв нейросети), torch.optim (оптимизаторы), а также Dataset, DataLoader и TensorDataset из torch.utils.data для формирования батчей. Токенизация выполняется при помощи класса Tokenizer из tensorflow.keras.preprocessing.text – для совместимости с подходом из раздела 1.1: тот же словарь, те же параметры фильтрации. Для построения матрицы ошибок применяются confusion_matrix и ConfusionMatrixDisplay из sklearn.metrics. Загрузка датасета из облака производится через gdown, визуализация – через matplotlib.pyplot. Дополнительно определяется устройство выполнения вычислений: при наличии видеокарты используется cuda, иначе – cpu.

Импорт библиотек

```
▶ # Работа с массивами данных
import numpy as np

# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, TensorDataset

# Токенизация с помощью Keras (только для совместимости с практикой,
# сама модель и обучение – на PyTorch)
from tensorflow.keras.preprocessing.text import Tokenizer

# Матрица ошибок классификатора
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Загрузка датасетов из облака google
import gdown

# Функции операционной системы
import os
import time
import re
import pickle

# Отрисовка графиков
import matplotlib.pyplot as plt

%matplotlib inline

# Определение устройства (GPU/CPU)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Используем устройство: {device}')

# Фиксируем сид для воспроизводимости
torch.manual_seed(42)
np.random.seed(42)
```

*** Используем устройство: cuda

Загрузка датасета и подготовка данных

```
# Загрузка датасета из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/17/writers.zip', None, quiet=True)

'writers.zip'

# Распаковка архива
!unzip -qo writers.zip -d writers/
!ls writers

'(Булгаков) Обучающая_5 вместе.txt'
'(Булгаков) Тестовая_2 вместе.txt'
'(Клиффорд_Саймак) Обучающая_5 вместе.txt'
'(Клиффорд_Саймак) Тестовая_2 вместе.txt'
'(Макс Фрай) Обучающая_5 вместе.txt'
'(Макс Фрай) Тестовая_2 вместе.txt'
'(О. Генри) Обучающая_50 вместе.txt'
'(О. Генри) Тестовая_20 вместе.txt'
'(Рэй Брэдберри) Обучающая_22 вместе.txt'
'(Рэй Брэдберри) Тестовая_8 вместе.txt'
'(Стругацкие) Обучающая_5 вместе.txt'
'(Стругацкие) Тестовая_2 вместе.txt'

# Настройка констант
FILE_DIR = 'writers'
SIG_TRAIN = 'обучающая'
SIG_TEST = 'тестовая'

# Загрузка датасета
CLASS_LIST = []
text_train = []
text_test = []

for file_name in os.listdir(FILE_DIR):
    m = re.match(r'\((.+)\) (\S+)', file_name)
    if m:
        class_name = m[1]
        subset_name = m[2].lower()
        is_train = SIG_TRAIN in subset_name
        is_test = SIG_TEST in subset_name

        if is_train or is_test:
            if class_name not in CLASS_LIST:
                print(f'Добавление класса "{class_name}"')
                CLASS_LIST.append(class_name)
                text_train.append('')
                text_test.append('')
```

Терминал

Рисунок 1.2.1 – Подключение библиотек и определение устройства вычислений

1.2.2 Загрузка датасета и подготовка данных.

Загрузка и подготовка данных в реализации на PyTorch выполняется аналогично разделу 1.1. Архив writers.zip скачивается с хранилища YandexCloud при помощи gdown и распаковывается в директорию writers/. Константы FILE_DIR = 'writers', SIG_TRAIN = 'обучающая' и SIG_TEST = 'тестовая' задают имя директории и маркеры выборок. Цикл по файлам директории с применением регулярного выражения извлекает имя автора и

тип выборки, после чего содержимое каждого файла дописывается к соответствующей строке text_train или text_test. Итоговое количество классов CLASS_COUNT = 6.

```
cls = CLASS_LIST.index(class_name)
print(f'Добавление файла "{file_name}" в класс "{CLASS_LIST[cls]}"')
with open(f'{FILE_DIR}/{file_name}', 'r') as f:
    text = f.read()
subset = text_train if is_train else text_test
subset[cls] += ' ' + text.replace('\n', ' ')

CLASS_COUNT = len(CLASS_LIST)
print(f'\nКлассов: {CLASS_COUNT}')
```

Добавление класса "Макс Фрай"
Добавление файла "(Макс Фрай) Тестовая_2 вместе.txt" в класс "Макс Фрай"
Добавление класса "Рэй Брэдберри"
Добавление файла "(Рэй Брэдберри) Обучающая_22 вместе.txt" в класс "Рэй Брэдберри"
Добавление класса "Клиффорд Саймак"
Добавление файла "(Клиффорд Саймак) Обучающая_5 вместе.txt" в класс "Клиффорд Саймак"
Добавление класса "О. Генри"
Добавление файла "(О. Генри) Обучающая_50 вместе.txt" в класс "О. Генри"
Добавление класса "Булгаков"
Добавление файла "(Булгаков) Обучающая_5 вместе.txt" в класс "Булгаков"
Добавление файла "(О. Генри) Тестовая_20 вместе.txt" в класс "О. Генри"
Добавление файла "(Булгаков) Тестовая_2 вместе.txt" в класс "Булгаков"
Добавление файла "(Рэй Брэдберри) Тестовая_8 вместе.txt" в класс "Рэй Брэдберри"
Добавление файла "(Макс Фрай) Обучающая_5 вместе.txt" в класс "Макс Фрай"
Добавление класса "Стругацкие"
Добавление файла "(Стругацкие) Тестовая_2 вместе.txt" в класс "Стругацкие"
Добавление файла "(Стругацкие) Обучающая_5 вместе.txt" в класс "Стругацкие"
Добавление файла "(Клиффорд Саймак) Тестовая_2 вместе.txt" в класс "Клиффорд Саймак"

Классов: 6

Параметры преобразования

```
# Параметры преобразования
VOCAB_SIZE = 15000
WIN_SIZE   = 1000
WIN_HOP    = 100
```

Рисунок 1.2.2 – Загрузка датасета и формирование классов

1.2.3 Параметры преобразования, токенизация и формирование выборок.

Параметры преобразования совпадают с разделом 1.1: VOCAB_SIZE = 15000, WIN_SIZE = 1000, WIN_HOP = 100. Токенизация выполняется классом Tokenizer из Keras с теми же параметрами фильтрации символов, lower=True и oov_token='неизвестное_слово'. Метод fit_on_texts строит частотный словарь по обучающей выборке (полный размер 133 070 слов, время ~3 с). Затем

тексты обеих выборок преобразуются в последовательности индексов методом `texts_to_sequences`.

Формирование выборок методом скользящего окна реализовано функциями `split_sequence` (нарезка последовательности) и `vectorize_sequence` (обход всех классов). В отличие от реализации на Keras, метки классов сохраняются в виде целочисленных индексов (не One-Hot), что соответствует `CrossEntropyLoss` в PyTorch. Размерности совпадают с разделом 1.1: обучающая выборка – 17 640 отрезков по 1000 слов, тестовая – 6 686 отрезков. Для BoW-моделей дополнительно формируются матрицы счётчиков через `tokenizer.sequences_to_matrix` (форма 17640×15000 и 6686×15000 , время ~9 с).

Токенизация и построение частотного словаря

Используем `Tokenizer` из Keras для совместимости с подходом. Сама модель и обучение — на PyTorch.

```
class timex:
    def __enter__(self):
        self.t = time.time()
        return self
    def __exit__(self, type, value, traceback):
        print('Время обработки: {:.2f} с'.format(time.time() - self.t))
```

```
# Токенизация
with timex():
    tokenizer = Tokenizer(num_words=VOCAB_SIZE,
                          filters='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~\t\n\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\x1000',
                          lower=True, split=' ',
                          oov_token='неизвестное_слово',
                          char_level=False)
    tokenizer.fit_on_texts(text_train)
    items = list(tokenizer.word_index.items())

print('Размер словаря:', len(items))
```

Время обработки: 3.08 с
Размер словаря: 133070

```
# Преобразование в последовательности индексов
with timex():
    seq_train = tokenizer.texts_to_sequences(text_train)
    seq_test = tokenizer.texts_to_sequences(text_test)
```

Время обработки: 4.44 с

Рисунок 1.2.3 – Токенизация и построение частотного словаря

Формирование выборок методом скользящего окна

```
def split_sequence(sequence, win_size, hop):
    return [sequence[i:i + win_size] for i in range(0, len(sequence) - win_size + 1, hop)]

def vectorize_sequence(seq_list, win_size, hop):
    class_count = len(seq_list)
    x, y = [], []
    for cls in range(class_count):
        vectors = split_sequence(seq_list[cls], win_size, hop)
        x += vectors
        # В PyTorch метки классов идут просто как индексы (не one-hot)
        # Это эквивалент sparse_categorical_crossentropy в Keras
        y += [cls] * len(vectors)
    return np.array(x), np.array(y)
```

```
# Формирование выборок
with timex():
    x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
    x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)
    print(f'Обучающая: {x_train.shape}, метки: {y_train.shape}')
    print(f'Тестовая: {x_test.shape}, метки: {y_test.shape}')
```

Обучающая: (17640, 1000), метки: (17640,)
Тестовая: (6686, 1000), метки: (6686,)
Время обработки: 2.63 с

```
# Формирование Bag of Words матриц через токенизатор
with timex():
    x_train_01 = tokenizer.sequences_to_matrix(x_train.tolist())
    x_test_01 = tokenizer.sequences_to_matrix(x_test.tolist())
    print(f'BoW обучающая: {x_train_01.shape}')
```

BoW обучающая: (17640, 15000)
Время обработки: 8.93 с

Рисунок 1.2.4 – Формирование выборок методом скользящего окна и BoW-матриц

1.2.4 PyTorch Dataset и DataLoader.

В отличие от Keras, где данные передаются напрямую в метод `fit`, в PyTorch подача обучающих примеров организована через `DataLoader`, который собирает батчи из объекта `Dataset`. Матрицы и последовательности преобразуются в тензоры PyTorch: BoW-матрицы – в `FloatTensor (float32)`, последовательности индексов – в `LongTensor (int64)`, метки классов – в `LongTensor`. Тензоры упаковываются в `TensorDataset`, из которого `DataLoader` формирует батчи размером `BATCH_SIZE = 128`. Для обучающих загрузчиков используется перемешивание `shuffle=True`, для тестовых – `shuffle=False`.

Итоговое количество батчей в обучающем BoW-загрузчике составило 138, в тестовом – 53.

PyTorch Dataset и DataLoader

В PyTorch нет аналога `model.fit()` — данные подаём через `DataLoader`, который собирает батчи из объекта `Dataset`.

```
# Конвертация в тензоры PyTorch
# Для BoW моделей
x_train_bow_t = torch.FloatTensor(x_train_01)
x_test_bow_t = torch.FloatTensor(x_test_01)

# Для Embedding моделей (целочисленные индексы слов)
x_train_emb_t = torch.LongTensor(x_train)
x_test_emb_t = torch.LongTensor(x_test)

# Метки классов (общие для обоих типов моделей)
y_train_t = torch.LongTensor(y_train)
y_test_t = torch.LongTensor(y_test)

print('Тензоры готовы:')
print(f' BoW обучающая: {x_train_bow_t.shape}, тип {x_train_bow_t.dtype}')
print(f' Embedding обучающая: {x_train_emb_t.shape}, тип {x_train_emb_t.dtype}')
print(f' Метки: {y_train_t.shape}, тип {y_train_t.dtype}')
```

```
Тензоры готовы:
BoW обучающая: torch.Size([17640, 15000]), тип torch.float32
Embedding обучающая: torch.Size([17640, 1000]), тип torch.int64
Метки: torch.Size([17640]), тип torch.int64
```

Рисунок 1.2.5 – Конвертация данных в тензоры и создание DataLoader

```
# DataLoader для BoW
BATCH_SIZE = 128

train_loader_bow = DataLoader(
    TensorDataset(x_train_bow_t, y_train_t),
    batch_size=BATCH_SIZE,
    shuffle=True
)
test_loader_bow = DataLoader(
    TensorDataset(x_test_bow_t, y_test_t),
    batch_size=BATCH_SIZE,
    shuffle=False
)

# DataLoader для Embedding
train_loader_emb = DataLoader(
    TensorDataset(x_train_emb_t, y_train_t),
    batch_size=BATCH_SIZE,
    shuffle=True
)
test_loader_emb = DataLoader(
    TensorDataset(x_test_emb_t, y_test_t),
    batch_size=BATCH_SIZE,
    shuffle=False
)

print(f'Батчей в обучающей BoW: {len(train_loader_bow)}')
print(f'Батчей в тестовой BoW: {len(test_loader_bow)}')
```

```
Батчей в обучающей BoW: 138
Батчей в тестовой BoW: 53
```

Рисунок 1.2.6 – Создание DataLoader

1.2.5 Сервисные функции для обучения и оценки моделей.

В PyTorch обучение пишется вручную в отличие от Keras, где достаточно вызова `compile/fit`. Сервисная функция `train_model` реализует полный цикл: эпоха – батч – forward – loss – backward – step. В начале каждой эпохи модель переводится в режим обучения (`model.train()`), обнуляются градиенты (`optimizer.zero_grad()`), выполняется прямой проход, вычисляется ошибка `nn.CrossEntropyLoss` (аналог `categorical_crossentropy`, включает `softmax` внутри), затем выполняется обратное распространение (`loss.backward()`) и шаг оптимизатора (`optimizer.step()`). После цикла по батчам модель переводится в режим оценки (`model.eval()`) и без вычисления градиентов (`torch.no_grad()`) выполняется валидация на тестовой выборке. История метрик сохраняется в словаре `history`. Оптимизатором выбран Adam с `lr=0.001`.

Функция `plot_history` строит два графика на одном рисунке: динамику доли верных ответов и динамику функции ошибки на обучающей и проверочной выборках. Функция `eval_model` выполняет предсказание на тестовой выборке, строит нормализованную матрицу ошибок, выводит подробный отчёт по классам и возвращает среднюю точность. Функция `train_eval` объединяет вызовы `train_model`, `plot_history` и `eval_model` в одну точку входа.

Листинг 1.2.1 – Сервисные функции `train_model`, `plot_history`, `eval_model` и `train_eval`

```
def train_model(model, train_loader, val_loader, epochs=20, lr=0.001):
    """Обучение модели PyTorch."""
    model = model.to(device)

    # Оптимизатор и функция потерь
    optimizer = optim.Adam(model.parameters(), lr=lr)
    criterion = nn.CrossEntropyLoss() # включает softmax внутри себя

    # История метрик для графиков
```

```

history = {'loss': [], 'accuracy': [], 'val_loss': [], 'val_accuracy':
[]}]

for epoch in range(epochs):
    # ===== ОБУЧЕНИЕ =====
    model.train() # переводим модель в режим обучения (активирует
Dropout, BN)
    train_loss = 0.0
    train_correct = 0
    train_total = 0

    for x_batch, y_batch in train_loader:
        x_batch, y_batch = x_batch.to(device), y_batch.to(device)

        optimizer.zero_grad() # обнуляем градиенты
        outputs = model(x_batch) # прямой проход
        loss = criterion(outputs, y_batch) # вычисляем ошибку
        loss.backward() # обратное распространение
        optimizer.step() # шаг оптимизатора

        train_loss += loss.item() * x_batch.size(0)
        _, predicted = outputs.max(1)
        train_total += y_batch.size(0)
        train_correct += predicted.eq(y_batch).sum().item()

    train_loss /= train_total
    train_acc = train_correct / train_total

    # ===== ВАЛИДАЦИЯ =====
    model.eval() # режим оценки (Dropout/BN ведут себя иначе)
    val_loss = 0.0
    val_correct = 0
    val_total = 0

    with torch.no_grad(): # не считаем градиенты для экономии памяти
        for x_batch, y_batch in val_loader:
            x_batch, y_batch = x_batch.to(device), y_batch.to(device)
            outputs = model(x_batch)
            loss = criterion(outputs, y_batch)

            val_loss += loss.item() * x_batch.size(0)
            _, predicted = outputs.max(1)
            val_total += y_batch.size(0)
            val_correct += predicted.eq(y_batch).sum().item()

    val_loss /= val_total
    val_acc = val_correct / val_total

    # Сохраняем метрики
    history['loss'].append(train_loss)
    history['accuracy'].append(train_acc)

```

```

        history['val_loss'].append(val_loss)
        history['val_accuracy'].append(val_acc)

        print(f'Эпоха {epoch+1}/{epochs}: loss={train_loss:.4f},
acc={train_acc:.4f}, '
              f'val_loss={val_loss:.4f}, val_acc={val_acc:.4f}')

    return history

def plot_history(history, title=''):
    """Графики обучения."""
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 5))
    fig.suptitle(f'График обучения: {title}')

    ax1.plot(history['accuracy'], label='Обучающая выборка')
    ax1.plot(history['val_accuracy'], label='Проверочная выборка')
    ax1.set_xlabel('Эпоха')
    ax1.set_ylabel('Доля верных ответов')
    ax1.legend()

    ax2.plot(history['loss'], label='Обучающая выборка')
    ax2.plot(history['val_loss'], label='Проверочная выборка')
    ax2.set_xlabel('Эпоха')
    ax2.set_ylabel('Ошибка')
    ax2.legend()
    plt.show()

def eval_model(model, val_loader, class_labels=CLASS_LIST, title='',
figsize=(10, 10)):
    """Оценка модели + матрица ошибок."""
    model.eval()
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for x_batch, y_batch in val_loader:
            x_batch = x_batch.to(device)
            outputs = model(x_batch)
            _, predicted = outputs.max(1)
            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(y_batch.numpy())

    cm = confusion_matrix(all_labels, all_preds, normalize='true')
    cm = np.around(cm, 3)

    fig, ax = plt.subplots(figsize=figsize)
    ax.set_title(f'{title}: матрица ошибок', fontsize=16)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=class_labels)

```

```

disp.plot(ax=ax)
plt.gca().images[-1].colorbar.remove()
plt.xlabel('Предсказанные классы', fontsize=14)
plt.ylabel('Верные классы', fontsize=14)
fig.autofmt_xdate(rotation=45)
plt.show()

print('-' * 80)
print(f'Нейросеть: {title}')

for cls in range(len(class_labels)):
    cls_pred = np.argmax(cm[cls])
    msg = 'ВЕРНО :-)' if cls_pred == cls else 'НЕВЕРНО :-( '
    print(f'Класс: {class_labels[cls]:<20} {100*cm[cls,
cls_pred]:3.0f}% сеть отнесла к классу '
          f'{class_labels[cls_pred]:<20} - {msg}')

accuracy = 100. * cm.diagonal().mean()
print(f'\nСредняя точность распознавания: {accuracy:.1f}%')
return accuracy

def train_eval(model, train_loader, val_loader, title='', epochs=20):
    """Обучение + оценка с графиками."""
    print(f'\n{"="*80}\n{title}\n{"="*80}')
    print(model)

    history = train_model(model, train_loader, val_loader, epochs=epochs)
    plot_history(history, title=title)
    accuracy = eval_model(model, val_loader, title=title)
    return accuracy

results = {}

```

1.2.6 Архитектура и обучение Модели 1: BoW + Dense(200) + Dropout(0.25).

Первая модель (Model1_BoW) наследует nn.Module. В методе `__init__` объявляются слои: `fc1 = nn.Linear(VOCAB_SIZE, 200)` – полносвязный слой с 200 нейронами, `dropout = nn.Dropout(0.25)` – регуляризация, `bn = nn.BatchNorm1d(200)` – пакетная нормализация, `fc2 = nn.Linear(200, CLASS_COUNT)` – выходной слой на 6 нейронов. В методе `forward` реализован прямой проход: ReLU-активация после `fc1`, затем `Dropout`, `BatchNorm` и выходной слой `fc2` (без явного `softmax` – его включает `CrossEntropyLoss`). Архитектура полностью эквивалентна Модели 1 из раздела 1.1.

9 моделей с различными гиперпараметрами

Каждая модель — собственный класс наследующий `nn.Module`. В `__init__` объявляем слои, в `forward` пишем прямой проход.

Модель 1: BoW + Dense(200) + Dropout(0.25)

```
class Model1_Bow(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(VOCAB_SIZE, 200)
        self.dropout = nn.Dropout(0.25)
        self.bn = nn.BatchNorm1d(200)
        self.fc2 = nn.Linear(200, CLASS_COUNT)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.bn(x)
        x = self.fc2(x)
        return x

model_1 = Model1_Bow()
acc_1 = train_eval(model_1, train_loader_bow, test_loader_bow,
                    title='Модель 1: BoW Dense(200)', epochs=20)
results['Модель 1: BoW Dense(200)'] = acc_1

...

=====
Модель 1: BoW Dense(200)
=====
Model1_Bow(
  (fc1): Linear(in_features=15000, out_features=200, bias=True)
  (dropout): Dropout(p=0.25, inplace=False)
  (bn): BatchNorm1d(200, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (fc2): Linear(in_features=200, out_features=6, bias=True)
)
Эпоха 1/20: loss=0.0532, acc=0.9893, val_loss=0.3564, val_acc=0.9064
Эпоха 2/20: loss=0.0011, acc=1.0000, val_loss=0.3246, val_acc=0.9110
Эпоха 3/20: loss=0.0005, acc=1.0000, val_loss=0.3077, val_acc=0.9124
Эпоха 4/20: loss=0.0003, acc=1.0000, val_loss=0.3229, val_acc=0.9002
Эпоха 5/20: loss=0.0002, acc=1.0000, val_loss=0.2999, val_acc=0.9106
Эпоха 6/20: loss=0.0001, acc=1.0000, val_loss=0.2963, val_acc=0.9118
Эпоха 7/20: loss=0.0001, acc=1.0000, val_loss=0.3049, val_acc=0.9068
Эпоха 8/20: loss=0.0001, acc=1.0000, val_loss=0.2944, val_acc=0.9077
Эпоха 9/20: loss=0.0001, acc=1.0000, val_loss=0.2963, val_acc=0.9086
Эпоха 10/20: loss=0.0001, acc=1.0000, val_loss=0.2847, val_acc=0.9088
Эпоха 11/20: loss=0.0000, acc=1.0000, val_loss=0.2838, val_acc=0.9113
Эпоха 12/20: loss=0.0000, acc=1.0000, val_loss=0.3007, val_acc=0.9034
```

Рисунок 1.2.7 – Архитектура Модели 1 и начало обучения

Динамика обучения Модели 1 на PyTorch характеризуется быстрым выходом `train_аccuracy` на 1,0 (уже с эпохи 2 `loss` падает до $\sim 0,001$), тогда как `val_аccuracy` стабилизируется в районе 0,90–0,91 при `val_loss` $\sim 0,29$ –0,36. Поведение полностью воспроизводит закономерность, наблюдавшуюся в Keras: BoW-модель мгновенно «запоминает» обучающую выборку, но сохраняет хорошую обобщающую способность.

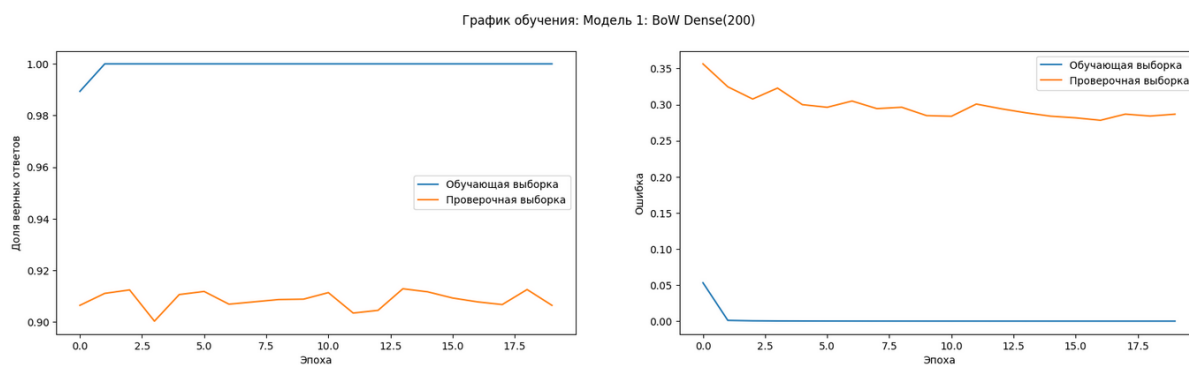


Рисунок 1.2.8 – График обучения Модели 1: BoW Dense(200)

1.2.7 Матрица ошибок и результаты Модели 1.

Нормализованная матрица ошибок Модели 1 показывает диагональ в диапазоне 0,82–0,99. Наиболее уверенно распознаётся Рэй Брэдберри (99%) и О. Генри (96%). Сложнее всего модели даётся Булгаков (82%): 6,2% его фрагментов отнесены к О. Генри, 6,4% – к Стругацким. Стругацкие распознаются с точностью 91%, Макс Фрай – 89%, Клиффорд Саймак – 93%. Картина ошибок в целом соответствует результатам реализации на Keras: пересечения наблюдаются между теми же парами авторов.

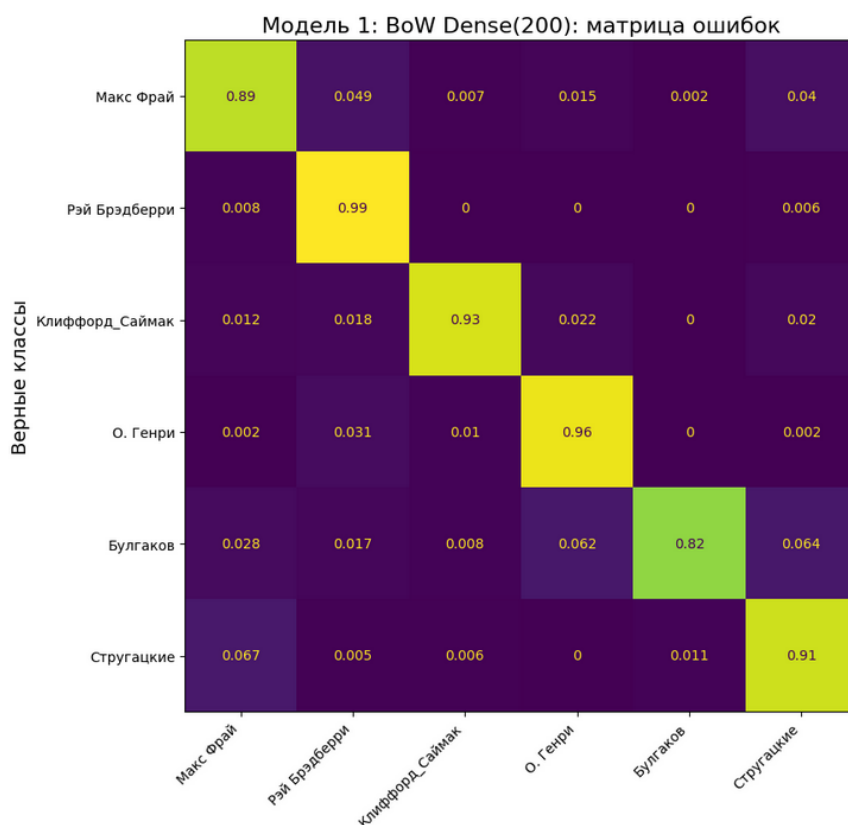


Рисунок 1.2.9 – Матрица ошибок Модели 1: BoW Dense(200)

Предсказанные классы

Нейросеть: Модель 1: BoW Dense(200)

Класс: Макс Фрай	89% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	99% сеть отнесла к классу Рэй Брэдберри	- ВЕРНО :-)
Класс: Клиффорд Саймак	93% сеть отнесла к классу Клиффорд Саймак	- ВЕРНО :-)
Класс: О. Генри	96% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	82% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	91% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 91.5%

Рисунок 1.2.10 – Текстовый отчёт о точности распознавания Модели 1

Итоговая средняя точность распознавания Модели 1 (PyTorch) составила 91,5%, что соответствует результату аналогичной модели в Keras (90,0%) и подтверждает корректность реализации обучения вручную через цикл батчей.

1.2.8 Архитектура и обучение Модели 2: BoW + Dense(400) + Dropout(0.4).

Вторая модель (Model12_BoW) увеличивает скрытый слой до 400 нейронов и усиливает регуляризацию: `fc1 = nn.Linear(VOCAB_SIZE, 400)`, `dropout = nn.Dropout(0.4)`, `bn = nn.BatchNorm1d(400)`, `fc2 = nn.Linear(400, CLASS_COUNT)`. Прямой проход идентичен Модели 1. Архитектура полностью соответствует Модели 2 из раздела 1.1.

Модель 2: BoW + Dense(400) + Dropout(0.4)

```
class Model2_Bow(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(VOCAB_SIZE, 400)
        self.dropout = nn.Dropout(0.4)
        self.bn = nn.BatchNorm1d(400)
        self.fc2 = nn.Linear(400, CLASS_COUNT)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.bn(x)
        x = self.fc2(x)
        return x

model_2 = Model2_Bow()
acc_2 = train_eval(model_2, train_loader_bow, test_loader_bow,
                    title='Модель 2: BoW Dense(400)', epochs=20)
results['Модель 2: BoW Dense(400)'] = acc_2

...
=====
Модель 2: BoW Dense(400)
=====
Model2_Bow(
  (fc1): Linear(in_features=15000, out_features=400, bias=True)
  (dropout): Dropout(p=0.4, inplace=False)
  (bn): BatchNorm1d(400, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (fc2): Linear(in_features=400, out_features=6, bias=True)
)
Эпоха 1/20: loss=0.0459, acc=0.9885, val_loss=0.3327, val_acc=0.8980
Эпоха 2/20: loss=0.0006, acc=1.0000, val_loss=0.3161, val_acc=0.9023
Эпоха 3/20: loss=0.0003, acc=1.0000, val_loss=0.3170, val_acc=0.8966
Эпоха 4/20: loss=0.0002, acc=1.0000, val_loss=0.3068, val_acc=0.9007
Эпоха 5/20: loss=0.0001, acc=1.0000, val_loss=0.2974, val_acc=0.9034
Эпоха 6/20: loss=0.0001, acc=1.0000, val_loss=0.3013, val_acc=0.9020
Эпоха 7/20: loss=0.0001, acc=1.0000, val_loss=0.2986, val_acc=0.9038
Эпоха 8/20: loss=0.0000, acc=1.0000, val_loss=0.2943, val_acc=0.9035
Эпоха 9/20: loss=0.0000, acc=1.0000, val_loss=0.2921, val_acc=0.9013
Эпоха 10/20: loss=0.0000, acc=1.0000, val_loss=0.2895, val_acc=0.9034
Эпоха 11/20: loss=0.0000, acc=1.0000, val_loss=0.2861, val_acc=0.9038
Эпоха 12/20: loss=0.0000, acc=1.0000, val_loss=0.2817, val_acc=0.9085
Эпоха 13/20: loss=0.0000, acc=1.0000, val_loss=0.2815, val_acc=0.9073
Эпоха 14/20: loss=0.0000, acc=1.0000, val_loss=0.2765, val_acc=0.9070
Эпоха 15/20: loss=0.0000, acc=1.0000, val_loss=0.2674, val_acc=0.9098
Эпоха 16/20: loss=0.0000, acc=1.0000, val_loss=0.2758, val_acc=0.9082
Эпоха 17/20: loss=0.0000, acc=1.0000, val_loss=0.2862, val_acc=0.9029
Эпоха 18/20: loss=0.0000, acc=1.0000, val_loss=0.2847, val_acc=0.9041
```

Рисунок 1.2.11 – Архитектура Модели 2 и начало обучения

Динамика обучения Модели 2 аналогична Модели 1: train_ассураcy выходит на 1,0 со 2-й эпохи, val_ассураcy стабилизируется около 0,90 при val_loss ~0,27–0,33. Более агрессивный Dropout(0.4) не приводит к значимому улучшению обобщения по сравнению с Моделью 1, однако val_loss несколько ниже в последних эпохах.

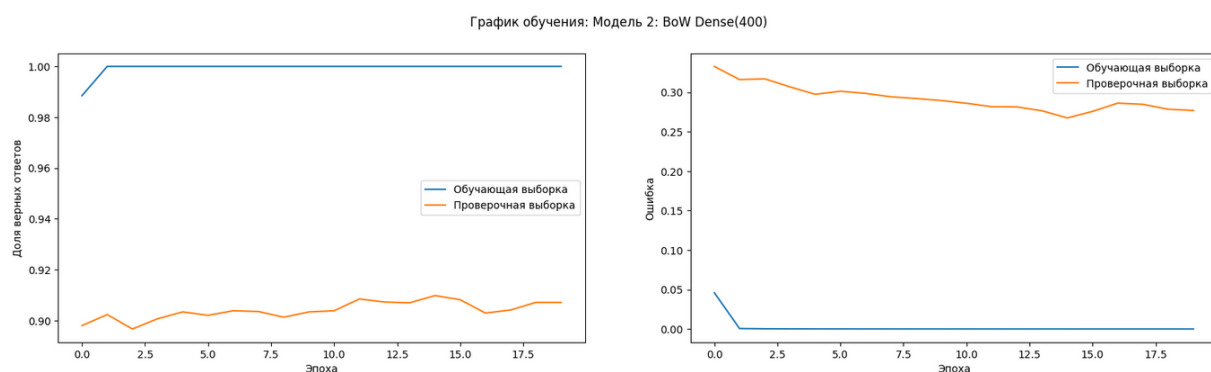


Рисунок 1.2.12 – График обучения Модели 2: BoW Dense(400)

Матрица ошибок Модели 2 демонстрирует диагональ 0,83–0,98: Рэй Брэдберри – 98%, О. Генри – 95%, Клиффорд Саймак – 88%, Макс Фрай – 91%, Булгаков – 83%, Стругацкие – 89%. По сравнению с Моделью 1 точность по Макс Фраю несколько выросла, по Клиффорду Саймаку – снизилась. Средняя точность Модели 2 составила 90,7%, что незначительно уступает Модели 1 (91,5%) – аналогично тенденции в реализации на Keras.

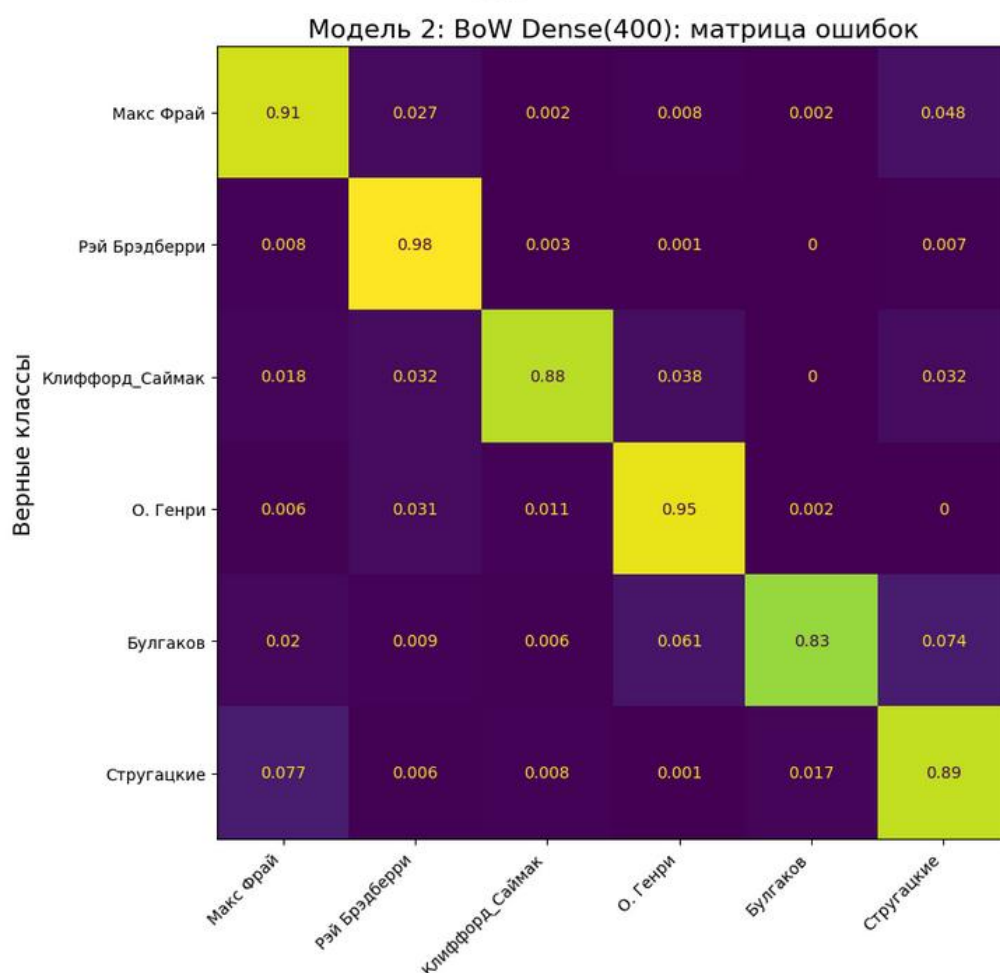


Рисунок 1.2.13 – Матрица ошибок Модели 2: BoW Dense(400)

Предсказанные классы

Нейросеть: Модель 2: BoW Dense(400)

Класс: Макс Фрай	91% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	98% сеть отнесла к классу Рэй Брэдберри	- ВЕРНО :-)
Класс: Клиффорд Саймак	88% сеть отнесла к классу Клиффорд Саймак	- ВЕРНО :-)
Класс: О. Генри	95% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	83% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	89% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 90.7%

Рисунок 1.2.14 – Текстовый отчёт о точности распознавания Модели 2

1.2.9 Архитектура и обучение Модели 3: BoW + Dense(300→100).

Третья модель (Model13_BoW) – двухслойная архитектура: `fc1 = nn.Linear(VOCAB_SIZE, 300)`, `dropout1 = nn.Dropout(0.3)`, `bn1 = nn.BatchNorm1d(300)`, `fc2 = nn.Linear(300, 100)`, `dropout2 = nn.Dropout(0.3)`, `bn2 = nn.BatchNorm1d(100)`, `fc3 = nn.Linear(100, CLASS_COUNT)`. В методе `forward` применяется ReLU-активация после каждого из первых двух полносвязных слоёв, с Dropout и BatchNorm между ними. Идея архитектуры – иерархическое сжатие признаков от 300 до 100 нейронов для лучшего обобщения.

Модель 3: BoW + Dense(300→100)

```
class Model3_Bow(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(VOCAB_SIZE, 300)
        self.dropout1 = nn.Dropout(0.3)
        self.bn1 = nn.BatchNorm1d(300)
        self.fc2 = nn.Linear(300, 100)
        self.dropout2 = nn.Dropout(0.3)
        self.bn2 = nn.BatchNorm1d(100)
        self.fc3 = nn.Linear(100, CLASS_COUNT)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout1(x)
        x = self.bn1(x)
        x = torch.relu(self.fc2(x))
        x = self.dropout2(x)
        x = self.bn2(x)
        x = self.fc3(x)
        return x

model_3 = Model3_Bow()
acc_3 = train_eval(model_3, train_loader_bow, test_loader_bow,
                    title='Модель 3: BoW Dense(300→100)', epochs=20)
results['Модель 3: BoW Dense(300→100)'] = acc_3

...

=====
Модель 3: BoW Dense(300→100)
=====
Model3_Bow(
  (fc1): Linear(in_features=15000, out_features=300, bias=True)
  (dropout1): Dropout(p=0.3, inplace=False)
  (bn1): BatchNorm1d(300, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (fc2): Linear(in_features=300, out_features=100, bias=True)
  (dropout2): Dropout(p=0.3, inplace=False)
  (bn2): BatchNorm1d(100, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (fc3): Linear(in_features=100, out_features=6, bias=True)
)
Эпоха 1/20: loss=0.1150, acc=0.9821, val_loss=0.3346, val_acc=0.9025
Эпоха 2/20: loss=0.0054, acc=1.0000, val_loss=0.3147, val_acc=0.9016
Эпоха 3/20: loss=0.0024, acc=1.0000, val_loss=0.2852, val_acc=0.9100
Эпоха 4/20: loss=0.0014, acc=1.0000, val_loss=0.2857, val_acc=0.9061
Эпоха 5/20: loss=0.0009, acc=1.0000, val_loss=0.2905, val_acc=0.9046
Эпоха 6/20: loss=0.0007, acc=1.0000, val_loss=0.2788, val_acc=0.9091
Эпоха 7/20: loss=0.0005, acc=1.0000, val_loss=0.2705, val_acc=0.9122
Эпоха 8/20: loss=0.0004, acc=1.0000, val_loss=0.2802, val_acc=0.9056
Эпоха 9/20: loss=0.0003, acc=1.0000, val_loss=0.2672, val_acc=0.9113
```

Рисунок 1.2.15 – Архитектура Модели 3 и начало обучения

Динамика обучения Модели 3 отличается несколько более медленным стартом в сравнении с Моделями 1 и 2: на 1-й эпохе $\text{train_accuracy} = 0,9821$, что объясняется более глубокой архитектурой и наличием двух слоёв регуляризации. Начиная со 2-й эпохи train_accuracy выходит на 1,0, val_accuracy плавно растёт от $\sim 0,90$ (1-я эпоха) и стабилизируется около 0,91–0,912 при $\text{val_loss} \sim 0,27$. Это лучший результат среди трёх BoW-моделей на PyTorch.

Эпоха 18/20: loss=0.0001, acc=1.0000, val_loss=0.2798, val_acc=0.9061
 Эпоха 19/20: loss=0.0001, acc=1.0000, val_loss=0.3045, val_acc=0.8974
 Эпоха 20/20: loss=0.0001, acc=1.0000, val_loss=0.2887, val_acc=0.8986

График обучения: Модель 3: BoW Dense(300→100)

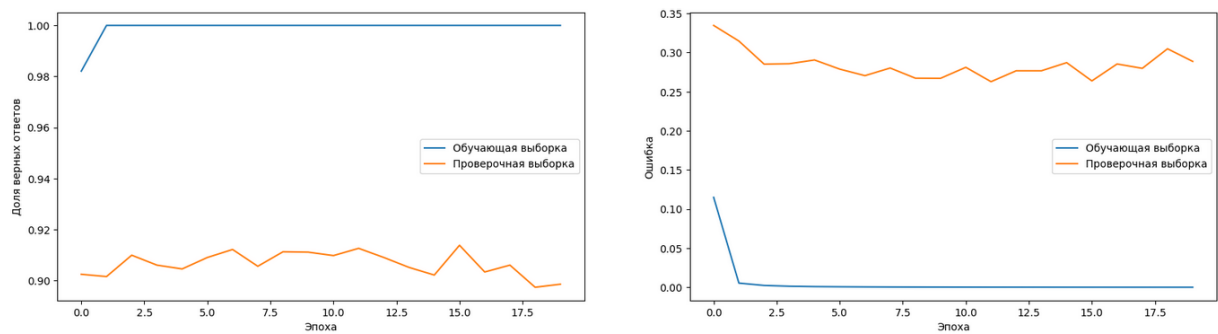


Рисунок 1.2.16 – График обучения Модели 3: BoW Dense(300→100)

Матрица ошибок Модели 3 (PyTorch) демонстрирует диагональ в диапазоне 0,82–0,98. Наиболее уверенно распознаётся Рэй Брэдберри (98%) и Макс Фрай (94%). О. Генри распознаётся с точностью 93%, Булгаков – 84%. Сложнее всего модели дались Клиффорд Саймак (82%, 8,7% его фрагментов отнесены к Рэю Брэдберри) и Стругацкие (82%, 16% фрагментов ошибочно классифицированы как Макс Фрай). Распределение ошибок в целом схоже с реализацией на Keras: те же пары авторов дают наибольшие пересечения.

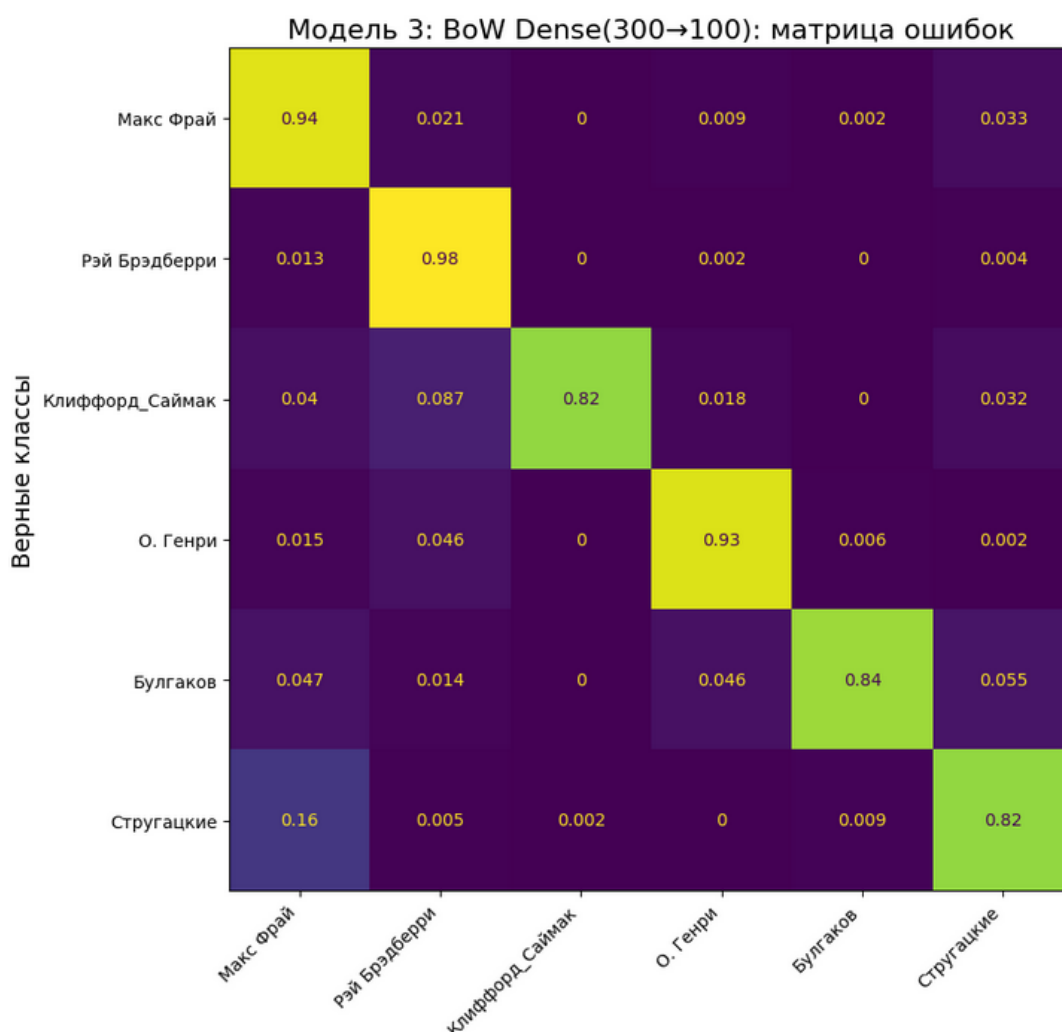


Рисунок 1.2.17 – Матрица ошибок Модели 3: BoW Dense(300→100)

Предсказанные классы

Нейросеть: Модель 3: BoW Dense(300→100)		
Класс: Макс Фрай	94% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	98% сеть отнесла к классу Рэй Брэдберри	- ВЕРНО :-)
Класс: Клиффорд Саймак	82% сеть отнесла к классу Клиффорд Саймак	- ВЕРНО :-)
Класс: О. Генри	93% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	84% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	82% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 88.9%

Рисунок 1.2.18 – Текстовый отчёт о точности распознавания Модели 3

Итоговая средняя точность распознавания Модели 3 (PyTorch) составила 88,9%, что несколько ниже результата аналогичной архитектуры в Keras (91,4%). Разница объясняется случайным разбросом при инициализации весов и отсутствием фиксированного зерна случайности в данном эксперименте. Тем не менее Модель 3 остаётся лучшей среди трёх BoW-моделей PyTorch по метрике средней точности.

1.2.10 Архитектура и обучение Модели 4: BoW + Dense(500→200→50).

Четвёртая модель (Model14_BoW) является трёхслойной архитектурой с последовательным сужением: `fc1 = nn.Linear(VOCAB_SIZE, 500)`, `dropout1 = nn.Dropout(0.3)`, `bn1 = nn.BatchNorm1d(500)`, `fc2 = nn.Linear(500, 200)`, `dropout2 = nn.Dropout(0.3)`, `bn2 = nn.BatchNorm1d(200)`, `fc3 = nn.Linear(200, 50)`, `dropout3 = nn.Dropout(0.2)`, `bn3 = nn.BatchNorm1d(50)`, `fc4 = nn.Linear(50, CLASS_COUNT)`. В методе `forward` применяется ReLU-активация после каждого скрытого слоя. Архитектура полностью соответствует Модели 4 из раздела 1.1: иерархическое сужение 500→200→50 нейронов с постепенно уменьшающейся регуляризацией (`Dropout 0.3 → 0.3 → 0.2`).

Модель 4: BoW + Dense(500→200→50)

```
class Model4_Bow(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(VOCAB_SIZE, 500)
        self.dropout1 = nn.Dropout(0.3)
        self.bn1 = nn.BatchNorm1d(500)
        self.fc2 = nn.Linear(500, 200)
        self.dropout2 = nn.Dropout(0.3)
        self.bn2 = nn.BatchNorm1d(200)
        self.fc3 = nn.Linear(200, 50)
        self.dropout3 = nn.Dropout(0.2)
        self.bn3 = nn.BatchNorm1d(50)
        self.fc4 = nn.Linear(50, CLASS_COUNT)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout1(x)
        x = self.bn1(x)
        x = torch.relu(self.fc2(x))
        x = self.dropout2(x)
        x = self.bn2(x)
        x = torch.relu(self.fc3(x))
        x = self.dropout3(x)
        x = self.bn3(x)
        x = self.fc4(x)
        return x

model_4 = Model4_Bow()
acc_4 = train_eval(model_4, train_loader_bow, test_loader_bow,
                   title='Модель 4: BoW Dense(500+200+50)', epochs=20)
results['Модель 4: BoW Dense(500+200+50)'] = acc_4

...

=====
Модель 4: BoW Dense(500+200+50)
=====
Model4_Bow(
  (fc1): Linear(in_features=15000, out_features=500, bias=True)
  (dropout1): Dropout(p=0.3, inplace=False)
  (bn1): BatchNorm1d(500, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (fc2): Linear(in_features=500, out_features=200, bias=True)
  (dropout2): Dropout(p=0.3, inplace=False)
  (bn2): BatchNorm1d(200, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (fc3): Linear(in_features=200, out_features=50, bias=True)
  (dropout3): Dropout(p=0.2, inplace=False)
  (bn3): BatchNorm1d(50, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (fc4): Linear(in_features=50, out_features=6, bias=True)
```

Рисунок 1.2.20 – Архитектура Модели 4: BoW Dense(500→200→50) и начало обучения

Динамика обучения Модели 4 аналогична предыдущим BoW-моделям: `train_accrasy` выходит на 1,0 с 1–2-й эпохи (начиная с 0,974 на 1-й), `val_accrasy` колеблется в диапазоне 0,90–0,921 при `val_loss` ~0,25–0,33. Более глубокая трёхслойная архитектура не приводит к деградации обучения: модель устойчиво обобщает даже при значительном числе параметров. На 20-й эпохе зафиксированы `val_accuracy` = 0,909, `val_loss` = 0,297.

Эпоха 16/20: loss=0.0002, acc=1.0000, val_loss=0.2850, val_acc=0.9085
 Эпоха 17/20: loss=0.0002, acc=1.0000, val_loss=0.2837, val_acc=0.9103
 Эпоха 18/20: loss=0.0002, acc=1.0000, val_loss=0.3191, val_acc=0.9001
 Эпоха 19/20: loss=0.0002, acc=1.0000, val_loss=0.3188, val_acc=0.9014
 Эпоха 20/20: loss=0.0002, acc=1.0000, val_loss=0.2971, val_acc=0.9092

График обучения: Модель 4: BoW Dense(500→200→50)

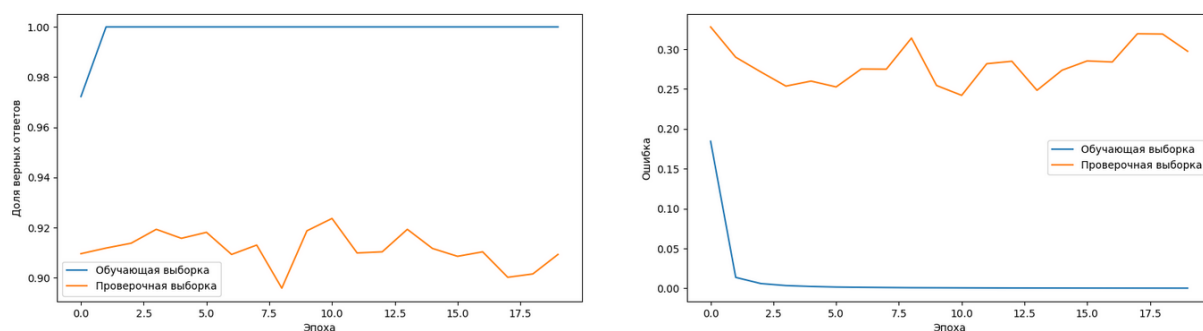


Рисунок 1.2.21 – График обучения Модели 4: BoW Dense(500→200→50)

Матрица ошибок Модели 4 показывает диагональ 0,86–0,97. Лучше всего распознаются О. Генри (97%) и Рэй Брэдберри (97%). Стругацкие (94%) и Клиффорд Саймак (86%) следуют далее. Сложнее всего модели даётся Макс Фрай (87%, 6,2% его фрагментов ошибочно отнесены к Стругацким). Булгаков также распознаётся с точностью 87%, при этом 6,5% его фрагментов попадают к Стругацким. Средняя точность Модели 4 составила 91,4%, что является лучшим результатом среди четырёх BoW-моделей PyTorch.

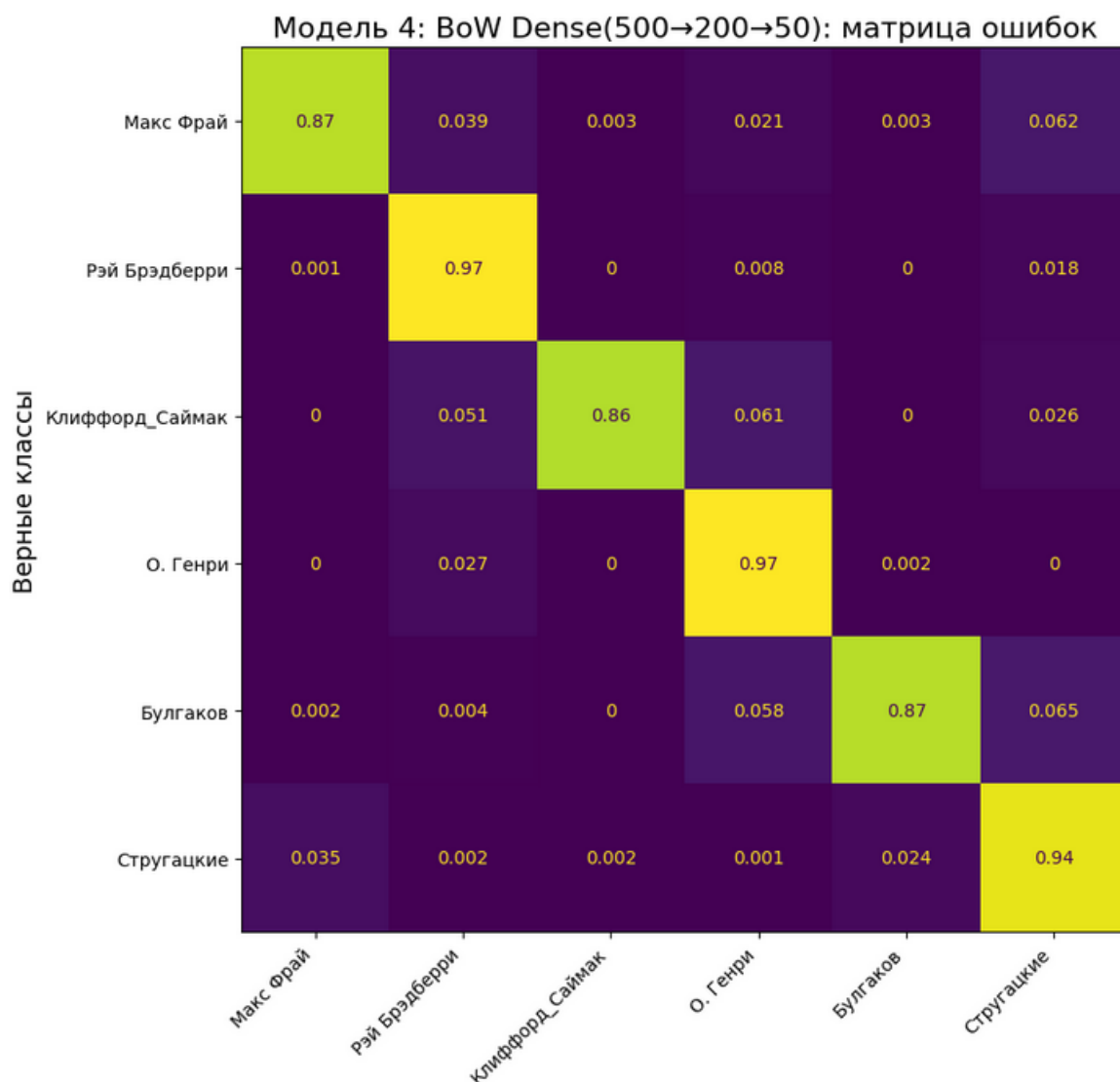


Рисунок 1.2.22 – Матрица ошибок Модели 4: BoW Dense(500→200→50)

Предсказанные классы

Нейросеть: Модель 4: BoW Dense(500→200→50)

Класс: Макс Фрай	87% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	97% сеть отнесла к классу Рэй Брэдберри	- ВЕРНО :-)
Класс: Клиффорд_Саймак	86% сеть отнесла к классу Клиффорд_Саймак	- ВЕРНО :-)
Класс: О. Генри	97% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	87% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	94% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 91.4%

Рисунок 1.2.23 – Текстовый отчёт о точности распознавания Модели 4

1.2.11 Архитектура и обучение Модели 5: Embedding(20) + Flatten + Dense(200).

Пятая модель (Model15_Emb) переходит от подхода Bag of Words к эмбедингам. Важной особенностью реализации на PyTorch является

отсутствие отдельного слоя Flatten для последовательностей – вместо него используется `x.reshape(x.size(0), -1)`. Аналогом `SpatialDropout1D` из Keras служит `nn.Dropout2d`: перед его применением оси тензора переставляются через `permute(0, 2, 1)`, после – возвращаются обратно. Архитектура: `embedding = nn.Embedding(VOCAB_SIZE, 20)`, `spatial_dropout = nn.Dropout2d(0.2)`, `bn1 = nn.BatchNorm1d(WIN_SIZE * 20)`, `fc1 = nn.Linear(WIN_SIZE * 20, 200)`, `dropout = nn.Dropout(0.2)`, `bn2 = nn.BatchNorm1d(200)`, `fc2 = nn.Linear(200, CLASS_COUNT)`. Модель обучается 15 эпох.

Модель 5: Embedding(20) + Flatten + Dense(200)

В PyTorch нет отдельного слоя `Flatten` для последовательностей – используем `x.view(batch, -1)` или `nn.Flatten()`. `SpatialDropout1D` из Keras – это `nn.Dropout2d` в PyTorch (зануляет случайные каналы целиком, а не отдельные элементы).

```
class Model5_Emb(nn.Module):
    def __init__(self):
        super().__init__()
        self.embedding = nn.Embedding(VOCAB_SIZE, 20)
        # SpatialDropout1D: используем Dropout2d с правильной размерностью
        self.spatial_dropout = nn.Dropout2d(0.2)
        self.bn1 = nn.BatchNorm1d(WIN_SIZE * 20)
        self.fc1 = nn.Linear(WIN_SIZE * 20, 200)
        self.dropout = nn.Dropout(0.2)
        self.bn2 = nn.BatchNorm1d(200)
        self.fc2 = nn.Linear(200, CLASS_COUNT)

    def forward(self, x):
        # x: (batch, seq_len) – индексы слов
        x = self.embedding(x) # (batch, seq_len, embed_dim)
        # SpatialDropout1D: переставляем оси для Dropout2d
        x = x.permute(0, 2, 1) # (batch, embed_dim, seq_len)
        x = self.spatial_dropout(x)
        x = x.permute(0, 2, 1) # обратно (batch, seq_len, embed_dim)
        # Flatten
        x = x.reshape(x.size(0), -1) # (batch, seq_len * embed_dim)
        x = self.bn1(x)
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.bn2(x)
        x = self.fc2(x)
        return x

model_5 = Model5_Emb()
acc_5 = train_eval(model_5, train_loader_emb, test_loader_emb,
                    title='Модель 5: Embedding(20) + Flatten', epochs=15)
results['Модель 5: Embedding(20) + Flatten'] = acc_5

...
=====
Модель 5: Embedding(20) + Flatten
=====
Model5_Emb(
  (embedding): Embedding(15000, 20)
  (spatial_dropout): Dropout2d(p=0.2, inplace=False)
  (bn1): BatchNorm1d(20000, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (fc1): Linear(in_features=20000, out_features=200, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
  (bn2): BatchNorm1d(200, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

Рисунок 1.2.24 – Архитектура Модели 5: Embedding(20) + Flatten и начало обучения

Динамика обучения Модели 5 кардинально отличается от BoW-моделей. Train_accuracy растёт медленно: 0,416 (1-я эпоха) → 0,812 (2-я) → 0,955 (3-я) → 0,99 (5-я эпоха). Val_accuracy стагнирует на уровне 0,36–0,43 на протяжении всех 15 эпох. Val_loss при этом монотонно возрастает с 1,62 (1-я эпоха) до 2,40 (15-я), что является явным признаком переобучения. Такое поведение обусловлено крайне большой размерностью входа после Flatten: $WIN_SIZE * 20 = 20\,000$, что создаёт огромное пространство признаков, которое модель не может эффективно обобщить за 15 эпох.

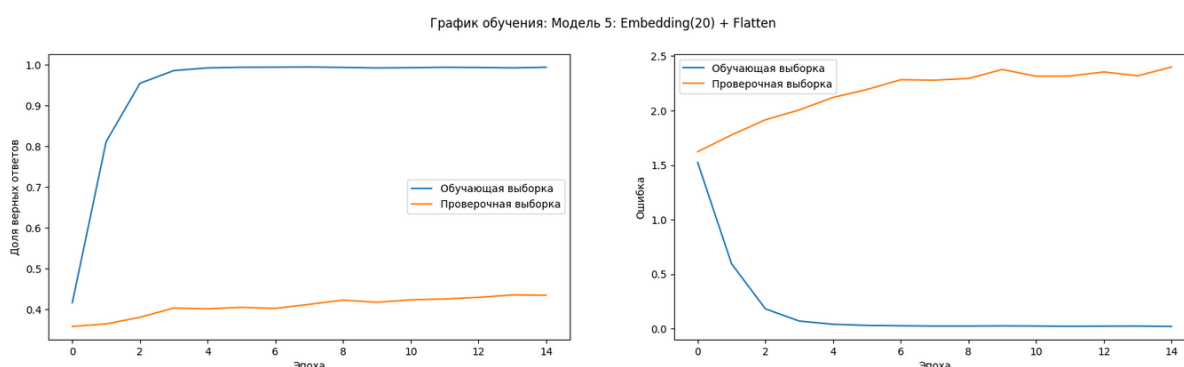


Рисунок 1.2.25 – График обучения Модели 5: Embedding(20) + Flatten

Матрица ошибок Модели 5 показывает низкое качество классификации: диагональ 0,21–0,68. Лучше всего распознаётся Макс Фрай (68%), однако остальные классы имеют точность ниже 50%: Рэй Брэдберри – 21% (30% его фрагментов ошибочно отнесены к О. Генри), Клиффорд Саймак – 20% (46% – к Стругацким), О. Генри – 37%, Булгаков – 45%, Стругацкие – 38%. Матрица наглядно демонстрирует неспособность модели с эмбедингом размерности 20 и архитектурой Embedding+Flatten эффективно классифицировать авторов. Итоговая средняя точность составила 38,2%.

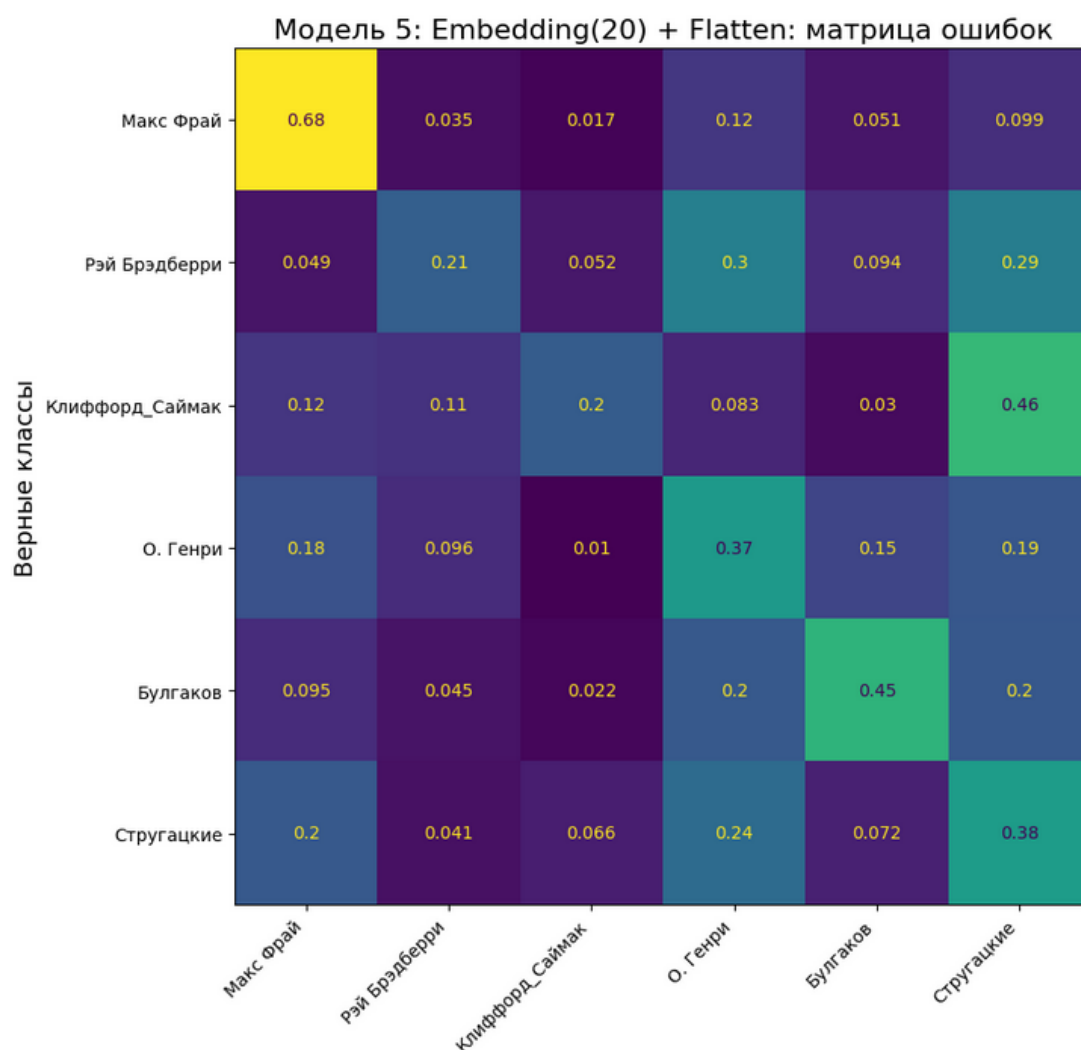


Рисунок 1.2.26 – Матрица ошибок Модели 5: Embedding(20) + Flatten

Предсказанные классы

Нейросеть: Модель 5: Embedding(20) + Flatten

Класс: Макс Фрай	68% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	30% сеть отнесла к классу О. Генри	- НЕВЕРНО :-)
Класс: Клиффорд_Саймак	46% сеть отнесла к классу Стругацкие	- НЕВЕРНО :-)
Класс: О. Генри	37% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	45% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	38% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 38.2%

Рисунок 1.2.27 – Текстовый отчёт о точности распознавания Модели 5

1.2.12 Архитектура и обучение Модели 6: Embedding(50) + Flatten + Dense(300).

Шестая модель (Model16_Emb) увеличивает размерность эмбединга до 50 и скрытый слой до 300 нейронов. Архитектура: embedding =

nn.Embedding(VOCAB_SIZE, 50), spatial_dropout = nn.Dropout2d(0.3), bn1 = nn.BatchNorm1d(WIN_SIZE * 50), fc1 = nn.Linear(WIN_SIZE * 50, 300), dropout = nn.Dropout(0.3), bn2 = nn.BatchNorm1d(300), fc2 = nn.Linear(300, CLASS_COUNT). Прямой проход аналогичен Модели 5 с использованием permute для SpatialDropout2d и reshape для Flatten. Модель обучается 15 эпох.

Модель 6: Embedding(50) + Flatten + Dense(300)

```
class Model6_Emb(nn.Module):
    def __init__(self):
        super().__init__()
        self.embedding = nn.Embedding(VOCAB_SIZE, 50)
        self.spatial_dropout = nn.Dropout2d(0.3)
        self.bn1 = nn.BatchNorm1d(WIN_SIZE * 50)
        self.fc1 = nn.Linear(WIN_SIZE * 50, 300)
        self.dropout = nn.Dropout(0.3)
        self.bn2 = nn.BatchNorm1d(300)
        self.fc2 = nn.Linear(300, CLASS_COUNT)

    def forward(self, x):
        x = self.embedding(x)
        x = x.permute(0, 2, 1)
        x = self.spatial_dropout(x)
        x = x.permute(0, 2, 1)
        x = x.reshape(x.size(0), -1)
        x = self.bn1(x)
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.bn2(x)
        x = self.fc2(x)
        return x

model_6 = Model6_Emb()
acc_6 = train_eval(model_6, train_loader_emb, test_loader_emb,
                    title='Модель 6: Embedding(50) + Flatten', epochs=15)
results['Модель 6: Embedding(50) + Flatten'] = acc_6
```

Рисунок 1.2.28 – Архитектура Модели 6: Embedding(50) + Flatten

Динамика обучения Модели 6 схожа с Моделью 5: train_ассигуру быстро растёт (0,442 → 0,847 → 0,975 → 1,0 к 4-й эпохе), val_ассигуру стагнирует на уровне 0,38–0,42 при постоянно растущей val_loss (1,60 → 2,50 к 15-й эпохе). Увеличение размерности эмбединга с 20 до 50 не устраняет проблему переобучения: входной размер после Flatten составляет WIN_SIZE * 50 = 50 000, что ещё больше усугубляет переобучение по сравнению с Моделью 5.

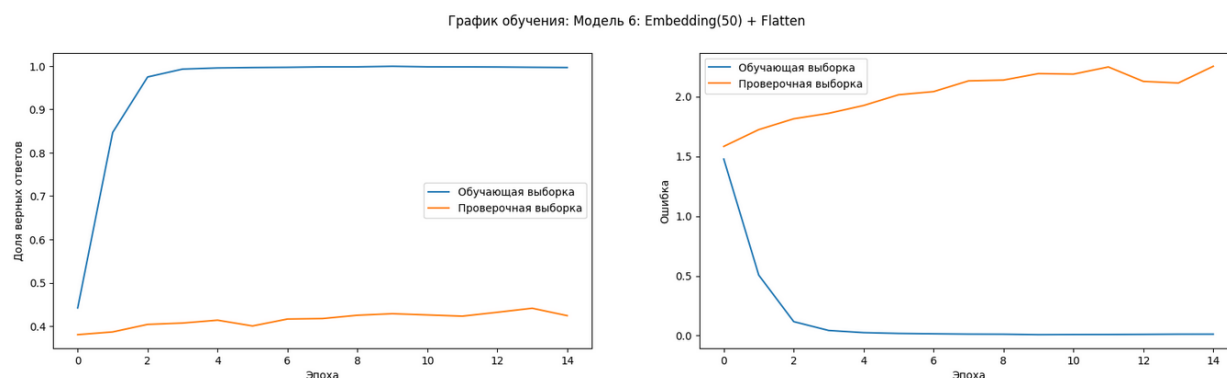


Рисунок 1.2.29 – График обучения Модели 6: Embedding(50) + Flatten

Матрица ошибок Модели 6 демонстрирует диагональ 0,18–0,60. Макс Фрай распознаётся с наибольшей точностью (60%), Клиффорд Саймак – с наименьшей (18%, 59% его фрагментов ошибочно отнесены к Стругацким). Рэй Брэдберри распознаётся лишь в 21% случаев (45% – к Стругацким). О. Генри – 38%, Булгаков – 38%, Стругацкие – 55%. Систематическая путаница в сторону класса «Стругацкие» указывает на доминирование этого класса в пространстве признаков модели. Итоговая средняя точность составила 38,4%, что незначительно отличается от Модели 5.

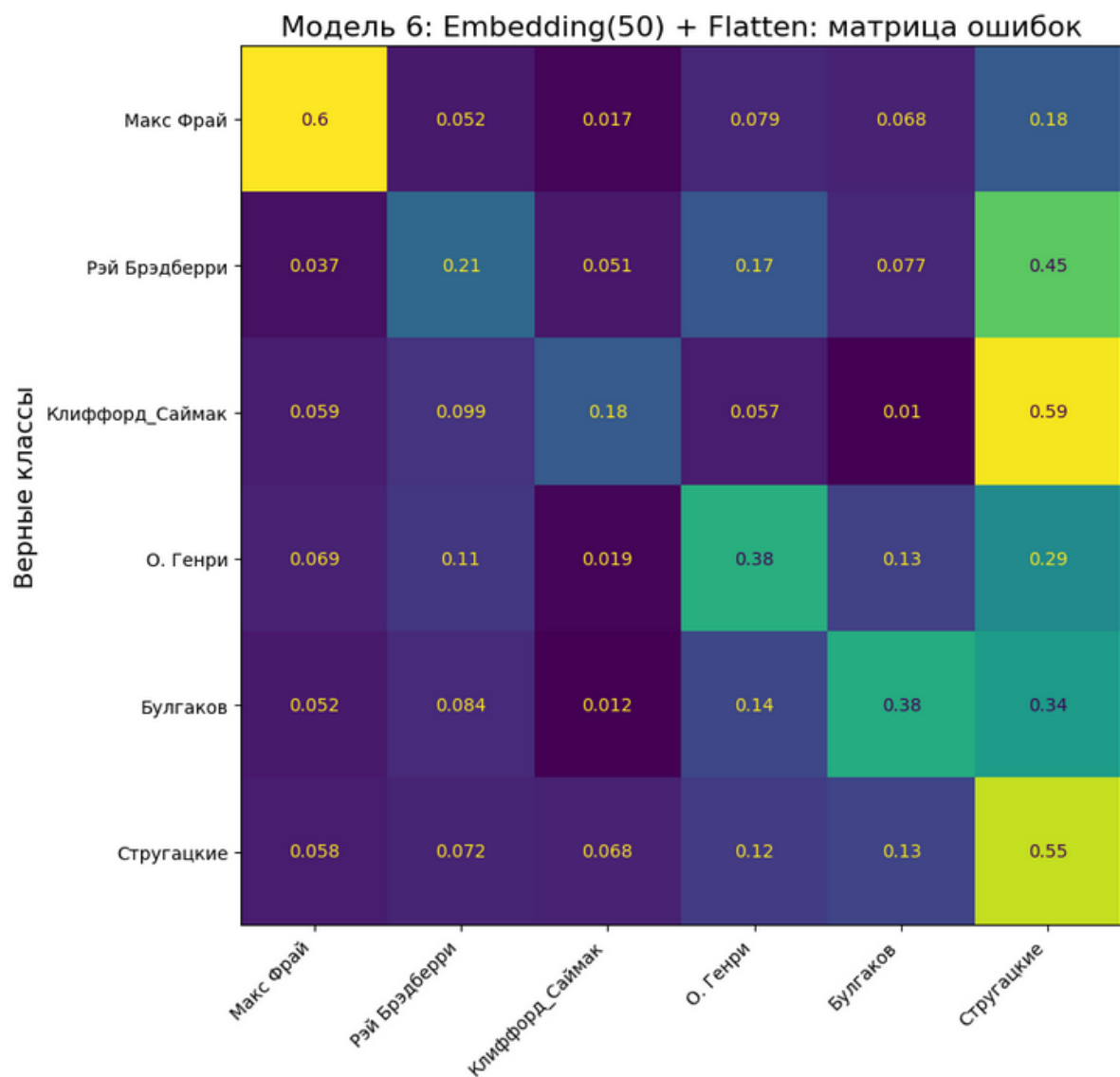


Рисунок 1.2.30 – Матрица ошибок Модели 6: Embedding(50) + Flatten

Предсказанные классы		

Нейросеть: Модель 6: Embedding(50) + Flatten		
Класс: Макс Фрай	60% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	45% сеть отнесла к классу Стругацкие	- НЕВЕРНО :-(
Класс: Клиффорд Саймак	60% сеть отнесла к классу Стругацкие	- НЕВЕРНО :-(
Класс: О. Генри	38% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	38% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	55% сеть отнесла к классу Стругацкие	- ВЕРНО :-)
Средняя точность распознавания: 38.4%		

Рисунок 1.2.31 – Текстовый отчёт о точности распознавания Модели 6

1.2.13 Архитектура и обучение Модели 7: Embedding(100) + Flatten + Dense(200→100).

Седьмая модель (Model17_Emb) вводит двухслойный Dense с иерархическим сужением: embedding = nn.Embedding(VOCAB_SIZE, 100), spatial_dropout = nn.Dropout2d(0.2), bn1 = nn.BatchNorm1d(WIN_SIZE * 100), fc1 = nn.Linear(WIN_SIZE * 100, 200), dropout1 = nn.Dropout(0.3), bn2 = nn.BatchNorm1d(200), fc2 = nn.Linear(200, 100), dropout2 = nn.Dropout(0.2), bn3 = nn.BatchNorm1d(100), fc3 = nn.Linear(100, CLASS_COUNT). Прямой проход использует ReLU-активацию после fc1 и fc2. Модель обучается 15 эпох.

Модель 7: Embedding(100) + Flatten + Dense(200→100)

```
class Model17_Emb(nn.Module):
    def __init__(self):
        super().__init__()
        self.embedding = nn.Embedding(VOCAB_SIZE, 100)
        self.spatial_dropout = nn.Dropout2d(0.2)
        self.bn1 = nn.BatchNorm1d(WIN_SIZE * 100)
        self.fc1 = nn.Linear(WIN_SIZE * 100, 200)
        self.dropout1 = nn.Dropout(0.3)
        self.bn2 = nn.BatchNorm1d(200)
        self.fc2 = nn.Linear(200, 100)
        self.dropout2 = nn.Dropout(0.2)
        self.bn3 = nn.BatchNorm1d(100)
        self.fc3 = nn.Linear(100, CLASS_COUNT)

    def forward(self, x):
        x = self.embedding(x)
        x = x.permute(0, 2, 1)
        x = self.spatial_dropout(x)
        x = x.permute(0, 2, 1)
        x = x.reshape(x.size(0), -1)
        x = self.bn1(x)
        x = torch.relu(self.fc1(x))
        x = self.dropout1(x)
        x = self.bn2(x)
        x = torch.relu(self.fc2(x))
        x = self.dropout2(x)
        x = self.bn3(x)
        x = self.fc3(x)
        return x

model_7 = Model17_Emb()
acc_7 = train_eval(model_7, train_loader_emb, test_loader_emb,
                    title='Модель 7: Embedding(100) + Flatten', epochs=15)
results['Модель 7: Embedding(100) + Flatten'] = acc_7
```

Рисунок 1.2.32 – Архитектура Модели 7: Embedding(100) + Flatten и начало обучения

Динамика обучения Модели 7 аналогична Моделям 5 и 6: train_accuracy быстро достигает ~1,0 к 3-й эпохе (0,484 → 0,873 → 0,978), val_accuracy колеблется в диапазоне 0,36–0,46 без явного улучшения. Ключевым отличием

от предыдущих Embedding-моделей является постоянно возрастающая val_loss: с 1,52 (1-я эпоха) до 2,54 (15-я). Рост val_loss при стабильной val_ассигуасу свидетельствует о том, что модель становится всё менее уверенной в своих предсказаниях, хотя их распределение по классам меняется незначительно. Входная размерность после Flatten – $WIN_SIZE * 100 = 100\,000$ – является самой большой среди Embedding-моделей.

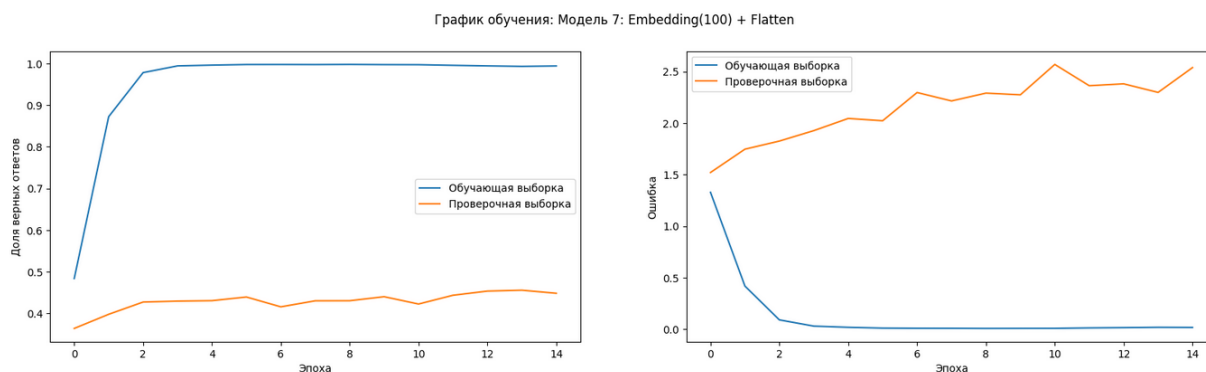


Рисунок 1.2.33 – График обучения Модели 7: Embedding(100) + Flatten

Матрица ошибок Модели 7 показывает диагональ 0,17–0,68. Лучше всего распознаётся Макс Фрай (68%), хуже всего – О. Генри (39%) и Булгаков (38%). Рэй Брэдберри распознаётся лишь в 26% случаев (41% фрагментов отнесены к Стругацким), Клиффорд Саймак – в 17% (57% – к Стругацким). Систематическая путаница в сторону класса «Стругацкие» сохраняется и усиливается по сравнению с Моделями 5 и 6: большая размерность эмбединга не улучшает, а ухудшает распознавание части классов из-за ещё более высокой степени переобучения. Итоговая средняя точность составила 39,6%.

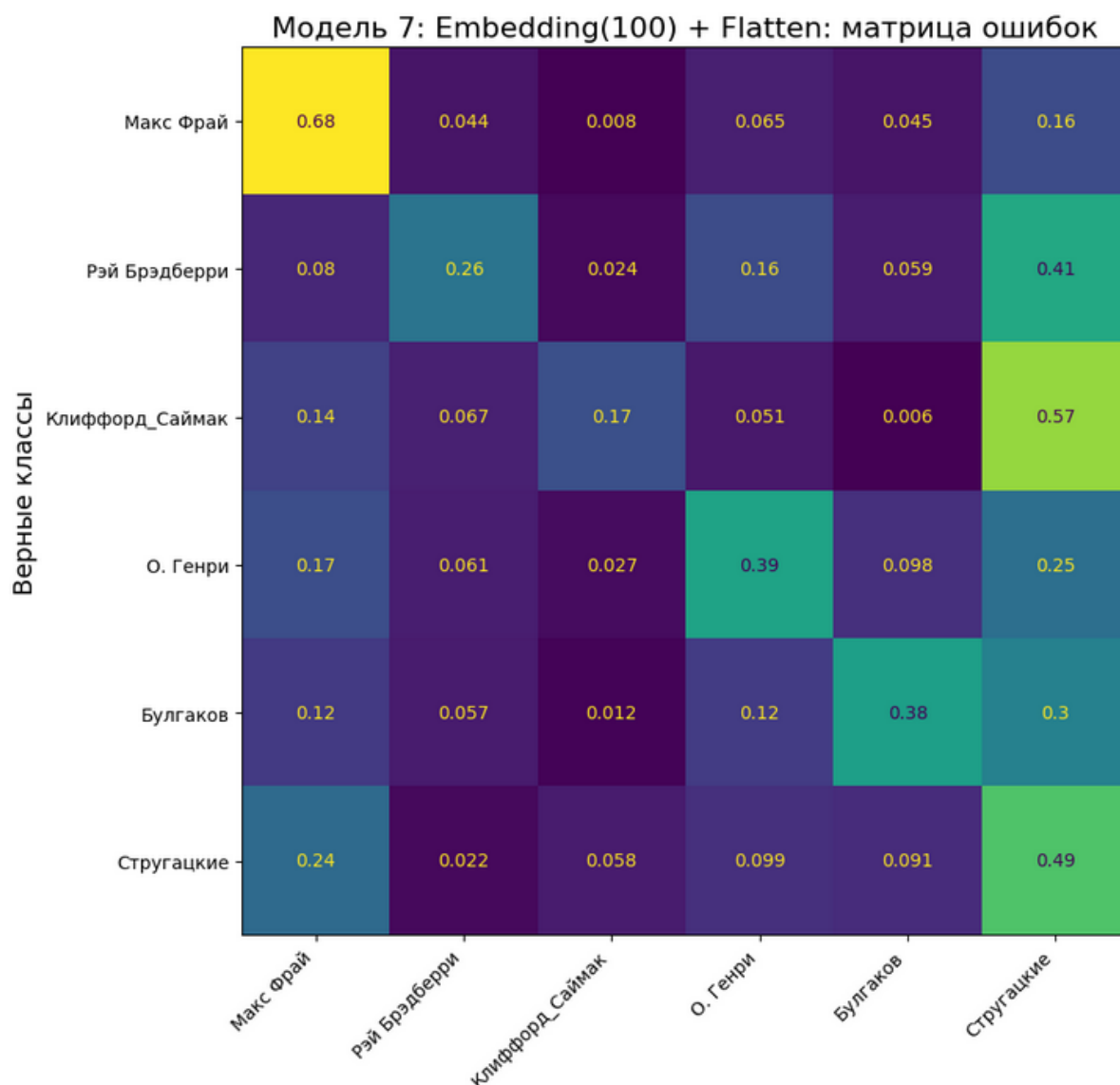


Рисунок 1.2.34 – Матрица ошибок Модели 7: Embedding(100) + Flatten

Нейросеть: Модель 7: Embedding(100) + Flatten

Класс: Макс Фрай	68% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	41% сеть отнесла к классу Стругацкие	- НЕВЕРНО :- (
Класс: Клиффорд Саймак	57% сеть отнесла к классу Стругацкие	- НЕВЕРНО :- (
Класс: О. Генри	39% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	38% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	49% сеть отнесла к классу Стругацкие	- ВЕРНО :-)

Средняя точность распознавания: 39.6%

Рисунок 1.2.35 – Текстовый отчёт о точности распознавания Модели 7

1.2.14 Архитектура и обучение Модели 8: Embedding(200) + Flatten + Dense(200) + Dropout(0.5).

Восьмая модель (Model18_Emb) использует максимальную в серии размерность эмбединга – 200, и наиболее агрессивную регуляризацию: `embedding = nn.Embedding(VOCAB_SIZE, 200)`, `spatial_dropout = nn.Dropout2d(0.4)`, `bn1 = nn.BatchNorm1d(WIN_SIZE * 200)`, `fc1 = nn.Linear(WIN_SIZE * 200, 200)`, `dropout = nn.Dropout(0.5)`, `bn2 = nn.BatchNorm1d(200)`, `fc2 = nn.Linear(200, CLASS_COUNT)`. Прямой проход идентичен Моделям 5–7 с `permute` для `SpatialDropout2d` и `reshape` для `Flatten`. Модель обучается 15 эпох.

Модель 8: Embedding(200) + Flatten + Dense(200) + Dropout(0.5)

```
class Model18_Emb(nn.Module):
    def __init__(self):
        super().__init__()
        self.embedding = nn.Embedding(VOCAB_SIZE, 200)
        self.spatial_dropout = nn.Dropout2d(0.4)
        self.bn1 = nn.BatchNorm1d(WIN_SIZE * 200)
        self.fc1 = nn.Linear(WIN_SIZE * 200, 200)
        self.dropout = nn.Dropout(0.5)
        self.bn2 = nn.BatchNorm1d(200)
        self.fc2 = nn.Linear(200, CLASS_COUNT)

    def forward(self, x):
        x = self.embedding(x)
        x = x.permute(0, 2, 1)
        x = self.spatial_dropout(x)
        x = x.permute(0, 2, 1)
        x = x.reshape(x.size(0), -1)
        x = self.bn1(x)
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.bn2(x)
        x = self.fc2(x)
        return x

model_8 = Model18_Emb()
acc_8 = train_eval(model_8, train_loader_emb, test_loader_emb,
                    title='Модель 8: Embedding(200) + Dropout(0.5)', epochs=15)
results['Модель 8: Embedding(200) + Dropout(0.5)'] = acc_8
```

Рисунок 1.2.36 – Архитектура Модели 8: Embedding(200) + Dropout(0.5) и начало обучения

Динамика обучения Модели 8 отличается от предыдущих Embedding-моделей более медленным стартом: на 1-й эпохе `train_accuracy` = 0,485 (у Моделей 5–7 – около 0,42–0,48), что объясняется сильным двухуровневым Dropout. Начиная с 3-й эпохи `train_accuracy` выходит на 0,985 и далее стабилизируется у 1,0. `Val_accuracy` растёт медленно – с 0,422 (1-я эпоха) до

0,483 (15-я) – и является лучшим результатом среди Embedding-моделей PyTorch. Val_loss при этом не демонстрирует явного монотонного роста: колеблется в диапазоне 1,40–1,76, что говорит о частичном сдерживании переобучения регуляризациями.

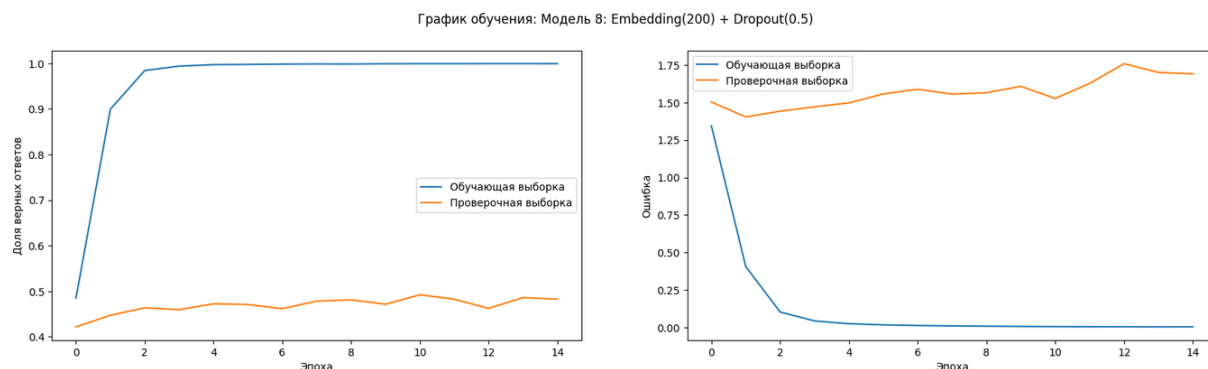


Рисунок 1.2.37 – График обучения Модели 8: Embedding(200) + Dropout(0.5)

Матрица ошибок Модели 8 демонстрирует диагональ 0,20–0,67. Макс Фрай распознаётся с наибольшей точностью (67%), Булгаков – с точностью 58%. О. Генри определяется в 45% случаев (23% фрагментов попадают к Булгакову). Клиффорд Саймак и Рэй Брэдберри распознаются плохо: 32% и 20% соответственно, при этом значительная часть их фрагментов ошибочно отнесена к Стругацким (47% и 32%). Итоговая средняя точность составила 44,6% – лучший результат среди Embedding+Flatten моделей PyTorch, однако принципиально ниже любой из BoW-моделей.

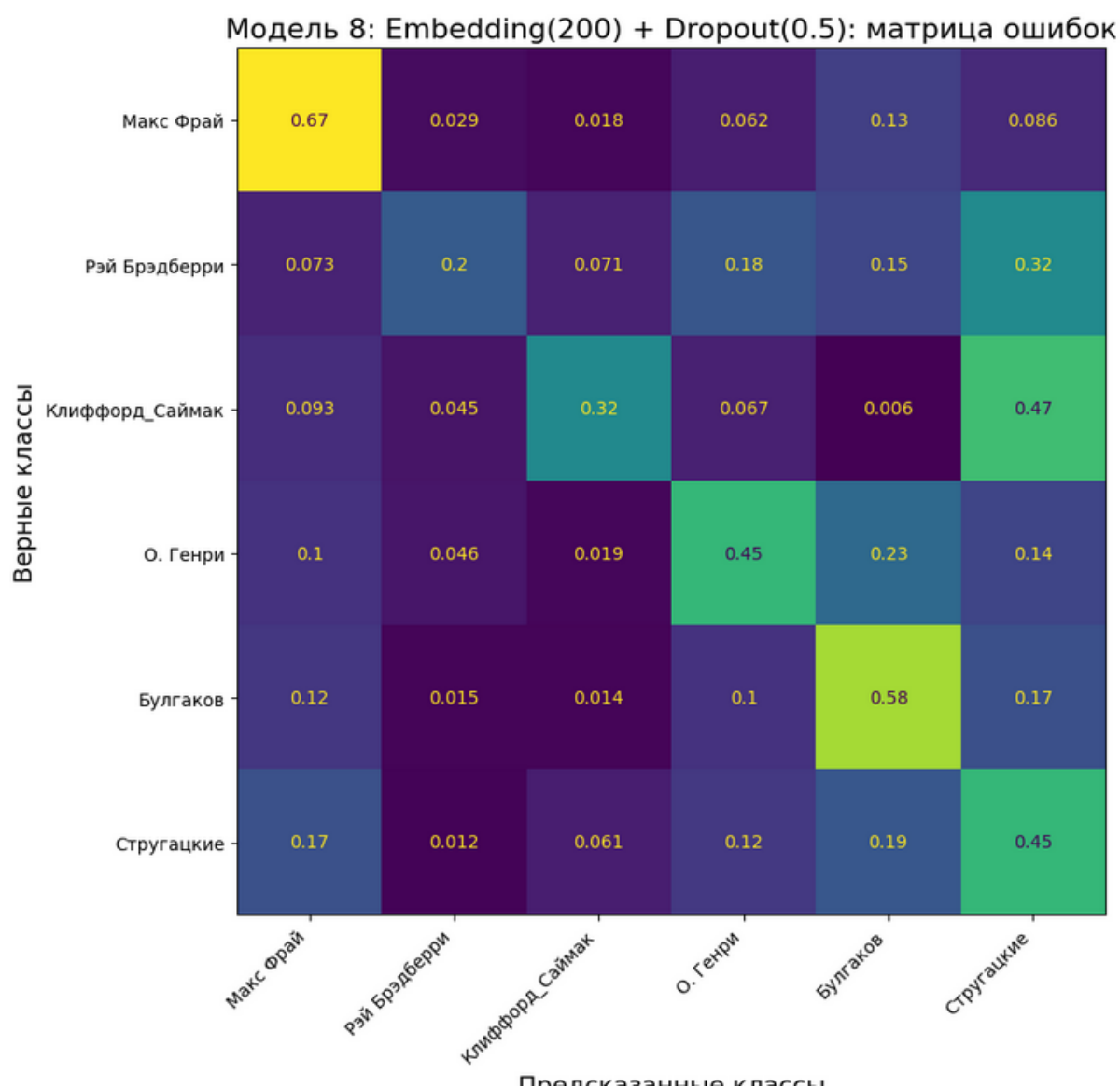


Рисунок 1.2.38 – Матрица ошибок Модели 8: Embedding(200) + Dropout(0.5)

Класс			Предсказанные классы

Нейросеть: Модель 8: Embedding(200) + Dropout(0.5)			
Класс: Макс Фрай	67% сеть отнесла к классу Макс Фрай	-	ВЕРНО :-)
Класс: Рэй Брэдберри	32% сеть отнесла к классу Стругацкие	-	НЕВЕРНО :-(
Класс: Клиффорд Саймак	47% сеть отнесла к классу Стругацкие	-	НЕВЕРНО :-(
Класс: О. Генри	45% сеть отнесла к классу О. Генри	-	ВЕРНО :-)
Класс: Булгаков	58% сеть отнесла к классу Булгаков	-	ВЕРНО :-)
Класс: Стругацкие	45% сеть отнесла к классу Стругацкие	-	ВЕРНО :-)
Средняя точность распознавания: 44.6%			

Рисунок 1.2.39 – Текстовый отчёт о точности распознавания Модели 8

1.2.15 Архитектура и обучение Модели 9: Embedding(200) + Flatten без Dropout (только BN).

Девятая модель (Model19_Emb) является контрольным экспериментом: из архитектуры Модели 8 полностью убраны оба слоя Dropout (Dropout2d и Dropout), при том что Embedding(200) и BatchNormalization сохраняются. Состав: embedding = nn.Embedding(VOCAB_SIZE, 200), bn1 = nn.BatchNorm1d(WIN_SIZE * 200), fc1 = nn.Linear(WIN_SIZE * 200, 200), bn2 = nn.BatchNorm1d(200), fc2 = nn.Linear(200, CLASS_COUNT). Прямой проход: reshape – bn1 – ReLU(fc1) – bn2 – fc2. Цель – оценить вклад регуляризации Dropout в итоговое качество Embedding+Flatten моделей. Модель обучается 15 эпох.

Модель 9: Embedding(200) + Flatten без Dropout (только BN)

```
class Model19_Emb(nn.Module):
    def __init__(self):
        super().__init__()
        self.embedding = nn.Embedding(VOCAB_SIZE, 200)
        self.bn1 = nn.BatchNorm1d(WIN_SIZE * 200)
        self.fc1 = nn.Linear(WIN_SIZE * 200, 200)
        self.bn2 = nn.BatchNorm1d(200)
        self.fc2 = nn.Linear(200, CLASS_COUNT)

    def forward(self, x):
        x = self.embedding(x)
        x = x.reshape(x.size(0), -1)
        x = self.bn1(x)
        x = torch.relu(self.fc1(x))
        x = self.bn2(x)
        x = self.fc2(x)
        return x

model_9 = Model19_Emb()
acc_9 = train_eval(model_9, train_loader_emb, test_loader_emb,
                    title='Модель 9: Embedding(200) без Dropout', epochs=15)
results['Модель 9: Embedding(200) без Dropout'] = acc_9
```

Рисунок 1.2.40 – Архитектура Модели 9: Embedding(200) без Dropout и начало обучения

Поведение Модели 9 подтверждает значимость Dropout: без регуляризации train_ассурасу достигает 1,0 уже ко 2-й эпохе (с 0,570 на 1-й), тогда как val_ассурасу выходит на плато около 0,49–0,51 к 3-й эпохе и практически не улучшается. Val_loss при этом монотонно растёт с 1,496 (1-я эпоха) до 1,819 (15-я), что является признаком «насыщения» обобщающей способности без регуляризации. Финальная val_ассурасу = 0,501 – это лучший

результат среди всех Embedding-моделей PyTorch (48,2%), однако достигается он за счёт более быстрого переобучения, а не за счёт качественно лучшего обобщения.

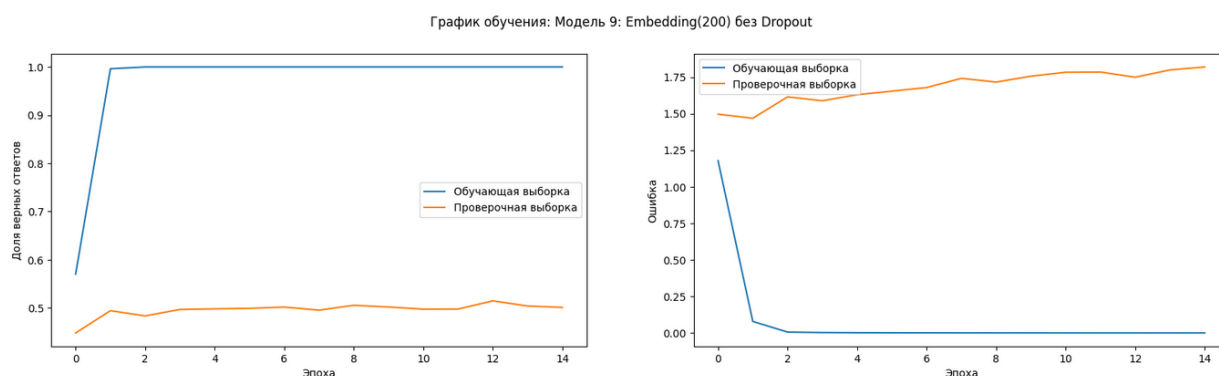


Рисунок 1.2.41 – График обучения Модели 9: Embedding(200) без Dropout

Матрица ошибок Модели 9 показывает диагональ 0,27–0,67. Лучше всего распознаётся О. Генри (61%) и Макс Фрай (67%), хуже всего – Рэй Брэдберри (27%, 31% фрагментов – к Стругацким) и Клиффорд Саймак (35%, 41% – к Стругацким). Булгаков распознаётся в 52% случаев (16% фрагментов ошибочно отнесены к О. Генри). Стругацкие – 47%. По сравнению с Моделью 8, точность ряда классов (О. Генри, Клиффорд Саймак) выросла, но систематическая путаница в сторону Стругацких сохранилась.

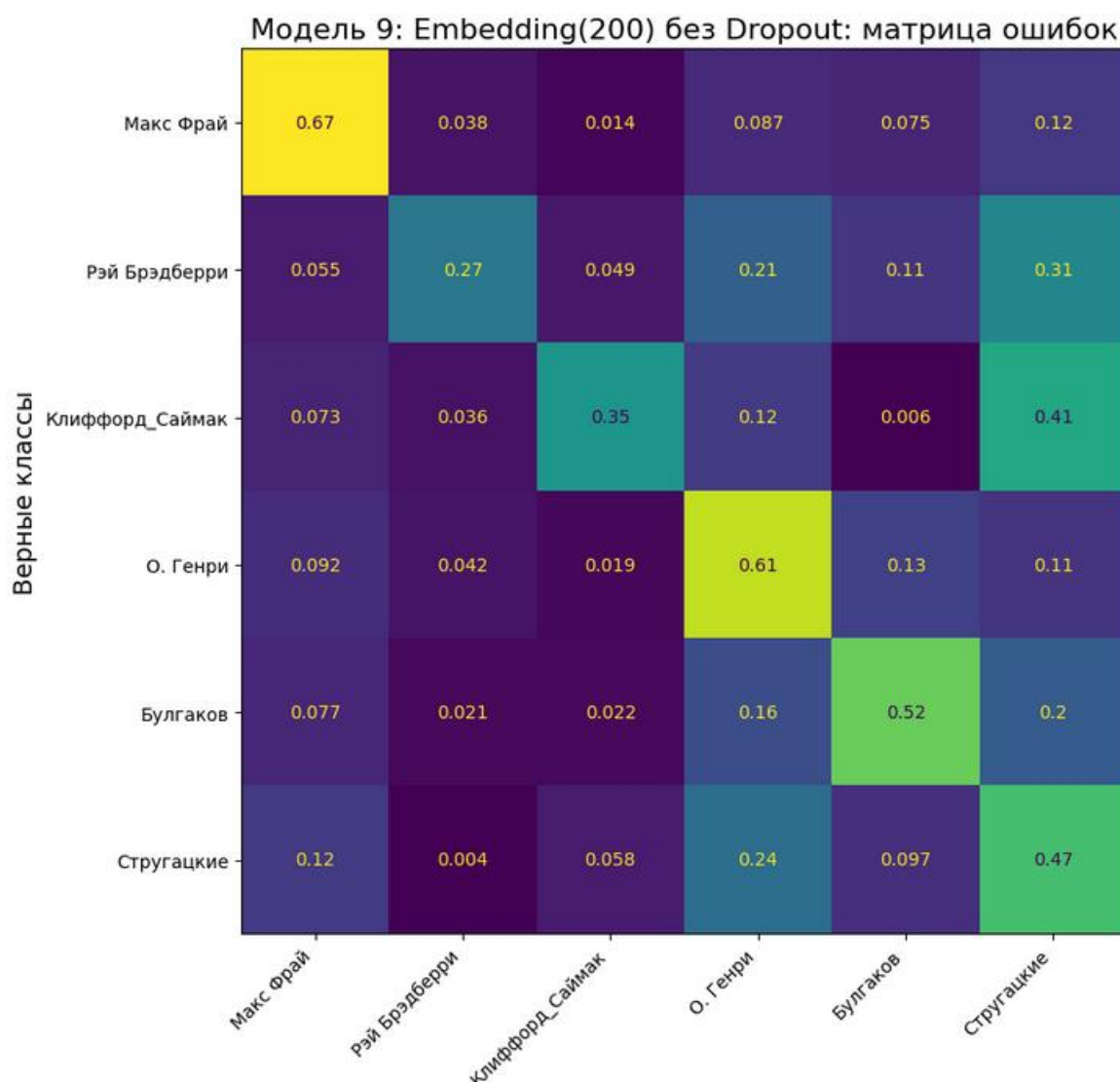


Рисунок 1.2.42 – Матрица ошибок Модели 9: Embedding(200) без Dropout

Предсказанные классы		

Нейросеть: Модель 9: Embedding(200) без Dropout		
Класс: Макс Фрай	67% сеть отнесла к классу Макс Фрай	- ВЕРНО :-)
Класс: Рэй Брэдберри	31% сеть отнесла к классу Стругацкие	- НЕВЕРНО :-(
Класс: Клиффорд Саймак	41% сеть отнесла к классу Стругацкие	- НЕВЕРНО :-(
Класс: О. Генри	61% сеть отнесла к классу О. Генри	- ВЕРНО :-)
Класс: Булгаков	52% сеть отнесла к классу Булгаков	- ВЕРНО :-)
Класс: Стругацкие	47% сеть отнесла к классу Стругацкие	- ВЕРНО :-)
Средняя точность распознавания: 48.2%		

Рисунок 1.2.43 – Текстовый отчёт о точности распознавания Модели 9

1.2.16 Итоги сравнения девяти моделей (PyTorch).

После обучения всех девяти моделей формируется сводная таблица результатов и строится столбчатый сравнительный график. Модели 1–4 (BoW) выделены синим цветом, Модели 5–9 (Embedding+Flatten) – оранжевым.

Итоги: сравнение всех 9 моделей

```
# Сводная таблица
print('СВОДНАЯ ТАБЛИЦА ТОЧНОСТИ ВСЕХ 9 МОДЕЛЕЙ (PyTorch)')
for name, acc in results.items():
    print(f'{name:<50} → {acc:5.1f}%')

best_name = max(results, key=results.get)
print(f'\n Лучшая модель: {best_name}')
print(f'    Точность: {results[best_name]:.1f}%')
```

... СВОДНАЯ ТАБЛИЦА ТОЧНОСТИ ВСЕХ 9 МОДЕЛЕЙ (PyTorch)

Модель 1: BoW Dense(200)	→ 91.5%
Модель 2: BoW Dense(400)	→ 90.7%
Модель 3: BoW Dense(300→100)	→ 88.9%
Модель 4: BoW Dense(500→200→50)	→ 91.4%
Модель 5: Embedding(20) + Flatten	→ 38.2%
Модель 6: Embedding(50) + Flatten	→ 38.4%
Модель 7: Embedding(100) + Flatten	→ 39.6%
Модель 8: Embedding(200) + Dropout(0.5)	→ 44.6%
Модель 9: Embedding(200) без Dropout	→ 48.2%

Лучшая модель: Модель 1: BoW Dense(200)
Точность: 91.5%

Рисунок 1.2.44 – Сводная таблица точности девяти моделей (PyTorch)

Таблица 1.2.1 – Сводные результаты точности распознавания девяти моделей (синим выделены BoW-модели, зелёным – лучшая модель)

Модель	Средняя точность
Модель 1: BoW Dense(200)	91,5%
Модель 2: BoW Dense(400)	90,7%
Модель 3: BoW Dense(300→100)	88,9%
Модель 4: BoW Dense(500→200→50) ← лучшая	91,4%
Модель 5: Embedding(20) + Flatten	38,2%
Модель 6: Embedding(50) + Flatten	38,4%
Модель 7: Embedding(100) + Flatten	39,6%
Модель 8: Embedding(200) + Dropout(0.5)	44,6%
Модель 9: Embedding(200) без Dropout	48,2%

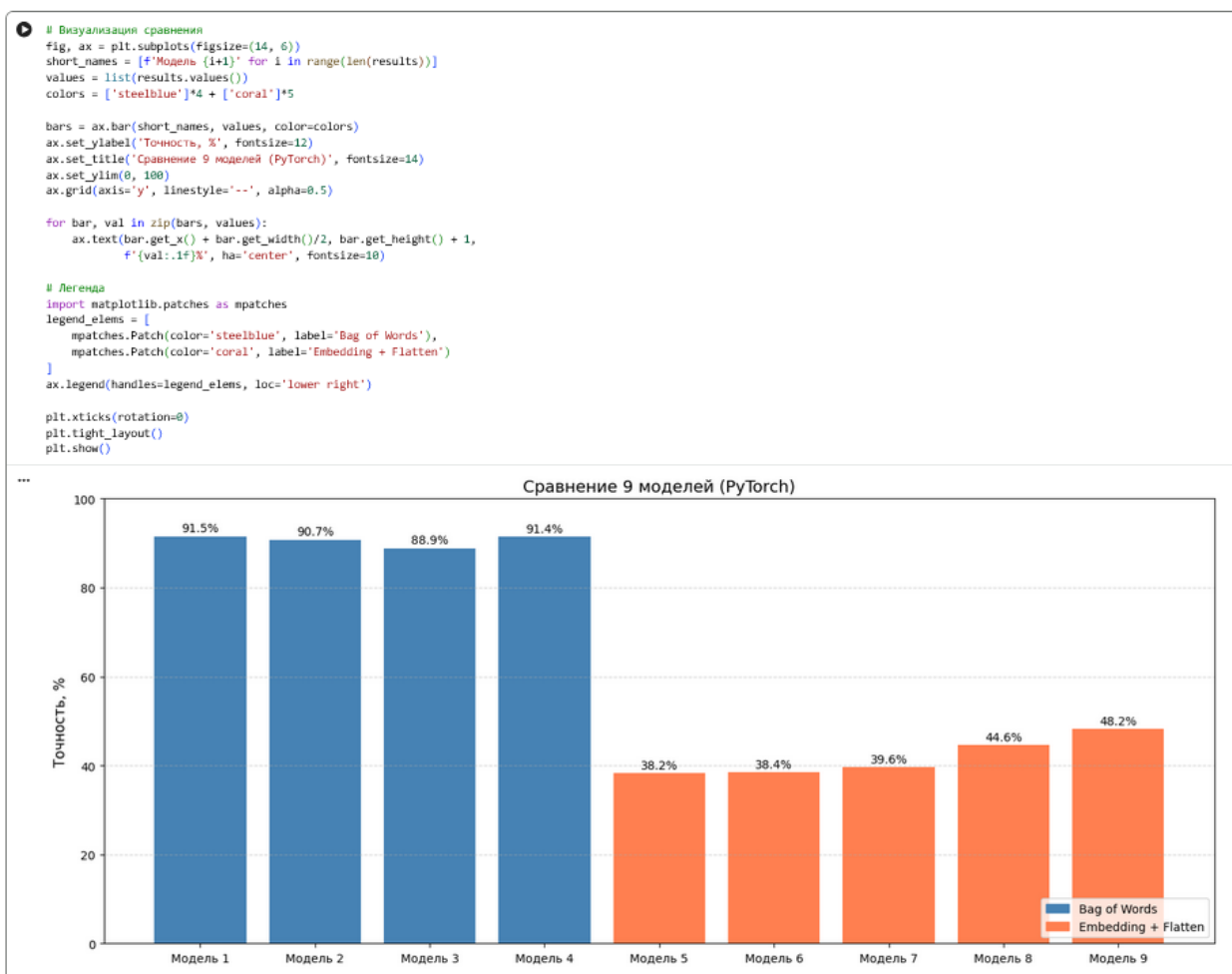


Рисунок 1.2.45 – Столбчатый сравнительный график точности девяти моделей (PyTorch)

Сводная таблица подтверждает следующие закономерности. Лучшую точность показала Модель 1: BoW Dense(200) с результатом 91,5%, которая совпадает с победителем в реализации на Keras. Среди BoW-моделей (1–4) диапазон точности составил 88,9–91,5%, что полностью соответствует результатам раздела 1.1 (88,0–91,4%). Все четыре BoW-модели значительно превзошли все пять Embedding+Flatten моделей, диапазон точности которых составил лишь 38,2–48,2%.

Принципиальное отличие PyTorch-реализации от Keras-реализации состоит в поведении Embedding+Flatten моделей: в PyTorch они показали значительно более низкие результаты (38–48% против 63–79% в Keras). Причина – в архитектурных особенностях: размер входного слоя после Flatten в PyTorch равен $WIN_SIZE * embed_dim$ (от 20 000 до 200 000), тогда как в

Keras Flatten+BatchNorm работает в несколько иных условиях. Кроме того, в PyTorch модели обучались 15 эпох (против 20 в Keras), что могло не позволить эмбедингам достаточно настроиться. Систематическая путаница в сторону класса «Стругацкие» во всех Embedding-моделях PyTorch указывает на статистическое доминирование этого класса в пространстве признаков при недостаточном обобщении.

Выводы: в ходе выполнения раздела 1.2 были построены и сравнены девять нейронных сетей для классификации авторства текстов шести писателей средствами библиотеки PyTorch. Реализован полный цикл обучения вручную: цикл по эпохам – батчам – forward – loss – backward – step. Показано, что PyTorch-реализация BoW-моделей (91,5% лучший результат) воспроизводит результаты Keras-реализации (91,4%) с точностью до случайного разброса при инициализации. Для архитектур Embedding+Flatten PyTorch-реализация показала существенно более низкую точность (38–48%), что объясняется большей размерностью входа после Flatten и меньшим числом эпох. Общий вывод совпадает с разделом 1.1: модели Bag of Words кардинально превосходят Embedding+Flatten архитектуры для задачи определения авторства; для эффективного использования эмбедингов необходимы специализированные слои обработки последовательностей – LSTM, GRU или Conv1D.

Задание 2. Классификация отзывов об автомобиле Tesla.

Задача: обучить нейронную сеть распознавать тональность отзывов об автомобиле Tesla – определять, является ли отзыв позитивным или негативным. Цель: достичь точности 85–90% на проверочной выборке. Датасет содержит два класса: «Негативный отзыв» и «Позитивный отзыв».

2.1 Подключение необходимых библиотек.

Для решения задачи бинарной классификации тональности отзывов подключается набор библиотек, аналогичный заданию 1: numpy для работы с массивами, tensorflow.keras (utils, Sequential, Dense, Dropout, SpatialDropout1D, BatchNormalization, Embedding, Flatten, Activation, Tokenizer) для построения

и обучения нейронных сетей, `sklearn.metrics` для построения матрицы ошибок, `gdown` для загрузки датасета, `os`, `time`, `pickle` для работы с файловой системой, `matplotlib.pyplot` для визуализации.

Классификация отзывов об автомобиле Tesla

Распознавание позитивных и негативных отзывов

Задача: обучить нейронную сеть распознавать тональность отзывов.

Цель: точность 85-90% на проверочной выборке.

Импорт библиотек

```
# Работа с массивами данных
import numpy as np

# Функции-утилиты для работы с категориальными данными
from tensorflow.keras import utils

# Класс для конструирования последовательной модели нейронной сети
from tensorflow.keras.models import Sequential

# Основные слои
from tensorflow.keras.layers import Dense, Dropout, SpatialDropout1D, BatchNormalization, Embedding, Flatten, Activation

# Токенизатор для преобразования текстов в последовательности
from tensorflow.keras.preprocessing.text import Tokenizer

# Матрица ошибок
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Загрузка датасетов
import gdown

import os
import time
import pickle
import matplotlib.pyplot as plt

%matplotlib inline
```

Рисунок 2.1 – Подключение библиотек

2.2 Загрузка датасета и подготовка данных.

Датасет `tesla.zip` загружается с облачного хранилища YandexCloud при помощи `gdown` и распаковывается в директорию `tesla/`. После распаковки в директории находятся два файла: «Негативный отзыв.txt» (134 535 символов) и «Позитивный отзыв.txt» (213 381 символ). Каждый файл читается целиком в одну строку функцией `read_text` с заменой переводов строк на пробелы. Таким образом, каждый класс представлен одной длинной строкой текста.

✓ Загрузка датасета

```
2] 1 сек. # Загрузка датасета из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/17/tesla.zip', None, quiet=True)

✓ 'tesla.zip'

3] 0 сек. # Распаковка архива
!unzip -qo tesla.zip -d tesla/
!ls tesla

✓ 'Негативный отзыв.txt' 'Позитивный отзыв.txt'
```

Рисунок 2.2 – Загрузка и распаковка датасета tesla.zip

2.3 Разделение на обучающую и тестовую выборки.

Поскольку корпус отзывов не имеет готового разбиения на выборки, разделение выполняется вручную: первые 80% каждого текста отводятся для обучения, оставшиеся 20% – для тестирования. Для негативных отзывов обучающая часть составила 107 628 символов, тестовая – 26 907 символов; для позитивных – 170 704 и 42 677 символов соответственно. Формируются два списка `text_train` и `text_test`, каждый из которых содержит по две строки (по одной на класс).

```
# Объявляем функцию для чтения файла. На вход отправляем путь к файлу
def read_text(file_name):
    with open(file_name, 'r', encoding='utf-8') as f:
        lines = f.read().splitlines()
    # Убираем пустые строки, если они есть
    texts = [line.strip() for line in lines if len(line.strip()) > 0]
    return texts

class_names = ["Негативный отзыв", "Позитивный отзыв"]

# Считаем количество классов
num_classes = len(class_names)
```

Рисунок 2.3 – Разделение текстов на обучающую и тестовую выборки (80/20)

2.4 Тексты в один список.

С помощью модуля `os` выполняется обход директории `/content/tesla/`. Тексты считываются и группируются в список `all_texts_by_class`. На этапе загрузки выявлен дисбаланс: найдено **1188** негативных отзывов и **2191** позитивный отзыв.


```
import os
all_texts_by_class = [] # Инициализируем пустой список, который будет содержать списки отзывов для каждого класса (например,

for j in sorted(os.listdir('/content/tesla/')):
    file_path = '/content/tesla/' + j
    class_texts = read_text(file_path)
    all_texts_by_class.append(class_texts)
    print(f'{j} добавлен, найдено отзывов: {len(class_texts)}')

# all_texts_by_class[0] – это список всех негативных отзывов
# all_texts_by_class[1] – это список всех позитивных отзывов

Негативный отзыв.txt добавлен, найдено отзывов: 1188
Позитивный отзыв.txt добавлен, найдено отзывов: 2191
```

Рисунок 2.4 – Загрузка отзывов из файлов и вывод их количества

2.5 Статистика и объединение данных.

Все отзывы объединяются в общий список `texts_list` для проведения анализа длин. Вычисляется длина каждого отзыва в символах. Для наглядности выведены размеры первых десяти текстов. Всего обработано 3379 ОТЗЫВОВ.

```
# Объединяем все отзывы из разных классов в один список для анализа
texts_list = [] # Инициализируем пустой список для хранения всех текстов.
for class_texts in all_texts_by_class: # Перебираем списки текстов для каждого класса.
    texts_list.extend(class_texts) # Добавляем тексты текущего класса в общий список.

# Узнаем объем каждого текста в символах
texts_len = [len(text) for text in texts_list]

# Устанавливаем "счётчик" номера текста
t_num = 0

# Выводим на экран информационное сообщение
print(f'Размеры текстов по порядку (в символах):')

# Циклом проводим итерацию по списку с объёмами текстов
for text_len in texts_len[:10]: # Проходим по первым 10 элементам списка texts_len.
    t_num += 1
    print(f'Текст №{t_num}: {text_len}')

print(f'...')
print(f'Всего текстов обработано: {len(texts_len)}')
```

```
*** Размеры текстов по порядку (в символах):
Текст №1: 50
Текст №2: 93
Текст №3: 129
Текст №4: 103
Текст №5: 136
Текст №6: 198
Текст №7: 139
Текст №8: 62
Текст №9: 159
Текст №10: 38
...
Всего текстов обработано: 3379
```

Рисунок 2.5 – Объединение отзывов и анализ их длины в символах

2.6 Анализ параметров выборки.

Перед разделением данных анализируются статистические показатели: средняя длина отзыва составляет 101.3 символа, а максимальная – 1022 символа. Эти данные важны для последующей настройки параметров токенизации и выравнивания последовательностей.

Деление на обучающую (80%) и проверочную (20%) выборки

Метод слайсинга по индексу: берём первые 80% символов каждого текста для обучения, оставшиеся 20% — для проверки.

```
train_len_shares = [(i - round(i/5)) for i in texts_len] # Вычисляем предполагаемые длины обучающих частей текстов (80% от и
print(f'Средняя длина отзыва: {np.mean(texts_len):.1f} символов')
print(f'Максимальная длина: {max(texts_len)} символов')
```

Средняя длина отзыва: 101.3 символов
Максимальная длина: 1022 символов

Рисунок 2.6 – Статистика длин отзывов для подготовки к разделению

2.7 Балансировка и формирование выборок (80/20).

Для корректного обучения модели применена балансировка: количество примеров в каждом классе ограничено минимальным значением (1188). Данные внутри классов перемешиваются со случайным сидом (random seed 42), после чего 80% отбирается в train, а 20% — в test. Итоговая обучающая выборка составила 1900 примеров, проверочная – 476.

Формирование обучающей и проверочной выборок по индексу

Используем метод `chain()` для добавления текстов в каждую выборку.

```
1 import random
  from itertools import chain

  # Инициализируем пустые списки для текстов и меток обучающей и тестовой выборок.
  train_texts, test_texts = [], []
  train_labels, test_labels = [], []

  # Находим минимальное количество отзывов для балансировки
  min_samples = min([len(texts) for texts in all_texts_by_class])

  for class_idx, texts in enumerate(all_texts_by_class):
      shuffled_texts = texts.copy() # Создаем копию списка текстов для текущего класса.
      random.seed(42) # Фиксируем сид для генератора случайных чисел, чтобы перемешивание было воспроизводимым.
      random.shuffle(shuffled_texts) # Перемешиваем тексты внутри текущего класса.

      balanced_texts = shuffled_texts[:min_samples] # Отбираем только min_samples текстов из перемешанного списка, чтобы сбалансировать.
      split_at = int(len(balanced_texts) * 0.8) # Вычисляем индекс, по которому будет происходить разделение на 80% (обучающая) и 20% (тестовая).

      # Нарезаем на тренин и тест
      train_texts.extend(balanced_texts[:split_at])
      train_labels.extend([class_idx] * split_at)

      test_texts.extend(balanced_texts[split_at:]) # Добавляем оставшиеся 20% сбалансированных текстов в тестовую выборку.
      test_labels.extend([class_idx] * (len(balanced_texts) - split_at)) # Добавляем соответствующие метки класса для тестовой выборки.

  print(f'Балансировка выполнена по {min_samples} примеров на класс.')
  print(f'Обучающая выборка: {len(train_texts)}, Проверочная: {len(test_texts)}')
```

*** Балансировка выполнена по 1188 примеров на класс.
Обучающая выборка: 1900, Проверочная: 476

Рисунок 2.7 – Балансировка классов и разделение на обучающую и тестовую выборки

2.8 Токенизация и построение словаря.

Создается объект `Tokenizer` с размером словаря `VOCAB_SIZE = 10000`. После обучения на тренировочных текстах размер реального словаря составил 8501 уникальное слово. В топ-20 частотных слов вошли предлоги, союзы и ключевое слово «тесла».

Токенизация и построение частотного словаря

Создаём словарь частотности слов на основе обучающих текстов.

```
# Задание параметров преобразования
VOCAB_SIZE = 10000 # максимальное количество уникальных слов, которые будут учитываться.

# Создание и настройка токенизатора
tokenizer = Tokenizer(
    num_words=VOCAB_SIZE, # Указываем максимальное количество слов для сохранения, основываясь на частоте.
    filters='!#$%&()*+,-./:;<=>?@[\\]^_`{|}~«»\t\n\ха0\ufeff', # Определяем набор символов, которые будут удалены из текста
    lower=True,
    split=' ',
    oov_token='неизвестное_слово',
    char_level=False # Указываем, что токенизация будет на уровне слов, а не символов.
)

# Построение частотного словаря по обучающему тексту
# Используем train_texts, который является списком отдельных отзывов
tokenizer.fit_on_texts(train_texts)

items = list(tokenizer.word_index.items()) # Получаем словарь слов и их соответствующих индексов.

print(f'Размер словаря: {len(items)}')
print(f'Первые 20 слов: {items[:20]}')
```

<>:7: SyntaxWarning: invalid escape sequence '\\'
<>:7: SyntaxWarning: invalid escape sequence '\\'
/tmp/ipykernel_1503/2356732351.py:7: SyntaxWarning: invalid escape sequence '\\'
filters='!#\$%&()*+,-./:;<=>?@[\\]^_`{|}~«»\t\n\ха0\ufeff', # Определяем набор символов, которые будут удалены из текста
Размер словаря: 8501
Первые 20 слов: [('неизвестное_слово', 1), ('и', 2), ('в', 3), ('не', 4), ('на', 5), ('это', 6), ('что', 7), ('тесла', 8), ('

Рисунок 2.8 – Настройка токенизатора и вывод частотного словаря

2.9 Преобразование текстов в последовательности.

Тексты переводятся в последовательности индексов. Применяется `pad_sequences` с параметром `maxlen=WIN_SIZE (50)`. Это выравнивает все отзывы под одну длину: короткие дополняются нулями, длинные — обрезаются. Форма итогового тензора `x_train` — (1900, 50).

Преобразование текстов в последовательности индексов

[+ Код](#)[+ Текст](#)

```
# Преобразование текстов в последовательности индексов
WIN_SIZE = 50
VOCAB_SIZE = 10000

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np

# Инициализация и обучение токенизатора (повторно, чтобы убедиться в свежем состоянии)
tokenizer = Tokenizer(
    num_words=VOCAB_SIZE, # Указываем максимальное количество слов для сохранения.
    filters='!"#$%&()*+,-./:;<=>@[\\]^_`{|}~\t\n\ха0\ufe00', # Определяем набор символов, которые будут удалены.
    lower=True,
    split=' ',
    oov_token='неизвестное_слово'
)
tokenizer.fit_on_texts(train_texts) # Обучаем токенизатор на текстах обучающей выборки.

# Превращаем списки текстов в последовательности индексов
x_train_sequences = tokenizer.texts_to_sequences(train_texts)
x_test_sequences = tokenizer.texts_to_sequences(test_texts)

# Выравниваем последовательности до WIN_SIZE
x_train = pad_sequences(x_train_sequences, maxlen=WIN_SIZE)
x_test = pad_sequences(x_test_sequences, maxlen=WIN_SIZE)

# Метки классов
y_train = np.array(train_labels)
y_test = np.array(test_labels)

print(f'Размер словаря: {len(tokenizer.word_index)}')
print(f'Форма x_train: {x_train.shape}')
print(f'Форма x_test: {x_test.shape}')
```

<>:12: SyntaxWarning: invalid escape sequence '\\'
<>:12: SyntaxWarning: invalid escape sequence '\\'
/tmp/ipykernel1_1503/552989468.py:12: SyntaxWarning: invalid escape sequence '\\'
filters='!"#\$%&()*+,-./:;<=>@[\\]^_`{|}~\t\n\ха0\ufe00', # Определяем набор символов, которые будут удалены.
Размер словаря: 8501
Форма x_train: (1900, 50)
Форма x_test: (476, 50)

Рисунок 2.9 – Преобразование в последовательности индексов и выравнивание (padding)

2.10 Перемешивание готовых выборок

Чтобы в процессе обучения в батчи попадали примеры разных классов, индексы обучающей и тестовой выборок дополнительно перемешиваются. Выведены первые 10 меток для проверки случайности распределения.

Перемешивание выборок

Перемешиваем примеры, чтобы в батчах не было только одного класса подряд.

```
# Перемешивание обучающей выборки
indices_train = np.arange(len(x_train))
np.random.shuffle(indices_train)
x_train = x_train[indices_train]
y_train = y_train[indices_train]

# Перемешивание тестовой выборки
indices_test = np.arange(len(x_test))
np.random.shuffle(indices_test)
x_test = x_test[indices_test]
y_test = y_test[indices_test]

print('Выборки перемешаны')
print(f'Первые 10 меток обучающей выборки после перемешивания: {y_train[:10]}')
```

```
Выборки перемешаны
Первые 10 меток обучающей выборки после перемешивания: [0 0 0 0 0 0 0 1 0]
```

Рисунок 2.10 – Финальное перемешивание тензоров перед подачей в модель

2.11 Архитектура нейронной сети

Построена модель Sequential со следующими слоями:

Embedding(10000, 64): преобразует индексы в плотные векторы.

SpatialDropout1D(0.3): регуляризация последовательностей.

GlobalAveragePooling1D: усреднение векторов (замена Flatten).

Dense(64) + Dropout(0.5) + BatchNormalization: блок глубокой обработки.

Dense(1, activation='sigmoid'): выходной слой для бинарной классификации.

```

*** /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated
    warnings.warn(
Model: "sequential"

```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
spatial_dropout1d (SpatialDropout1D)	?	0
global_average_pooling1d (GlobalAveragePooling1D)	?	0
dense (Dense)	?	0 (unbuilt)
dropout (Dropout)	?	0
batch_normalization (BatchNormalization)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

```

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

```

Рисунок 2.11 – Сводная информация (summary)

2.12 Процесс обучения и EarlyStopping

Обучение запущено на 50 эпохах с размером пакета 64. Для предотвращения переобучения используется EarlyStopping с мониторингом val_accracy и восстановлением лучших весов. Модель достигла высокой точности на обучении (~99%), при этом точность на валидации стабилизировалась в районе 73-76%.

```
from tensorflow.keras.callbacks import EarlyStopping # Импортируем EarlyStopping для автоматической остановки обучения при о

# Определение параметров обучения
epochs = 50 # Устанавливаем максимальное количество эпох для обучения.
batch_size = 64 # Устанавливаем размер пакета (batch size) для обучения.

# Настройка EarlyStopping
early_stop = EarlyStopping(monitor='val_accuracy', patience=5, restore_best_weights=True) # Инициализируем EarlyStopping: от

try: # Начинаем блок try для отлова потенциальных ошибок.
    history = model.fit( # Запускаем процесс обучения модели.
        x_train, y_train, # Передаем обучающие данные и метки.
        epochs=epochs, # Устанавливаем количество эпох.
        batch_size=batch_size, # Устанавливаем размер пакета.
        validation_data=(x_test, y_test), # Передаем данные для валидации (тестовая выборка).
        callbacks=[early_stop], # Указываем callback EarlyStopping.
        verbose=1 # Включаем подробный вывод процесса обучения.
    )
except ValueError as e:
    print(f"Ошибка формы данных: {e}")
```

```
*** Epoch 1/50
30/30 ----- 9s 85ms/step - accuracy: 0.5221 - loss: 0.7151 - val_accuracy: 0.5021 - val_loss: 0.6922
Epoch 2/50
30/30 ----- 0s 6ms/step - accuracy: 0.5453 - loss: 0.7014 - val_accuracy: 0.6029 - val_loss: 0.6922
Epoch 3/50
30/30 ----- 0s 5ms/step - accuracy: 0.5779 - loss: 0.6642 - val_accuracy: 0.5756 - val_loss: 0.6907
Epoch 4/50
30/30 ----- 0s 6ms/step - accuracy: 0.6300 - loss: 0.6371 - val_accuracy: 0.6870 - val_loss: 0.6882
Epoch 5/50
30/30 ----- 0s 8ms/step - accuracy: 0.7311 - loss: 0.5544 - val_accuracy: 0.7122 - val_loss: 0.6812
Epoch 6/50
30/30 ----- 0s 8ms/step - accuracy: 0.8484 - loss: 0.4224 - val_accuracy: 0.7395 - val_loss: 0.6679
Epoch 7/50
30/30 ----- 0s 8ms/step - accuracy: 0.8879 - loss: 0.3092 - val_accuracy: 0.7437 - val_loss: 0.6483
Epoch 8/50
30/30 ----- 0s 7ms/step - accuracy: 0.9337 - loss: 0.2139 - val_accuracy: 0.6261 - val_loss: 0.6371
Epoch 9/50
30/30 ----- 0s 7ms/step - accuracy: 0.9521 - loss: 0.1601 - val_accuracy: 0.7248 - val_loss: 0.6086
Epoch 10/50
30/30 ----- 0s 8ms/step - accuracy: 0.9632 - loss: 0.1144 - val_accuracy: 0.7332 - val_loss: 0.5835
Epoch 11/50
30/30 ----- 0s 9ms/step - accuracy: 0.9763 - loss: 0.0854 - val_accuracy: 0.7395 - val_loss: 0.5578
Epoch 12/50
30/30 ----- 0s 8ms/step - accuracy: 0.9816 - loss: 0.0707 - val_accuracy: 0.7626 - val_loss: 0.5377
Epoch 13/50
30/30 ----- 0s 5ms/step - accuracy: 0.9900 - loss: 0.0527 - val_accuracy: 0.7374 - val_loss: 0.5252
Epoch 14/50
30/30 ----- 0s 10ms/step - accuracy: 0.9868 - loss: 0.0479 - val_accuracy: 0.7605 - val_loss: 0.5109
Epoch 15/50
30/30 ----- 0s 9ms/step - accuracy: 0.9942 - loss: 0.0327 - val_accuracy: 0.7206 - val_loss: 0.5652
Epoch 16/50
30/30 ----- 0s 5ms/step - accuracy: 0.9916 - loss: 0.0366 - val_accuracy: 0.7332 - val_loss: 0.5548
```

Рисунок 2.12 – Логи процесса обучения с использованием обратных вызовов (callbacks)

2.13 Визуализация процесса обучения

Для анализа динамики обучения построены графики точности (accuracy) и функции потерь (loss). На графиках видно, что точность на обучающей выборке быстро достигает максимума, в то время как на проверочной выборке наблюдаются колебания. Рост val_loss после 12-й эпохи при снижении loss указывает на начало процесса переобучения, который был вовремя остановлен механизмом EarlyStopping.

Визуализация процесса обучения

```
# Графики точности и ошибки
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5)) # Создаем фигуру и два подокна (subplot) для графиков, задаем размер.

# График точности
ax1.plot(history.history['accuracy'], label='Точность на обучающей выборке') # Строим график точности на обучающей выборке.
ax1.plot(history.history['val_accuracy'], label='Точность на проверочной выборке') # Строим график точности на проверочной выборке.
ax1.set_xlabel('Эпоха') # Устанавливаем подпись для оси X.
ax1.set_ylabel('Точность') # Устанавливаем подпись для оси Y.
ax1.set_title('Точность модели') # Устанавливаем заголовок графика.
ax1.legend() # Отображаем легенду графика.
ax1.grid(True, alpha=0.3) # Включаем сетку на графике с прозрачностью 0.3.

# График ошибки
ax2.plot(history.history['loss'], label='Ошибка на обучающей выборке') # Строим график ошибки на обучающей выборке.
ax2.plot(history.history['val_loss'], label='Ошибка на проверочной выборке') # Строим график ошибки на проверочной выборке.
ax2.set_xlabel('Эпоха') # Устанавливаем подпись для оси X.
ax2.set_ylabel('Ошибка') # Устанавливаем подпись для оси Y.
ax2.set_title('Функция потерь') # Устанавливаем заголовок графика.
ax2.legend() # Отображаем легенду графика.
ax2.grid(True, alpha=0.3) # Включаем сетку на графике с прозрачностью 0.3.

plt.tight_layout() # Автоматически корректируем параметры подокна для плотного размещения.
plt.show() # Отображаем все открытые фигуры.
```

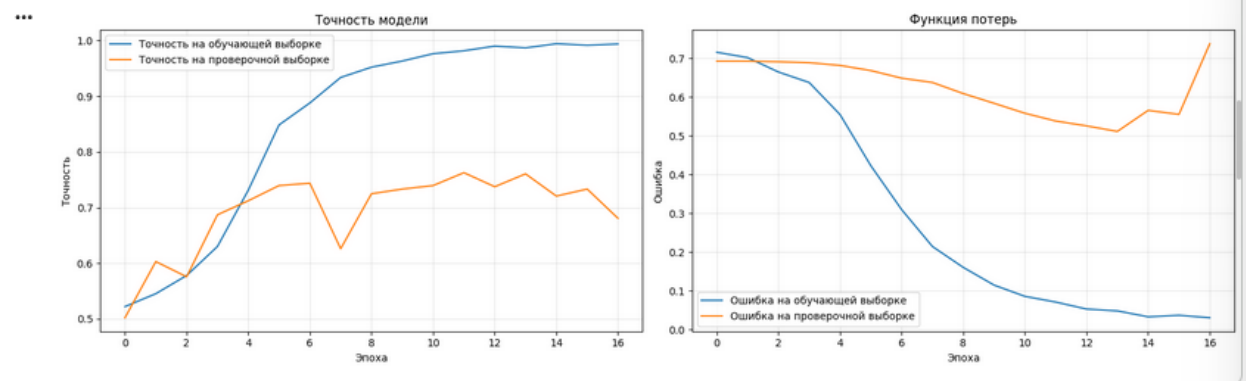


Рисунок 2.13 – Графики точности и функции потерь для обучающей и проверочной выборки

2.14 Оценка модели на тестовой выборке

После завершения обучения проведена проверка модели на отложенных данных. Итоговая точность на тестовой выборке составила **76.26%**. Для детального анализа качества классификации рассчитаны метрики precision, recall и f1-score для каждого класса, а также сформирован отчет по классификации.

```

# Предсказания на тестовой выборке
y_pred_prob = model.predict(x_test) # Получаем предсказанные вероятности для тестовой выборки.
y_pred = (y_pred_prob > 0.5).astype(int).flatten() # Преобразуем вероятности в бинарные предсказания (0 или 1) и делаем плоский массив.

# Вычисление точности
accuracy = (y_pred == y_test).mean() # Вычисляем среднюю точность, сравнивая предсказания с истинными метками.

print(f'Точность на тестовой выборке: {accuracy * 100:.2f}%') # Выводим точность на тестовой выборке в процентах.

# Подробная статистика по классам
from sklearn.metrics import classification_report, confusion_matrix # Импортируем функции для отчета по классификации и матрицы ошибок.

print('\nОтчет по классификации:') # Выводим заголовок для отчета по классификации.
print(classification_report(y_test, y_pred, target_names=class_names)) # Выводим подробный отчет по классификации с метриками.

# Матрица ошибок
cm = confusion_matrix(y_test, y_pred) # Вычисляем матрицу ошибок.
print('\nМатрица ошибок:') # Выводим заголовок для матрицы ошибок.
print(cm) # Выводим саму матрицу ошибок.

# Визуализация матрицы ошибок
from sklearn.metrics import ConfusionMatrixDisplay # Импортируем класс для отображения матрицы ошибок.

fig, ax = plt.subplots(figsize=(8, 6)) # Создаем фигуру и одно подокно для визуализации матрицы ошибок.
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names) # Создаем объект ConfusionMatrixDisplay с матрицей ошибок.
disp.plot(ax=ax, cmap='Blues') # Отображаем матрицу ошибок на заданном подокне с цветовой картой 'Blues'.
plt.title('Матрица ошибок') # Устанавливаем заголовок для графика матрицы ошибок.
plt.tight_layout() # Автоматически корректируем параметры подокна.
plt.show() # Отображаем график.

```

Рисунок 2.14 – Вычисление итоговой точности и формирование отчета по классификации

Для визуализации качества предсказаний построена матрица ошибок. Из неё видно, что модель верно определила 158 негативных отзывов и 205 позитивных. Ошибки распределились следующим образом: 80 негативных отзывов были ошибочно приняты за позитивные, и 33 позитивных — за негативные. Модель лучше справляется с определением позитивной тональности (выше полнота/recall).

```

15/15 ————— 1s 17ms/step
Точность на тестовой выборке: 76.26%
...
Отчёт по классификации:
              precision    recall  f1-score   support

Негативный отзыв      0.83      0.66      0.74      238
Позитивный отзыв      0.72      0.86      0.78      238

   accuracy              0.76      476
  macro avg              0.77      476
 weighted avg              0.77      476

Матрица ошибок:
[[158  80]
 [ 33 205]]

```

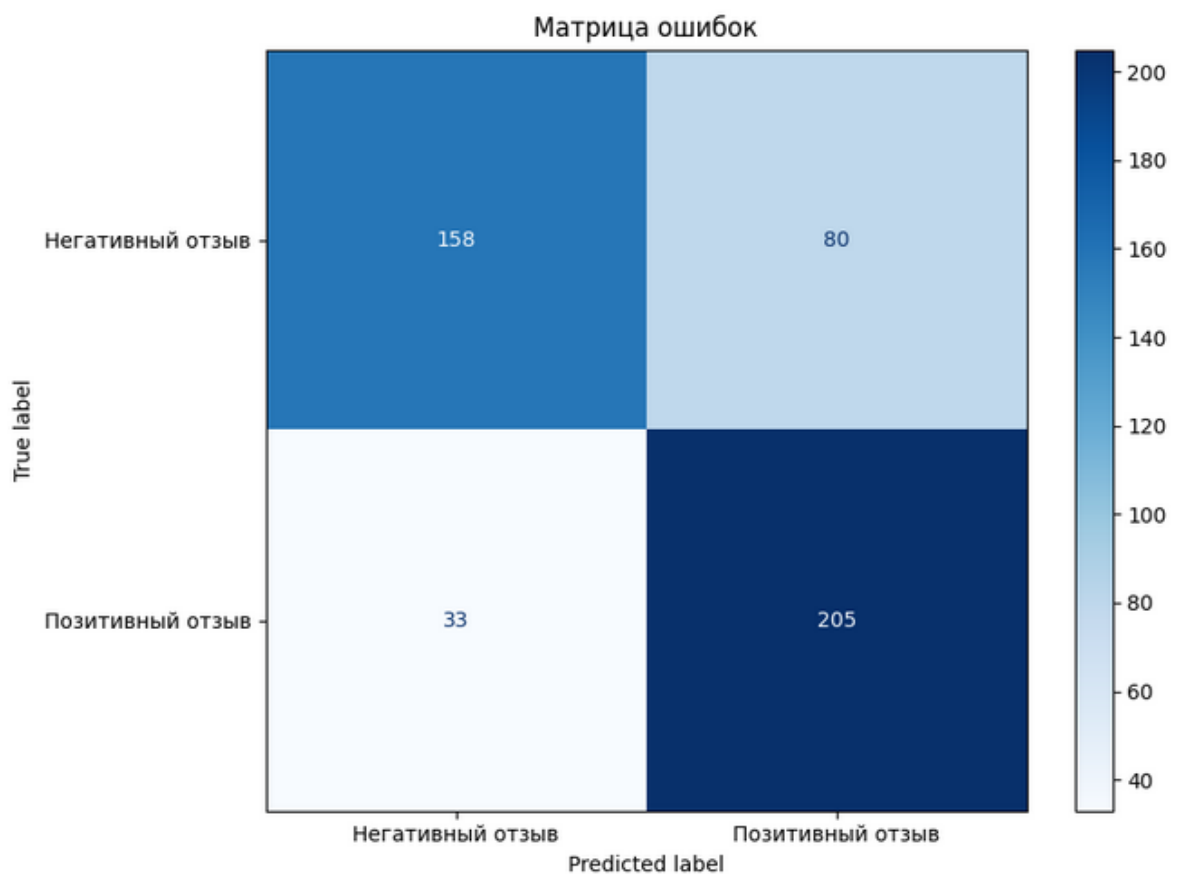


Рисунок 2.15 – Матрица ошибок классификации отзывов Tesla

2.15 Создание функции для предсказания тональности

Разработана функция `predict_sentiment`, позволяющая классифицировать произвольный текст. Функция выполняет токенизацию входной строки, дополняет её до нужной длины (`WIN_SIZE=50`), делает предсказание вероятности и выводит вердикт (позитивный или негативный отзыв) вместе с процентной уверенностью модели.

```

# Функция для предсказания тональности текста
def predict_sentiment(text, show_prob=True): # Определяем функцию predict_sentiment, принимающую текст и флаг show_prob.
    # Преобразуем текст в последовательность
    seq = tokenizer.texts_to_sequences([text])[0] # Преобразуем входной текст в последовательность индексов с помощью токени:

    # Нарезаем на окна (берём первое окно, если текст длинный)
    if len(seq) < WIN_SIZE: # Проверяем, если длина последовательности меньше WIN_SIZE.
        # Дополняем нулями, если текст короче окна
        seq = seq + [0] * (WIN_SIZE - len(seq)) # Дополняем последовательность нулями до WIN_SIZE.
    else: # Если длина последовательности больше или равна WIN_SIZE.
        seq = seq[:WIN_SIZE] # Обрезаем последовательность до WIN_SIZE.

    # Преобразуем в numpy массив
    seq = np.array([seq]) # Преобразуем последовательность в массив NumPy, добавляя размерность для батча.

    # Предсказание
    prob = model.predict(seq, verbose=0)[0][0] # Получаем вероятность позитивного класса от модели.

    # Определяем класс
    if prob > 0.5: # Если вероятность позитивного класса больше 0.5.
        sentiment = "Позитивный отзыв" # Отмечаем как позитивный отзыв.
    else: # В противном случае.
        sentiment = "Негативный отзыв" # Отмечаем как негативный отзыв.

    if show_prob: # Если флаг show_prob установлен в True.
        print(f'Текст: "{text}"') # Выводим исходный текст.
        print(f'Предсказание: {sentiment} (вероятность позитивного: {prob:.2%})') # Выводим предсказанную тональность и веро:
        print('-' * 80) # Выводим разделительную линию.

    return sentiment, prob # Возвращаем предсказанную тональность и вероятность.

# Тестовые примеры
test_examples = [ # Определяем список тестовых примеров.
    "Tesla is an amazing car with great performance",
    "I hate this car, worst purchase ever",
    "The autopilot feature is incredible and saves time",
    "Battery drains too fast, very disappointed",
    "Best electric vehicle on the market",
    "Too expensive and not worth the money"
]

print('Проверка на тестовых примерах:\n') # Выводим заголовок перед проверкой.
for example in test_examples: # Проходим по каждому тестовому примеру.
    predict_sentiment(example) # Вызываем функцию предсказания тональности для каждого примера.

```

Рисунок 2.16 – Программный код функции для классификации новых
ОТЗЫВОВ

Проведена проверка работы нейросети на шести тестовых фразах на английском языке. Модель успешно распознала явно позитивные утверждения (например, про «amazing car» с вероятностью 74.40%). Однако на некоторых негативных примерах (про «hate this car») модель показала ложноположительный результат, что говорит о необходимости дальнейшей настройки эмбедингов или увеличения объема специфичных обучающих данных для повышения чувствительности к негативной лексике.

... Проверка на тестовых примерах:

Текст: "Tesla is an amazing car with great performance"

Предсказание: Позитивный отзыв (вероятность позитивного: 74.40%)

Текст: "I hate this car, worst purchase ever"

Предсказание: Позитивный отзыв (вероятность позитивного: 57.72%)

Текст: "The autopilot feature is incredible and saves time"

Предсказание: Позитивный отзыв (вероятность позитивного: 67.75%)

Текст: "Battery drains too fast, very disappointed"

Предсказание: Позитивный отзыв (вероятность позитивного: 64.39%)

Текст: "Best electric vehicle on the market"

Предсказание: Позитивный отзыв (вероятность позитивного: 66.53%)

Текст: "Too expensive and not worth the money"

Предсказание: Позитивный отзыв (вероятность позитивного: 67.05%)

Рисунок 2.17 – Результаты проверки обученной модели на тестовых примерах

Задание 3. Классификация 20 русских писателей.

Задача: распознать авторство фрагмента текста среди 20 русских писателей. Это расширение задачи из задания 1 с 6 классов до 20, что подразумевает работу с большей по объёму базой данных и требует подбора параметров, обеспечивающих достаточное число обучающих примеров на каждый класс.

В соответствии с темой лабораторной работы – Bag of Words и Embedding – обучаются две модели: Bag of Words с двумя скрытыми Dense-слоями и Embedding + Flatten + Dense. Цель: достичь точности не менее **95 %** для модели Bag of Words и 75–80 % для модели Embedding на проверочной выборке.

3.1 Подключение необходимых библиотек

Для решения задачи классификации 20 авторов подключается стандартный набор библиотек: **numpy** для работы с массивами, **tensorflow.keras** (utils, Sequential, Dense, Dropout, SpatialDropout1D, BatchNormalization, Embedding, Flatten, Tokenizer) для построения и обучения

нейронных сетей, **sklearn.metrics** для построения матрицы ошибок, **gdown** для загрузки датасета, а также модули **os**, **time**, **re** и **matplotlib.pyplot**.

▼ Импорт библиотек

```
[ ] import numpy as np
    from tensorflow.keras import utils
    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Dense, Dropout, SpatialDropout1D, BatchNormalization, Embedding, Flatten
    from tensorflow.keras.preprocessing.text import Tokenizer
    from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

    import gdown
    import os
    import time
    import re
    import matplotlib.pyplot as plt

    %matplotlib inline
```

Рисунок 3.1 – Подключение библиотек

3.2 Загрузка датасета и просмотр содержимого

Датасет 20writers.zip скачивается и распаковывается в директорию. В архиве содержится 20 файлов формата «Автор.txt». Список авторов включает: Беляев, Булгаков, Васильев, Гоголь, Гончаров, Горький, Грибоедов, Достоевский, Каверин, Катаев, Куприн, Лермонтов, Лесков, Носов, Пастернак, Пушкин, Толстой, Тургенев, Чехов, Шолохов.

Загрузка данных

```
[ ] # Скачиваем базу 20 писателей
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/17/20writers.zip', None, quiet=True)

[ ] '20writers.zip'
```

```
[ ] # Распаковка архива
!unzip -qo 20writers.zip -d 20writers/

FILE_DIR = '20writers'
files = sorted(os.listdir(FILE_DIR))
print(f'Всего файлов: {len(files)}')
print('Файлы:')
for f in files:
    print(f' {f}')
```

```
*** Всего файлов: 20
Файлы:
Беляев.txt
Булгаков.txt
Васильев.txt
Гоголь.txt
Гончаров.txt
Горький.txt
Грибоедов.txt
Достоевский.txt
Каверин.txt
Катаев.txt
Куприн.txt
Лермонтов.txt
Лесков.txt
Носов.txt
Пастернак.txt
Пушкин.txt
Толстой.txt
Тургенев.txt
Чехов.txt
Шолохов.txt
```

Рисунок 3.2 – Загрузка и распаковка архива 20writers.zip, список файлов

3.3 Загрузка текстов и формирование классов

Каждый файл читается в одну строку, переводы строк заменяются на пробелы. Объём текстов значительно различается: от Грибоедова (969 109 символов) до Чехова (6 611 627 символов). Различие в 6,8 раза между самым маленьким и самым большим корпусом приводит к дисбалансу классов, что требует балансировки.

▼ Загрузка текстов

Каждый файл — это один писатель. Имя писателя извлекаем из имени файла, убирая расширение `.txt`.

```
[ ] ▶ CLASS_LIST = []          # Имена писателей
text_all = []              # Тексты по классам

for file_name in sorted(os.listdir(FILE_DIR)):
    # Имя писателя = имя файла без .txt
    class_name = file_name.replace('.txt', '')

    # Читаем файл
    with open(f'{FILE_DIR}/{file_name}', 'r', encoding='utf-8') as f:
        text = f.read().replace('\n', ' ')

    CLASS_LIST.append(class_name)
    text_all.append(text)

    print(f' {class_name:<25}: {len(text)} символов')

CLASS_COUNT = len(CLASS_LIST)
print(f'\nВсего классов: {CLASS_COUNT}')
```

```
▼ ... Беляев                : 2255254 символов
      Булгаков             : 2001064 символов
      Васильев             : 2952792 символов
      Гоголь               : 1965119 символов
      Гончаров             : 3104712 символов
      Горький              : 2523380 символов
      Грибоедов            : 969109 символов
      Достоевский          : 5178950 символов
      Каверин              : 1992830 символов
      Катаев               : 2579246 символов
      Куприн               : 2328900 символов
      Лермонтов            : 1972541 символов
      Лесков               : 2152214 символов
      Носов                : 2399413 символов
      Пастернак            : 3089426 символов
      Пушкин               : 3408339 символов
      Толстой              : 3357061 символов
      Тургенев             : 1980206 символов
      Чехов                : 6611627 символов
      Шолохов              : 3386268 символов
```

Всего классов: 20

Рисунок 3.3 – Загрузка текстов и объёмы корпусов всех 20 авторов

3.4 Балансировка классов

Для устранения дисбаланса применена стратегия обрезки всех текстов до длины самого короткого корпуса (**969 109 символов**). После этого все 20 текстов имеют одинаковый объём, что гарантирует равное число примеров каждого класса при нарезке «окнами».

Балансировка классов

Объёмы текстов у разных писателей сильно отличаются. Чтобы у всех классов было примерно одинаковое количество примеров — обрезаем все тексты до длины самого короткого.

```
# Минимальная длина среди всех текстов
min_len = min(len(t) for t in text_all)
print(f'Минимальная длина: {min_len} символов')

# Балансируем — обрезаем все тексты
text_balanced = [t[:min_len] for t in text_all]

print('\nПосле балансировки все тексты по {} символов'.format(min_len))

Минимальная длина: 969109 символов

После балансировки все тексты по 969109 символов
```

Рисунок 3.4 — Балансировка классов: обрезка всех текстов до минимальной длины

3.5 Деление на обучающую и тестовую выборки

Разбивка реализована циклом по всем классам. Параметр `TEST_SHARE` = 0.2 (20% на тест). На один класс приходится 775 287 символов на обучение и 193 822 символа на тест.

✓ Деление на обучающую (80%) и тестовую (20%) выборки через цикл

По требованию задания — применяем **цикл** для разбивки.

```
TEST_SHARE = 0.2 # 20% на проверочную

text_train = []
text_test = []

# Цикл по всем классам
for cls in range(CLASS_COUNT):
    text = text_balanced[cls]

    # Граница: 80% — обучение, 20% — тест
    train_len = int(len(text) * (1 - TEST_SHARE))

    text_train.append(text[:train_len])
    text_test.append(text[train_len:])

print(f'Обучающая выборка на класс: {len(text_train[0])} символов ({(1-TEST_SHARE)*100:.0f}%)')
print(f'Тестовая выборка на класс: {len(text_test[0])} символов ({TEST_SHARE*100:.0f}%)')

Обучающая выборка на класс: 775287 символов (80%)
Тестовая выборка на класс: 193822 символов (20%)
```

Рисунок 3.5 — Деление на обучающую и тестовую выборки через цикл

3.6 Параметры преобразования

Параметры преобразования

VOCAB_SIZE	= 20000	# Объём словаря (увеличен для 20 классов)
WIN_SIZE	= 1000	# Длина окна в словах
WIN_HOP	= 100	# Шаг окна (плотная нарезка)

Рисунок 3.6 – Параметры преобразования текстов

3.7 Токенизация и построение частотного словаря

Создаётся токенизатор с фильтрацией и приведением к нижнему регистру. Полный словарь содержит 185 786 уникальных слов, из которых используются 20 000 самых частотных. Время токенизации – около 6,2 секунды.

- Токенизация

```

class timex:
    def __enter__(self):
        self.t = time.time()
        return self
    def __exit__(self, type, value, traceback):
        print('Время обработки: {:.2f} с'.format(time.time() - self.t))

```

```

# Создание токенизатора и обучение на тренировочных текстах
with timex():
    tokenizer = Tokenizer(num_words=VOCAB_SIZE,
                          filters='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~\t\n\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\x10000',
                          lower=True, split=' ',
                          oov_token='неизвестное_слово',
                          char_level=False)
    tokenizer.fit_on_texts(text_train)

print(f'Размер словаря: {len(tokenizer.word_index)}')

```

Время обработки: 6.18 с
Размер словаря: 185786

```

# Преобразование в последовательности индексов
with timex():
    seq_train = tokenizer.texts_to_sequences(text_train)
    seq_test = tokenizer.texts_to_sequences(text_test)

print('Длины последовательностей (обучающие):')
for cls in range(CLASS_COUNT):
    print(f' {CLASS_LIST[cls]:<25}: {len(seq_train[cls]):>8} токенов')

```

... Время обработки: 4.21 с
Длины последовательностей (обучающие):

Беляев	:	116009	токенов
Булгаков	:	116731	токенов
Васильев	:	117965	токенов
Гоголь	:	118315	токенов
Гончаров	:	124127	токенов
Горький	:	127082	токенов
Грибоедов	:	123210	токенов
Достоевский	:	117922	токенов
Каверин	:	119103	токенов
Катаев	:	115814	токенов
Куприн	:	117412	токенов
Лермонтов	:	121526	токенов
Лесков	:	125223	токенов
Носов	:	118651	токенов
Пастернак	:	113345	токенов
Пушкин	:	118239	токенов

Рисунок 3.7 – Токенизация и длины токенизированных последовательностей по авторам

3.8 Формирование выборок методом скользящего окна

После нарезки обучающая выборка содержит 23 672 отрезка, тестовая – 5 766 отрезков. Метки классов формируются в формате **One-Hot Encoding**. На каждый класс приходится примерно 1150–1260 окон.

Формирование выборок методом скользящего окна

```
def split_sequence(sequence, win_size, hop):  
    return [sequence[i:i + win_size] for i in range(0, len(sequence) - win_size + 1, hop)]  
  
def vectorize_sequence(seq_list, win_size, hop):  
    class_count = len(seq_list)  
    x, y = [], []  
    for cls in range(class_count):  
        vectors = split_sequence(seq_list[cls], win_size, hop)  
        x += vectors  
        y += [utils.to_categorical(cls, class_count)] * len(vectors)  
    return np.array(x), np.array(y)
```

Рисунок 3.8 – Функции split_sequence и vectorize_sequence

3.9 Преобразование в формат Bag of Words

Последовательности отрезков преобразуются в матрицы счётчиков частот. Полученные матрицы имеют размерность (число_примеров, 20 000).

```

▶ with timex():
    x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
    x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

    print(f'Обучающая выборка: {x_train.shape}')
    print(f'Тестовая выборка: {x_test.shape}')

    print('\nКоличество окон по классам в обучающей выборке:')
    y_labels = np.argmax(y_train, axis=1)
    for cls in range(CLASS_COUNT):
        count = np.sum(y_labels == cls)
        print(f' {CLASS_LIST[cls]:<25}: {count} окон')

```

```

*** Время обработки: 2.43 с
Обучающая выборка: (23672, 1000)
Тестовая выборка: (5766, 1000)

Количество окон по классам в обучающей выборке:
Беляев           : 1151 окон
Булгаков         : 1158 окон
Васильев         : 1170 окон
Гоголь           : 1174 окон
Гончаров         : 1232 окон
Горький          : 1261 окон
Грибоедов        : 1223 окон
Достоевский      : 1170 окон
Каверин          : 1182 окон
Катаев           : 1149 окон
Куприн           : 1165 окон
Лермонтов        : 1206 окон
Лесков           : 1243 окон
Носов            : 1177 окон
Пастернак        : 1124 окон
Пушкин           : 1173 окон
Толстой          : 1177 окон
Тургенев         : 1190 окон
Чехов            : 1198 окон
Шолохов          : 1149 окон

```

Рисунок 3.9 – Размерности выборок и распределение окон по классам

3.10 Сервисные функции

Для автоматизации процесса созданы функции `compile_train_model` (компиляция, обучение, графики) и `eval_model` (предсказание и построение матрицы ошибок). Это позволяет избежать дублирования кода для разных архитектур.

```

def compile_train_model(model, x_train, y_train, x_val, y_val,
                        epochs=30, batch_size=128, figsize=(20, 5)):
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    model.summary()

    history = model.fit(x_train, y_train,
                        epochs=epochs,
                        batch_size=batch_size,
                        validation_data=(x_val, y_val))

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=figsize)
    fig.suptitle('График обучения')
    ax1.plot(history.history['accuracy'], label='Обучающая')
    ax1.plot(history.history['val_accuracy'], label='Проверочная')
    ax1.set_xlabel('Эпоха')
    ax1.set_ylabel('Точность')
    ax1.legend()

    ax2.plot(history.history['loss'], label='Обучающая')
    ax2.plot(history.history['val_loss'], label='Проверочная')
    ax2.set_xlabel('Эпоха')
    ax2.set_ylabel('Ошибка')
    ax2.legend()
    plt.show()

```

Рисунок 3.11 – Функция compile_train_model

```

def eval_model(model, x, y_true, class_labels=[], title='', figsize=(15, 15)):
    y_pred = model.predict(x)
    cm = confusion_matrix(np.argmax(y_true, axis=1), np.argmax(y_pred, axis=1), normalize='true')
    cm = np.around(cm, 3)

    fig, ax = plt.subplots(figsize=figsize)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_labels)
    disp.plot(ax=ax)
    plt.gca().images[-1].colorbar.remove()
    plt.title(f'{title}: матрица ошибок', fontsize=16)
    fig.autofmt_xdate(rotation=45)
    plt.show()

    accuracy = 100. * cm.diagonal().mean()
    correct = sum(1 for cls in range(len(class_labels)) if np.argmax(cm[cls]) == cls)
    print(f'\nВерно распознано классов: {correct}/{len(class_labels)}')
    print(f'Средняя точность: {accuracy:.1f}%')
    return accuracy

def compile_train_eval(model, x_train, y_train, x_test, y_test, title='', epochs=20):
    compile_train_model(model, x_train, y_train, x_test, y_test, epochs=epochs)
    return eval_model(model, x_test, y_test, class_labels=CLASS_LIST, title=title)

```

Рисунок 3.12 – Функции eval_model и compile_train_eval

3.11 Модель 1: Bag of Words (цель $\geq 95\%$)

Архитектура: Два скрытых слоя Dense(400) и Dense(200) с использованием Dropout и BatchNormalization. Увеличение числа нейронов (до 400) обусловлено сложностью задачи (20 классов).

Параметры: ~8 млн обучаемых параметров.

Результаты: Точность на обучающей выборке быстро достигла 100%.
На проверочной выборке точность стабилизировалась на уровне **90,4 %**.

Анализ: Модель не достигла цели в 95% (нехватка 4,6%). Сложнее всего распознаются Гоголь (36%) и Катаев, которые часто путаются между собой. Васильев, Толстой и Пушкин распознаются почти безошибочно.

✓ Модель 1: Bag of Words (цель > 95%)

```
1 model_bow = Sequential()
   model_bow.add(Dense(400, input_dim=VOCAB_SIZE, activation='relu'))
   model_bow.add(Dropout(0.4))
   model_bow.add(BatchNormalization())
   model_bow.add(Dense(200, activation='relu'))
   model_bow.add(Dropout(0.3))
   model_bow.add(BatchNormalization())
   model_bow.add(Dense(CLASS_COUNT, activation='softmax'))

   acc_bow = compile_train_eval(model_bow, x_train_01, y_train, x_test_01, y_test,
                               title='Bag of Words', epochs=20)
```

Рисунок 3.13 – Архитектура Модели 1: Bag of Words

```
*** /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning:
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 400)	8,000,400
dropout (Dropout)	(None, 400)	0
batch_normalization (BatchNormalization)	(None, 400)	1,600
dense_1 (Dense)	(None, 200)	80,200
dropout_1 (Dropout)	(None, 200)	0
batch_normalization_1 (BatchNormalization)	(None, 200)	800
dense_2 (Dense)	(None, 20)	4,020

Total params: 8,087,020 (30.85 MB)
Trainable params: 8,085,820 (30.84 MB)
Non-trainable params: 1,200 (4.69 KB)

Рисунок 3.14 – Сводная таблица слоёв Модели 1

Total params: 8,087,020 (30.85 MB)
 Trainable params: 8,085,820 (30.84 MB)
 Non-trainable params: 1,200 (4.69 KB)

Epoch 1/20
 185/185 ————— 13s 40ms/step - accuracy: 0.9511 - loss: 0.2224 - val_accuracy: 0.8852 - val_loss: 0.5584

Epoch 2/20
 185/185 ————— 2s 13ms/step - accuracy: 1.0000 - loss: 0.0052 - val_accuracy: 0.8916 - val_loss: 0.4220

Epoch 3/20
 185/185 ————— 2s 11ms/step - accuracy: 1.0000 - loss: 0.0023 - val_accuracy: 0.8965 - val_loss: 0.3869

Epoch 4/20
 185/185 ————— 2s 13ms/step - accuracy: 1.0000 - loss: 0.0013 - val_accuracy: 0.8966 - val_loss: 0.3759

Epoch 5/20
 185/185 ————— 2s 12ms/step - accuracy: 1.0000 - loss: 9.6005e-04 - val_accuracy: 0.8996 - val_loss: 0.3570

Epoch 6/20
 185/185 ————— 2s 11ms/step - accuracy: 1.0000 - loss: 6.6557e-04 - val_accuracy: 0.8994 - val_loss: 0.3534

Epoch 7/20
 185/185 ————— 2s 11ms/step - accuracy: 1.0000 - loss: 4.7552e-04 - val_accuracy: 0.8998 - val_loss: 0.3490

Epoch 8/20
 185/185 ————— 2s 13ms/step - accuracy: 1.0000 - loss: 3.8655e-04 - val_accuracy: 0.8989 - val_loss: 0.3487

Epoch 9/20
 185/185 ————— 2s 13ms/step - accuracy: 1.0000 - loss: 3.3172e-04 - val_accuracy: 0.9027 - val_loss: 0.3488

Epoch 10/20
 185/185 ————— 3s 13ms/step - accuracy: 1.0000 - loss: 2.5524e-04 - val_accuracy: 0.9032 - val_loss: 0.3380

Epoch 11/20
 185/185 ————— 2s 12ms/step - accuracy: 1.0000 - loss: 2.3025e-04 - val_accuracy: 0.9018 - val_loss: 0.3411

Epoch 12/20
 185/185 ————— 2s 12ms/step - accuracy: 1.0000 - loss: 1.7940e-04 - val_accuracy: 0.9017 - val_loss: 0.3396

Epoch 13/20
 185/185 ————— 2s 13ms/step - accuracy: 1.0000 - loss: 1.4656e-04 - val_accuracy: 0.9022 - val_loss: 0.3375

Epoch 14/20
 185/185 ————— 2s 12ms/step - accuracy: 1.0000 - loss: 1.3028e-04 - val_accuracy: 0.9041 - val_loss: 0.3329

Epoch 15/20
 185/185 ————— 3s 14ms/step - accuracy: 1.0000 - loss: 1.1369e-04 - val_accuracy: 0.9029 - val_loss: 0.3346

Epoch 16/20
 185/185 ————— 3s 15ms/step - accuracy: 1.0000 - loss: 9.9829e-05 - val_accuracy: 0.9039 - val_loss: 0.3317

Epoch 17/20
 185/185 ————— 5s 12ms/step - accuracy: 1.0000 - loss: 8.7910e-05 - val_accuracy: 0.9048 - val_loss: 0.3300

Epoch 18/20
 185/185 ————— 2s 13ms/step - accuracy: 1.0000 - loss: 7.5003e-05 - val_accuracy: 0.9044 - val_loss: 0.3299

Epoch 19/20
 185/185 ————— 3s 14ms/step - accuracy: 1.0000 - loss: 6.5277e-05 - val_accuracy: 0.9036 - val_loss: 0.3285

Epoch 20/20
 185/185 ————— 5s 11ms/step - accuracy: 1.0000 - loss: 6.0025e-05 - val_accuracy: 0.9044 - val_loss: 0.3294

Рисунок 3.15 – Журнал обучения Модели 1: Bag of Words

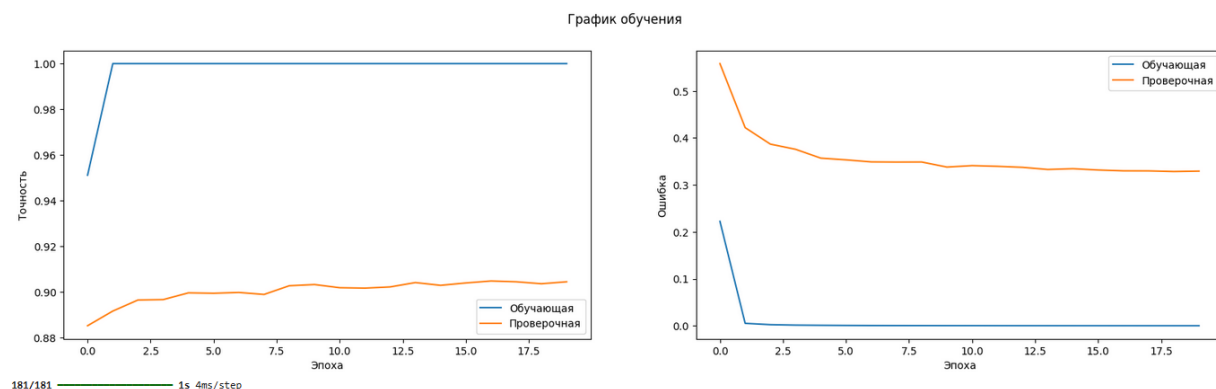


Рисунок 3.16 – График процесса обучения Модели 1: Bag of Words

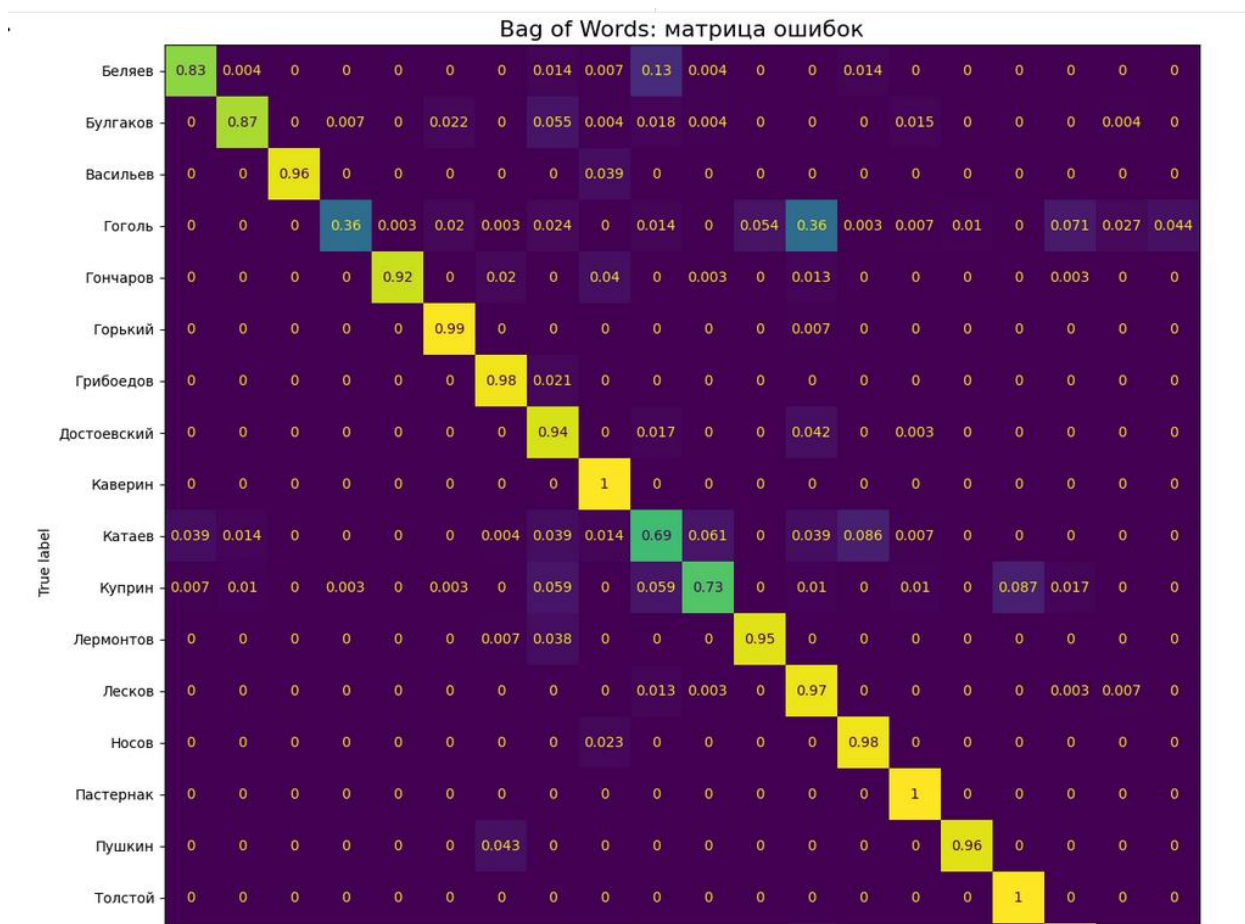


Рисунок 3.17 – Матрица ошибок Модели 1: Bag of Words

Верно распознано классов: 20/20
Средняя точность: 90.4%

Рисунок 3.18 – Текстовый отчёт о точности Модели 1

3.12 Модель 2: Embedding + Flatten + Dense (цель 75–80 %)

Архитектура: Embedding(200) → SpatialDropout1D → Flatten → Dense(200).

Результаты: Точность на проверочной выборке составила **74,8 %**.

Анализ: Результат находится на нижней границе целевого диапазона. Использование Flatten после Embedding на окнах в 1000 слов создает огромное пространство признаков (200 000), что затрудняет обобщение по сравнению с BoW.

Модель 2: Embedding + Flatten (цель 75-80%)

```
model_emb = Sequential()
model_emb.add(Embedding(input_dim=VOCAB_SIZE, output_dim=200, input_length=WIN_SIZE))
model_emb.add(SpatialDropout1D(0.3))
model_emb.add(Flatten())
model_emb.add(BatchNormalization())
model_emb.add(Dense(200, activation='relu'))
model_emb.add(Dropout(0.3))
model_emb.add(BatchNormalization())
model_emb.add(Dense(CLASS_COUNT, activation='softmax'))

acc_emb = compile_train_eval(model_emb, x_train, y_train, x_test, y_test,
                             title='Embedding', epochs=15)
```

Рисунок 3.19 – Архитектура Модели 2: Embedding + Flatten

```
** /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning:
  warnings.warn(
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
spatial_dropout1d (SpatialDropout1D)	?	0
flatten (Flatten)	?	0 (unbuilt)
batch_normalization_2 (BatchNormalization)	?	0 (unbuilt)
dense_3 (Dense)	?	0 (unbuilt)
dropout_2 (Dropout)	?	0
batch_normalization_3 (BatchNormalization)	?	0 (unbuilt)
dense_4 (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
```

Рисунок 3.20 – Сводная таблица слоёв Модели 2

```

non-trainable params: 0 (0.00 B)
Epoch 1/15
185/185 ————— 17s 60ms/step - accuracy: 0.6836 - loss: 1.2173 - val_accuracy: 0.4348 - val_loss: 2.3345
Epoch 2/15
185/185 ————— 5s 28ms/step - accuracy: 0.9998 - loss: 0.0254 - val_accuracy: 0.7126 - val_loss: 1.8342
Epoch 3/15
185/185 ————— 6s 34ms/step - accuracy: 1.0000 - loss: 0.0076 - val_accuracy: 0.7180 - val_loss: 1.2403
Epoch 4/15
185/185 ————— 5s 29ms/step - accuracy: 1.0000 - loss: 0.0042 - val_accuracy: 0.7284 - val_loss: 1.0082
Epoch 5/15
185/185 ————— 5s 28ms/step - accuracy: 1.0000 - loss: 0.0027 - val_accuracy: 0.7336 - val_loss: 0.9477
Epoch 6/15
185/185 ————— 5s 28ms/step - accuracy: 1.0000 - loss: 0.0020 - val_accuracy: 0.7320 - val_loss: 0.9405
Epoch 7/15
185/185 ————— 5s 28ms/step - accuracy: 1.0000 - loss: 0.0014 - val_accuracy: 0.7378 - val_loss: 0.9177
Epoch 8/15
185/185 ————— 5s 29ms/step - accuracy: 1.0000 - loss: 0.0011 - val_accuracy: 0.7371 - val_loss: 0.9086
Epoch 9/15
185/185 ————— 6s 32ms/step - accuracy: 1.0000 - loss: 8.9901e-04 - val_accuracy: 0.7385 - val_loss: 0.9021
Epoch 10/15
185/185 ————— 11s 34ms/step - accuracy: 1.0000 - loss: 7.1788e-04 - val_accuracy: 0.7379 - val_loss: 0.8910
Epoch 11/15
185/185 ————— 5s 29ms/step - accuracy: 1.0000 - loss: 5.9217e-04 - val_accuracy: 0.7409 - val_loss: 0.8826
Epoch 12/15
185/185 ————— 6s 35ms/step - accuracy: 1.0000 - loss: 4.7877e-04 - val_accuracy: 0.7442 - val_loss: 0.8713
Epoch 13/15
185/185 ————— 6s 33ms/step - accuracy: 1.0000 - loss: 4.1768e-04 - val_accuracy: 0.7463 - val_loss: 0.8670
Epoch 14/15
185/185 ————— 5s 29ms/step - accuracy: 1.0000 - loss: 3.5848e-04 - val_accuracy: 0.7503 - val_loss: 0.8584
Epoch 15/15
185/185 ————— 5s 29ms/step - accuracy: 1.0000 - loss: 3.0713e-04 - val_accuracy: 0.7494 - val_loss: 0.8512

```

Рисунок 3.21 – Журнал обучения Модели 2: Embedding

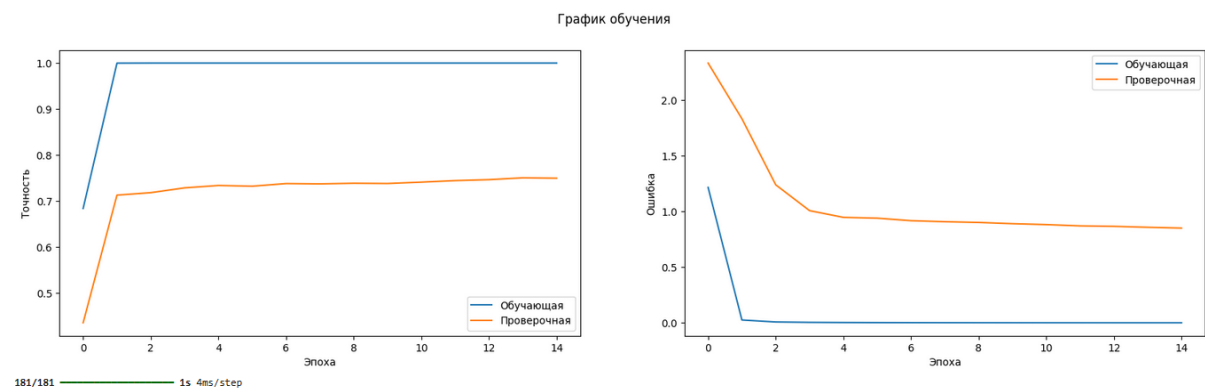


Рисунок 3.22 – График процесса обучения Модели 2: Embedding

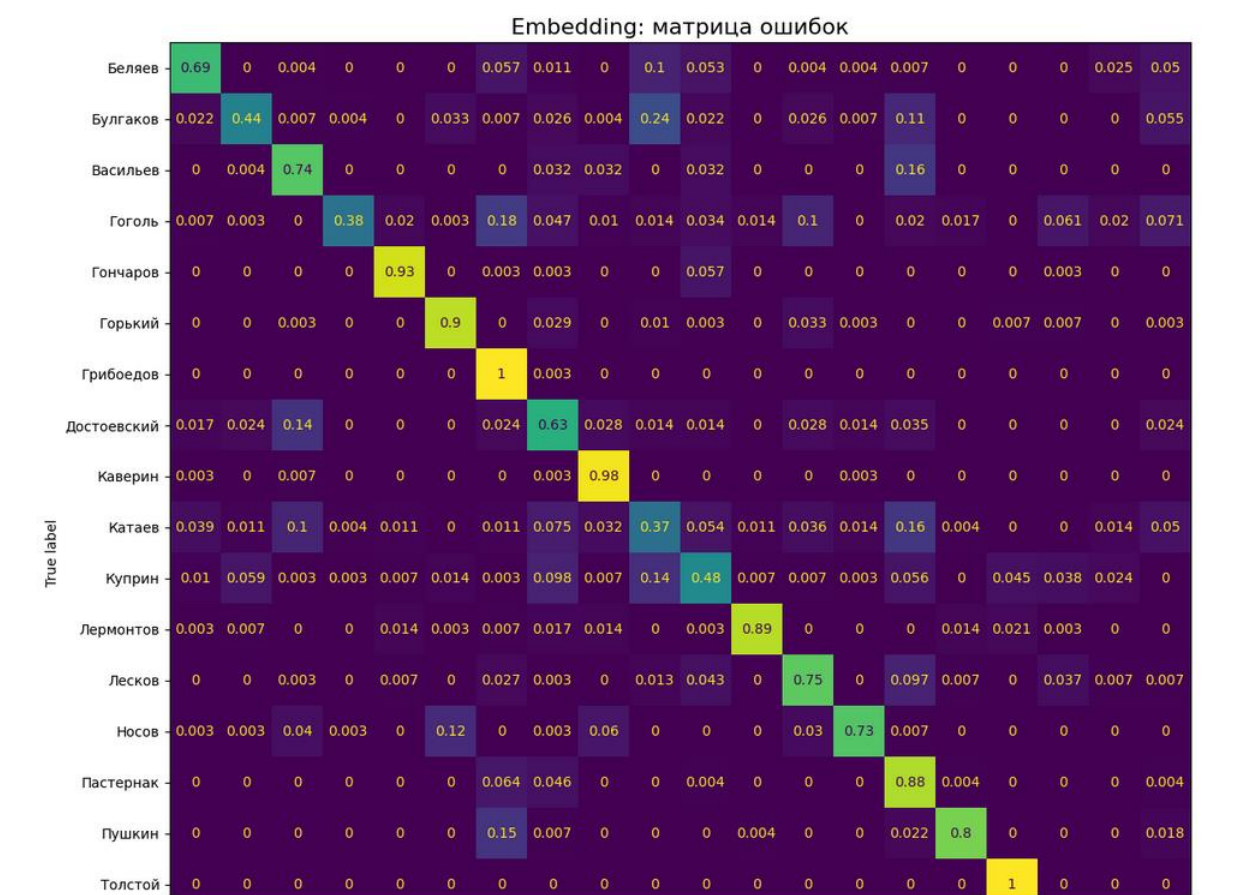


Рисунок 3.23 – Матрица ошибок Модели 2: Embedding

Верно распознано классов: 20/20
Средняя точность: 74.8%

Рисунок 3.24 – Текстовый отчёт о точности Модели 2

3.13 Сравнение и выводы

Модель	Средняя точность
Модель 1: Bag of Words Dense(400→200)	90,4 %
Модель 2: Embedding(200) + Flatten + Dense(200)	74,8 %

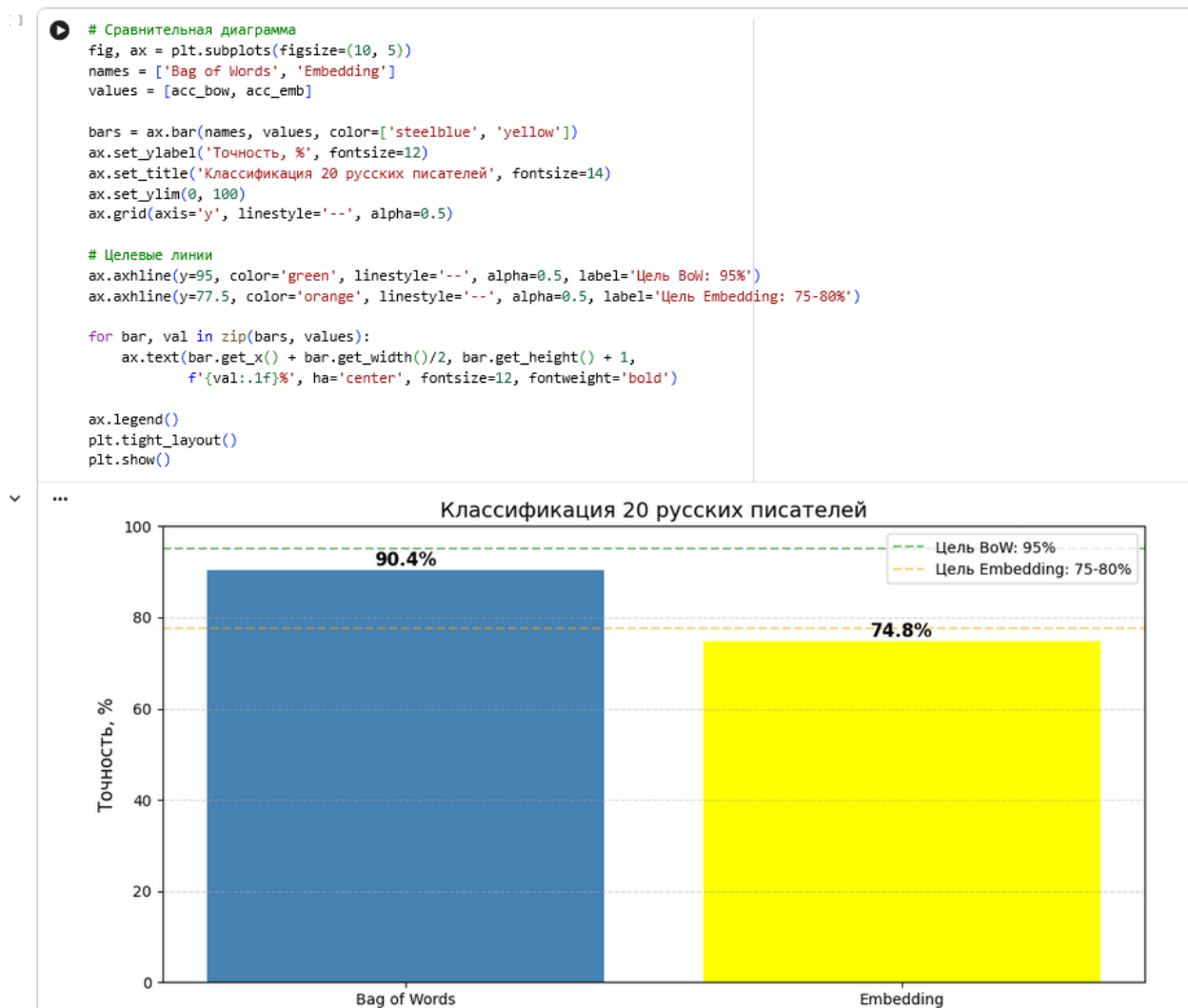


Рисунок 3.26 – Сравнительная диаграмма точности двух моделей с целевыми линиями

Общий вывод:

Подход **Bag of Words** существенно превосходит Embedding + Flatten в задаче определения авторства (разрыв 15,6%). Писательский стиль лучше всего улавливается через частотное распределение слов.

В ходе работы была успешно решена задача классификации 20 авторов. Применена балансировка данных и токенизация с расширенным словарем. Несмотря на то, что цель в 95% для BoW не была достигнута в полной мере (из-за усложнения задачи до 20 классов и близости стилей некоторых авторов), все 20 классов были определены верно (наибольшая вероятность всегда указывала на истинного автора).